

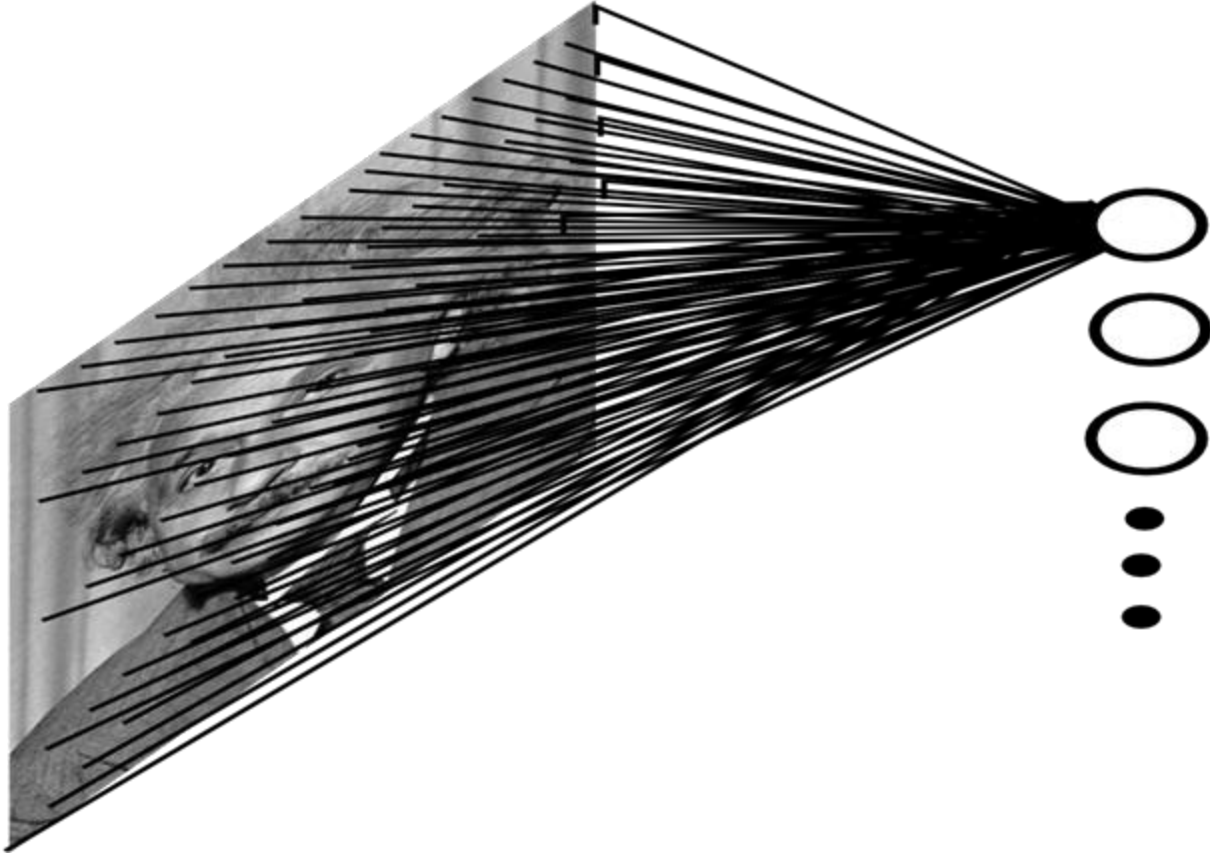
Arquiteturas de Deep Learning

REDES NEURAIS CONVOLUCIONAIS

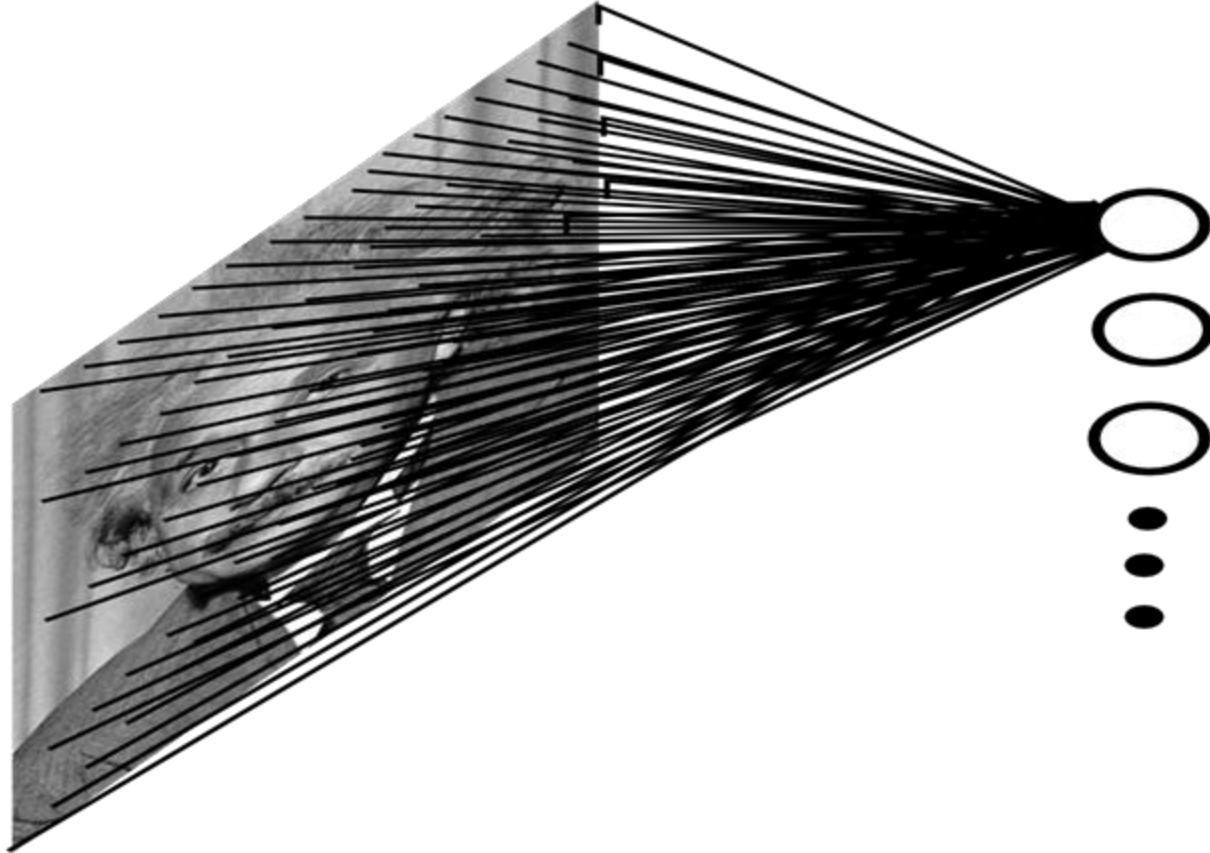
Zenilton K. G. Patrocínio Jr

zenilton@pucminas.br

Camada Completamente Conectada em Imagens



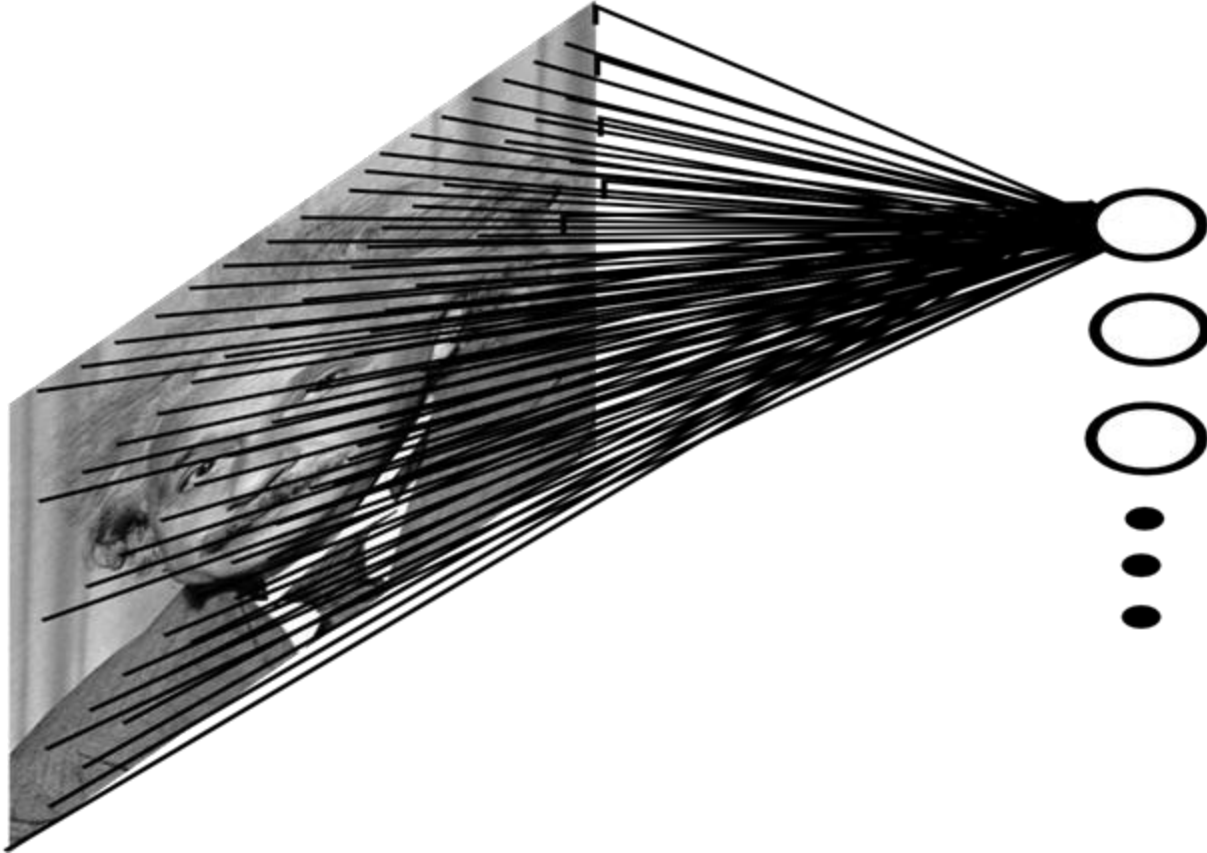
Camada Completamente Conectada em Imagens



Exemplo: Imagem 200×200

40K Neurônios

Camada Completamente Conectada em Imagens

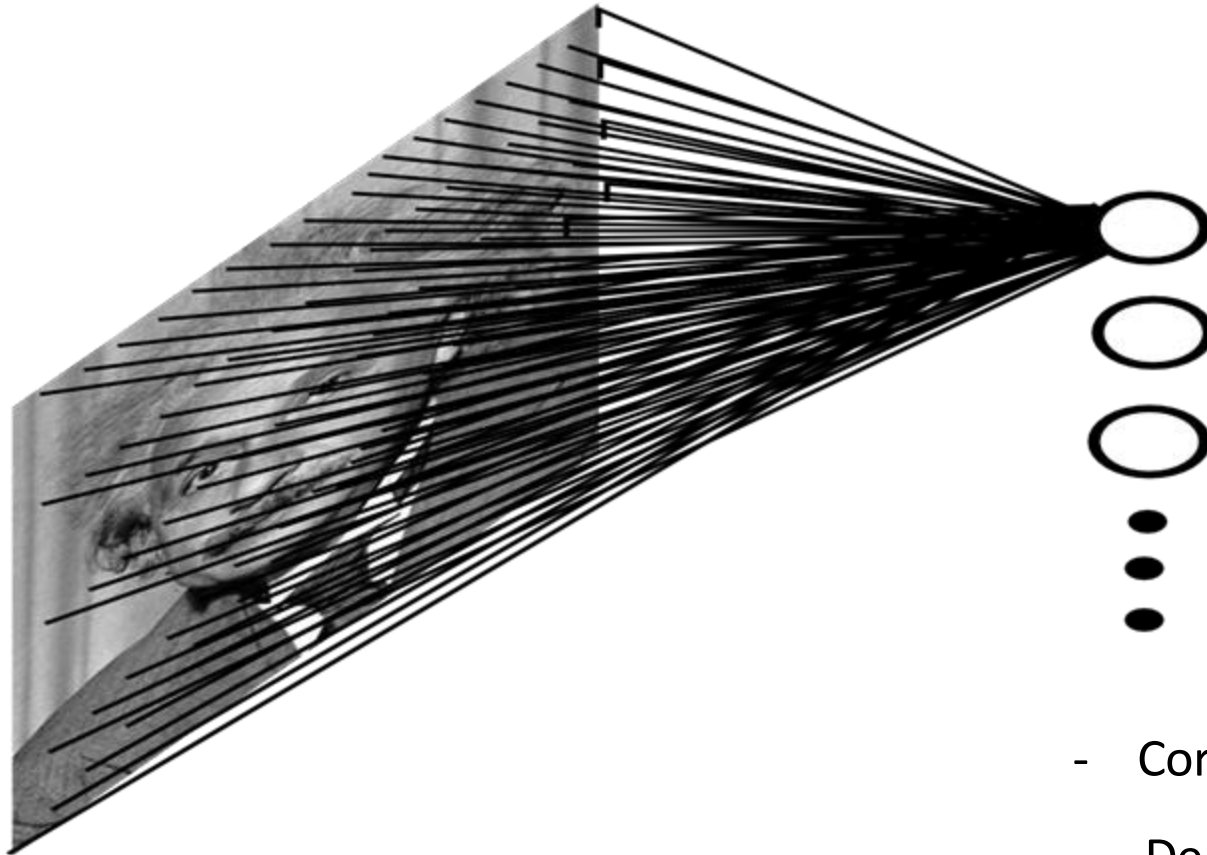


Exemplo: Imagem 200×200

40K Neurônios

➡ ~1,6B parâmetros !!!

Camada Completamente Conectada em Imagens



Exemplo: Imagem 200×200

40K Neurônios

➡ ~1,6B parâmetros !!!

- Correlação espacial é fenômeno local
- Desperdício de recursos + ausência de dados suficientes para treinamento . . .

Filtragem de Imagem

- Técnicas de filtragem podem ser utilizadas para extração de informação de interesse em uma determinada imagem

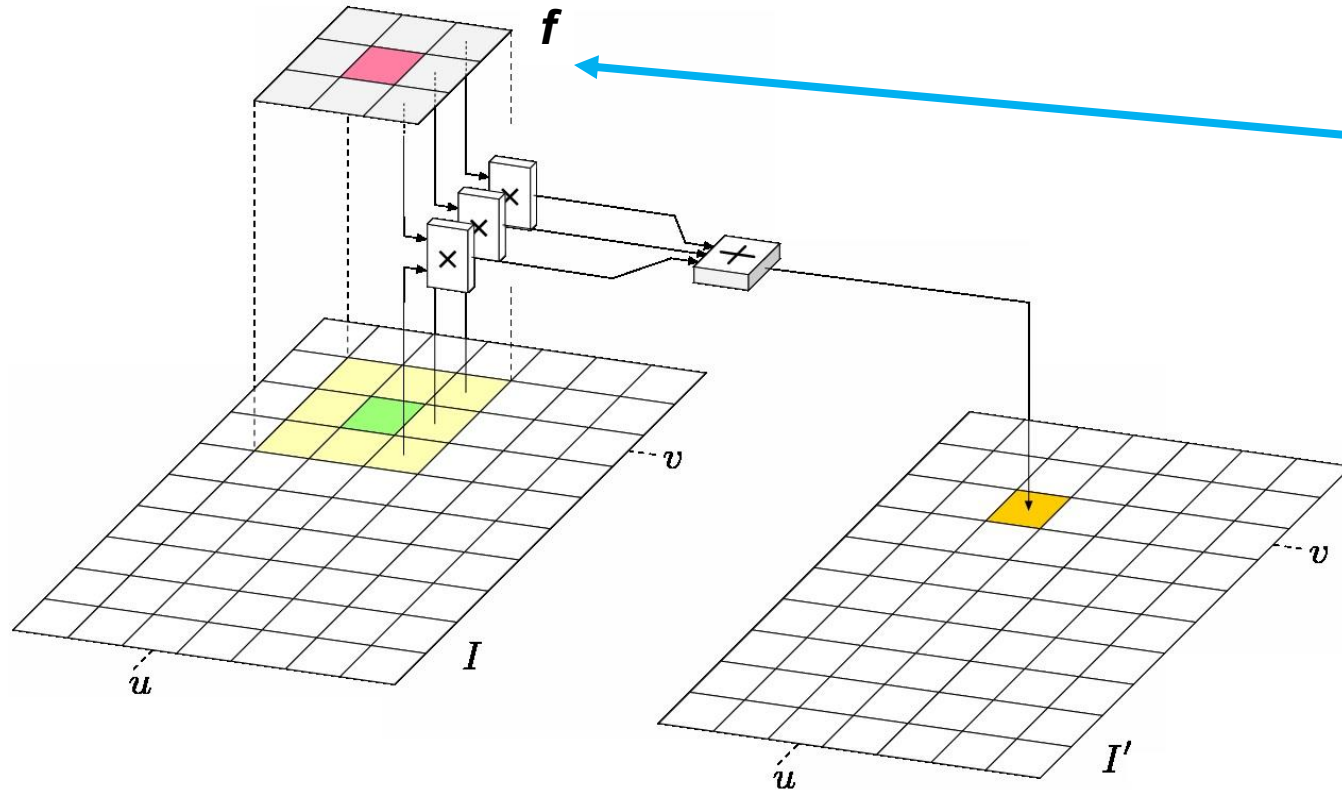
Filtragem de Imagem

- Técnicas de filtragem podem ser utilizadas para extração de informação de interesse em uma determinada imagem
- Filtragem de imagem pode ser feita:
 - No domínio do espaço
 - No domínio da frequência

Filtragem de Imagem

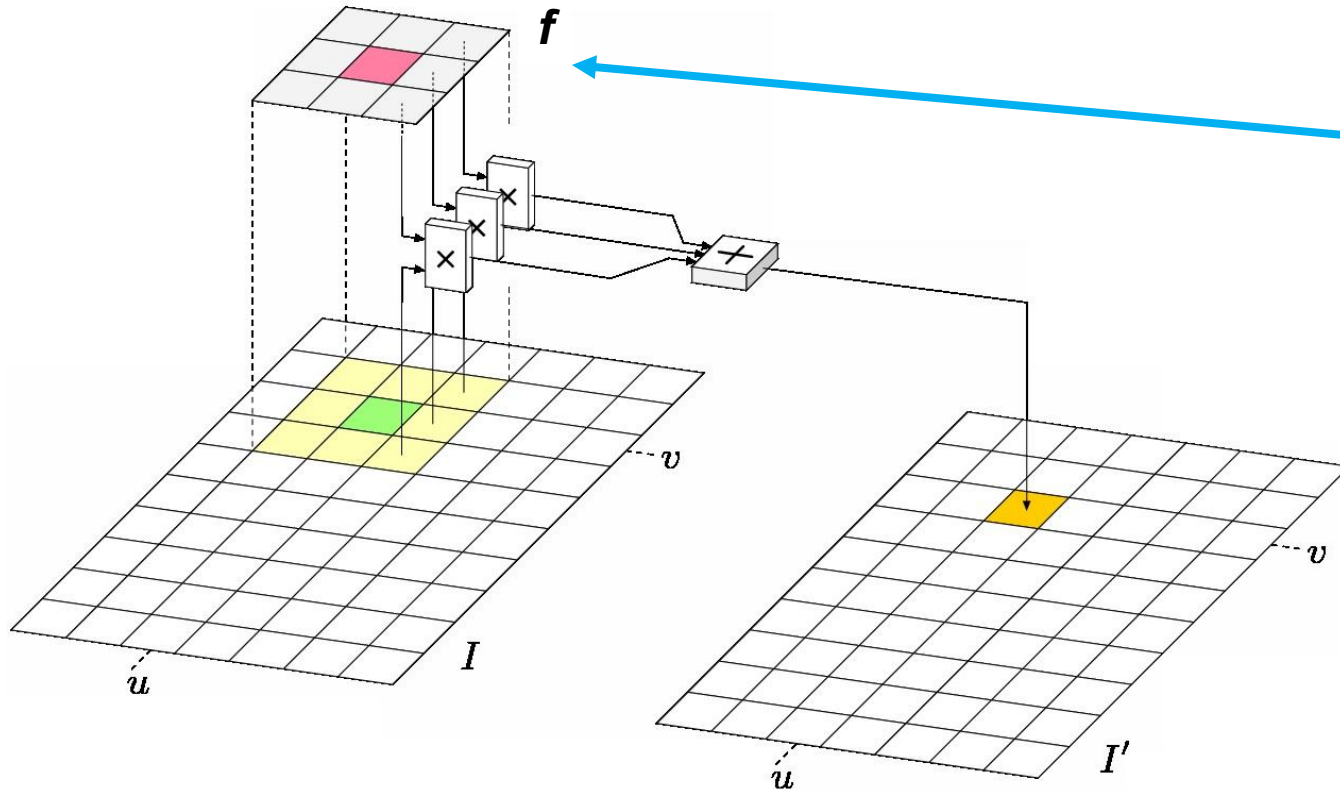
- Técnicas de filtragem podem ser utilizadas para extração de informação de interesse em uma determinada imagem
- Filtragem de imagem pode ser feita:
 - No domínio do espaço
 - No domínio da frequência
- **Filtros Espaciais Lineares** calculam o valor resultante de cada pixel como uma combinação linear dos valores de pixels em uma vizinhança local do pixel na imagem original

Filtragem de Imagem



O processo de filtragem normalmente é realizado por meio de matrizes denominadas **máscaras (ou filtros)** em que cada posição está associado um valor numérico chamado peso ou coeficiente

Filtragem de Imagem



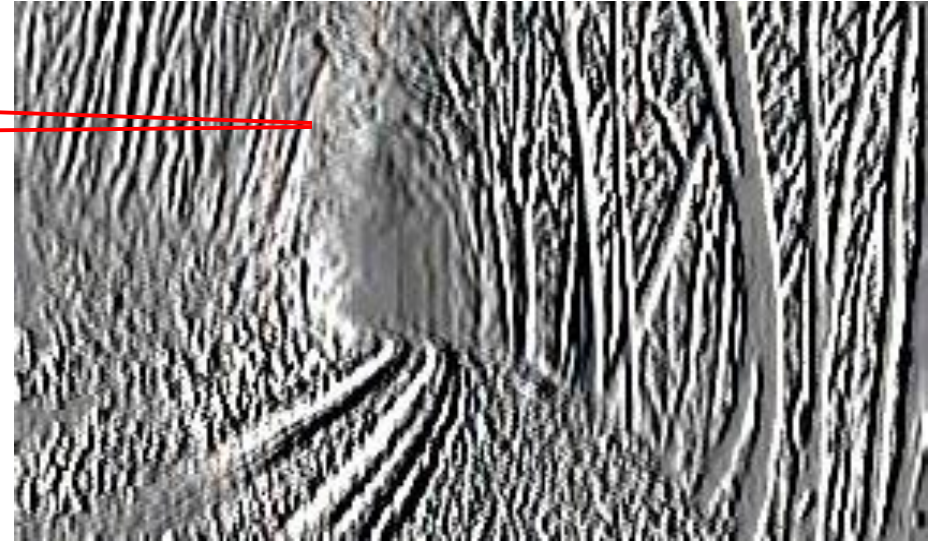
O processo de filtragem normalmente é realizado por meio de matrizes denominadas **máscaras (ou filtros)** em que cada posição está associado um valor numérico chamado peso ou coeficiente

Os coeficientes do filtro são multiplicados pelos valores dos pixels correspondentes e então somados, substituindo o valor do pixel central

Filtragem de Imagem → Convolução



$$* \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} =$$



$$(f * g)(x, y) = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} f(i, j) g(x - i, y - j)$$

Filtragem de Imagem → Convolução

$$f[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[\cdot, \cdot]$

Filtragem de Imagem → Convolução

$$f[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$h = f * g$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$g[\cdot, \cdot]$$

Filtragem de Imagem → Convolução

$$f[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$h = f * g$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[\cdot, \cdot]$

$h[\cdot, \cdot]$

Filtragem de Imagem → Convolução

$$f[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$h = f * g$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[\cdot, \cdot]$

$h[\cdot, \cdot]$

Filtragem de Imagem → Convolução

$$f[\cdot,\cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$h = f * g$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[\cdot,\cdot]$

	0								

$h[\cdot,\cdot]$

Filtragem de Imagem → Convolução

$$f[\cdot,\cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$h = f * g$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[\cdot,\cdot]$

	0	10							

$h[\cdot,\cdot]$

Filtragem de Imagem → Convolução

$$f[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$h = f * g$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[\cdot, \cdot]$

	0	10	20						

$h[\cdot, \cdot]$

Filtragem de Imagem → Convolução

$$f[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$h = f * g$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[\cdot, \cdot]$

	0	10	20	30					

$h[\cdot, \cdot]$

Filtragem de Imagem → Convolução

$$f[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$h = f * g$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[\cdot, \cdot]$

	0	10	20	30	30				

$h[\cdot, \cdot]$

Filtragem de Imagem → Convolução

$$f[\cdot,\cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$h = f * g$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[\cdot,\cdot]$

	0	10	20	30	30				

$h[\cdot,\cdot]$

Filtragem de Imagem → Convolução

$$f[\cdot,\cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$h = f * g$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[\cdot,\cdot]$

	0	10	20	30	30				

$h[\cdot,\cdot]$

Filtragem de Imagem → Convolução

$$f[\cdot,\cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$h = f * g$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[\cdot,\cdot]$

	0	10	20	30	30				
							?		
				50					

$h[\cdot,\cdot]$

Filtragem de Imagem → Convolução

$$f[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$h = f * g$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[\cdot, \cdot]$

	0	10	20	30	30				
						90			
				50					

$h[\cdot, \cdot]$

Filtragem de Imagem → Convolução

$$f[\cdot, \cdot] = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

$$h = f * g$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$g[\cdot, \cdot]$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

$h[\cdot, \cdot]$

Filtros Lineares – Exemplos



Original

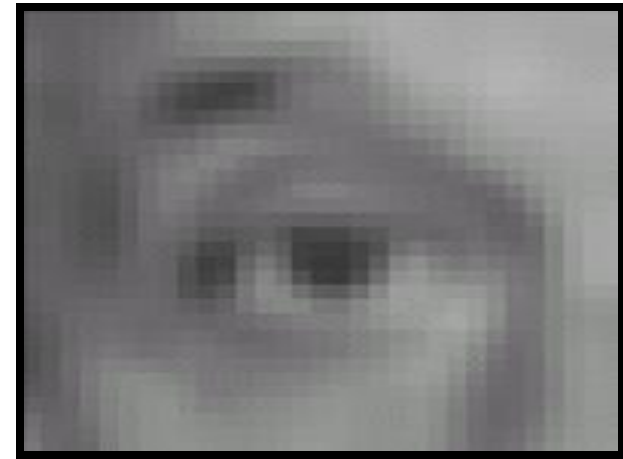
$$* \frac{1}{9} \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline \end{array} = ?$$

Filtros Lineares – Exemplos



Original

$$* \frac{1}{9} \begin{array}{|c|c|c|} \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline 1 & 1 & 1 \\ \hline \end{array} =$$



Suavização (com
filtro da média)

Filtros Lineares – Exemplos



Original

*

0	0	0
0	1	0
0	0	0

=

?

Filtros Lineares – Exemplos



Original

*

0	0	0
0	1	0
0	0	0

=



Identidade (sem
filtragem)

Filtros Lineares – Exemplos



Original

*

0	0	0
0	0	1
0	0	0

=

?

Filtros Lineares – Exemplos



Original

*

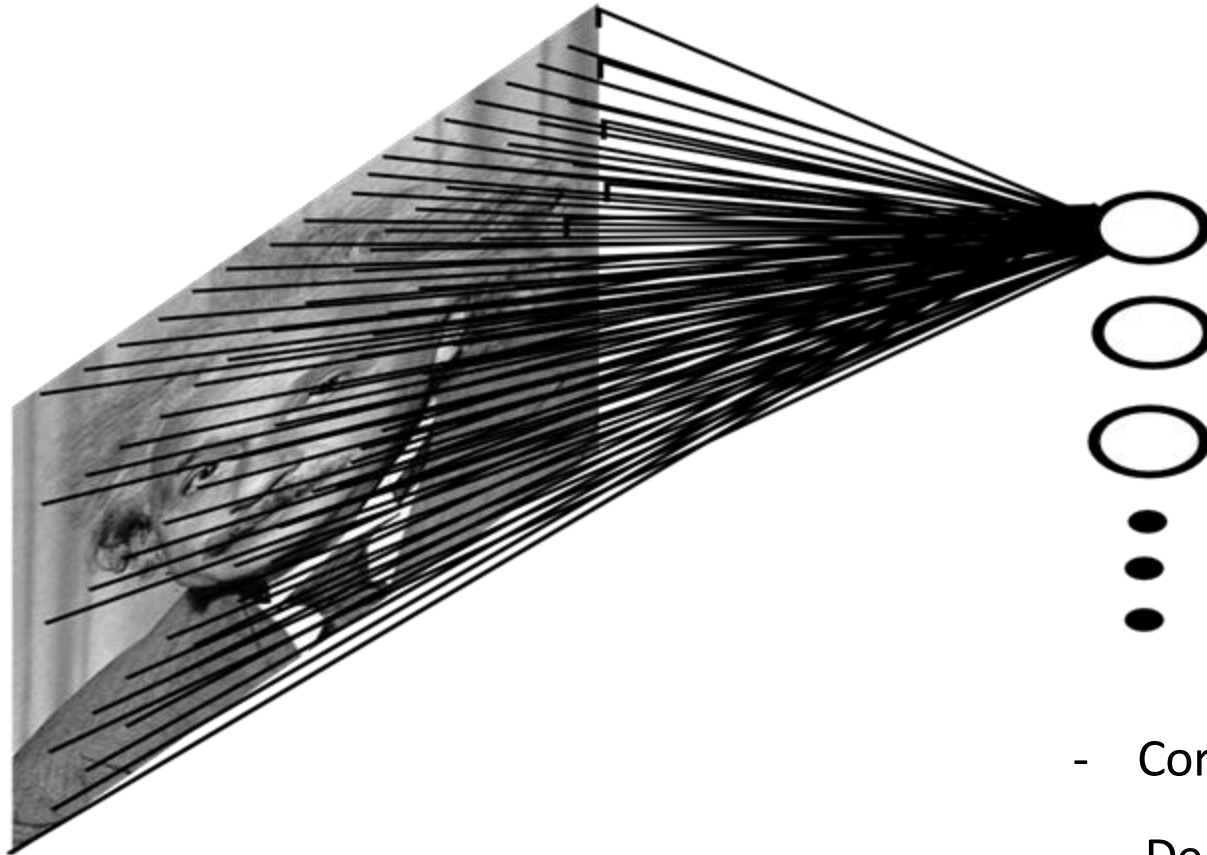
0	0	0
0	0	1
0	0	0

=



Deslocamento para
esquerda de um
pixel

Camada Completamente Conectada em Imagens



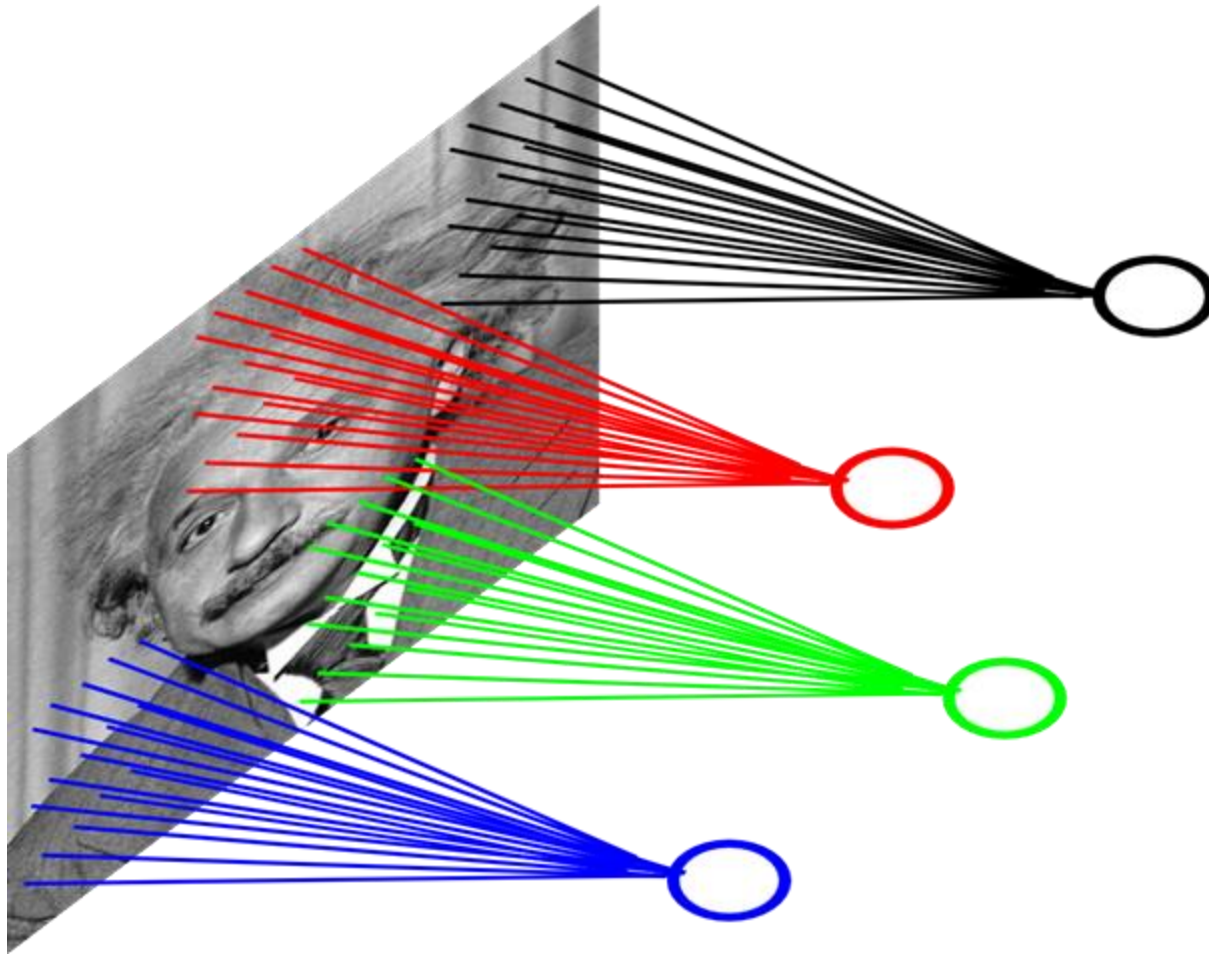
Exemplo: Imagem 200×200

40K Neurônios

➡ ~1,6B parâmetros !!!

- Correlação espacial é fenômeno local
- Desperdício de recursos + ausência de dados suficientes para treinamento . . .

Camada Conectada Localmente em Imagens

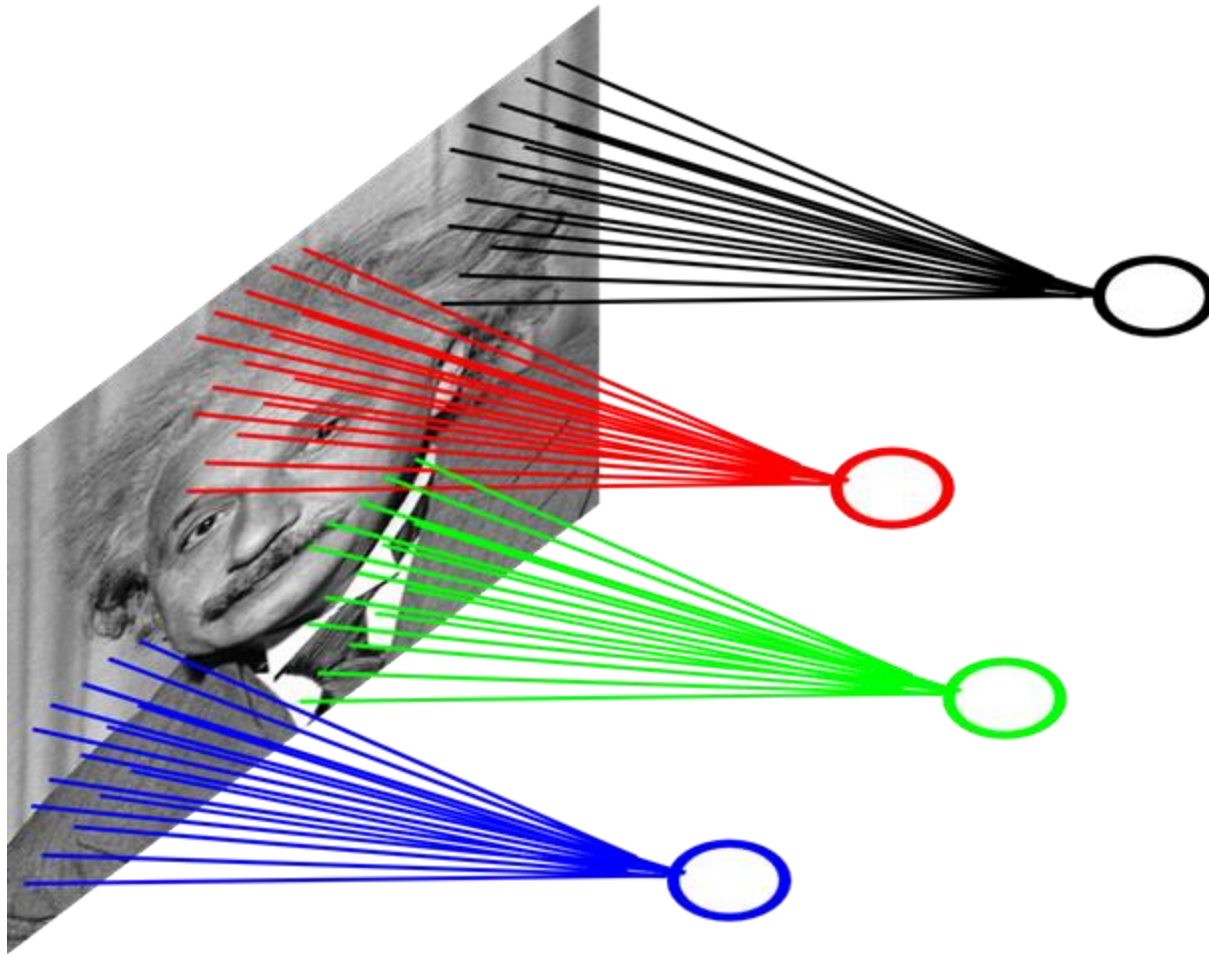


Exemplo: Imagem 200×200

40K Neurônios

Filtro 10×10

Camada Conectada Localmente em Imagens



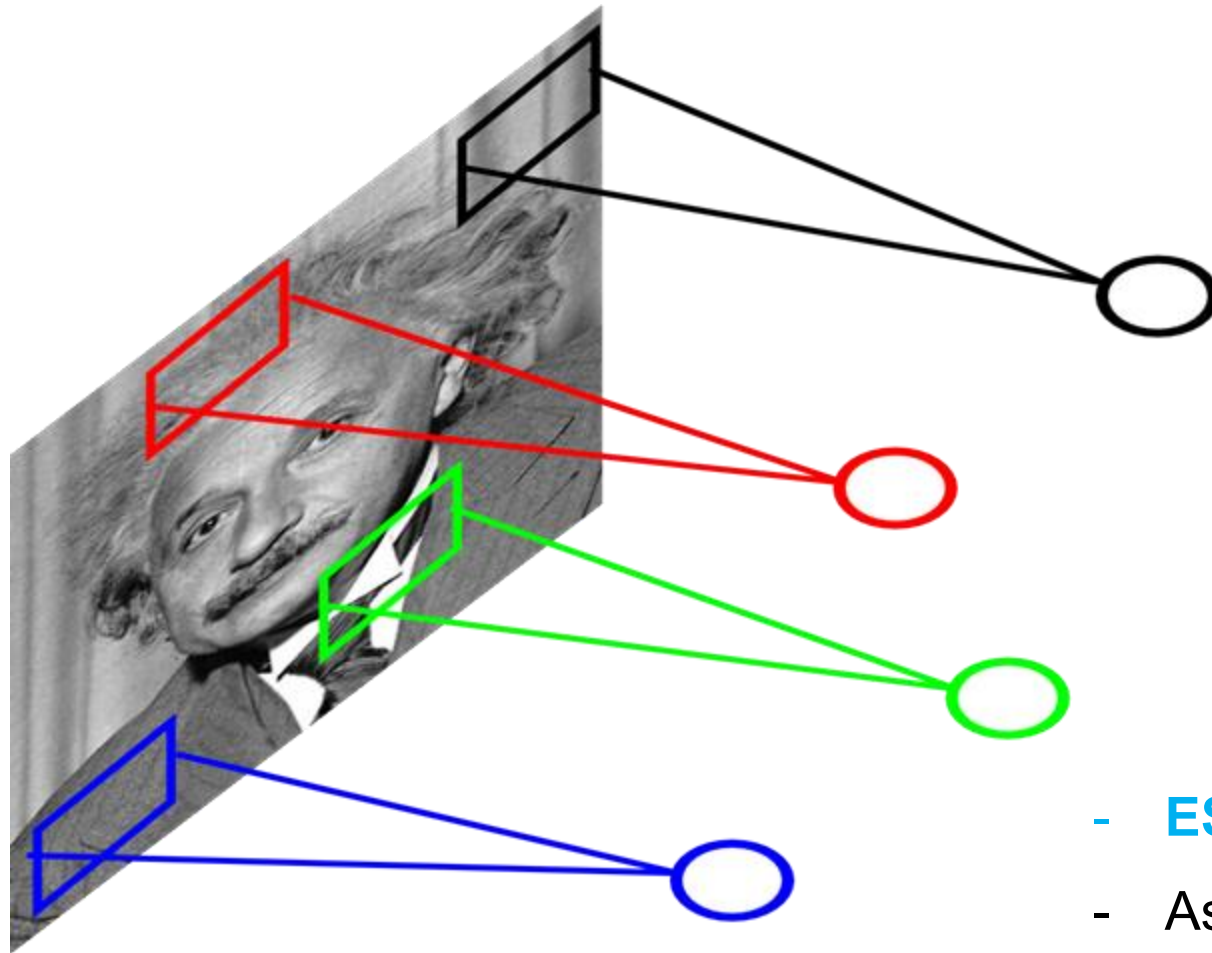
Exemplo: Imagem 200×200

40K Neurônios

Filtro 10×10

➡ ~4M parâmetros !!!

Camada Conectada Localmente em Imagens



Exemplo: Imagem 200×200

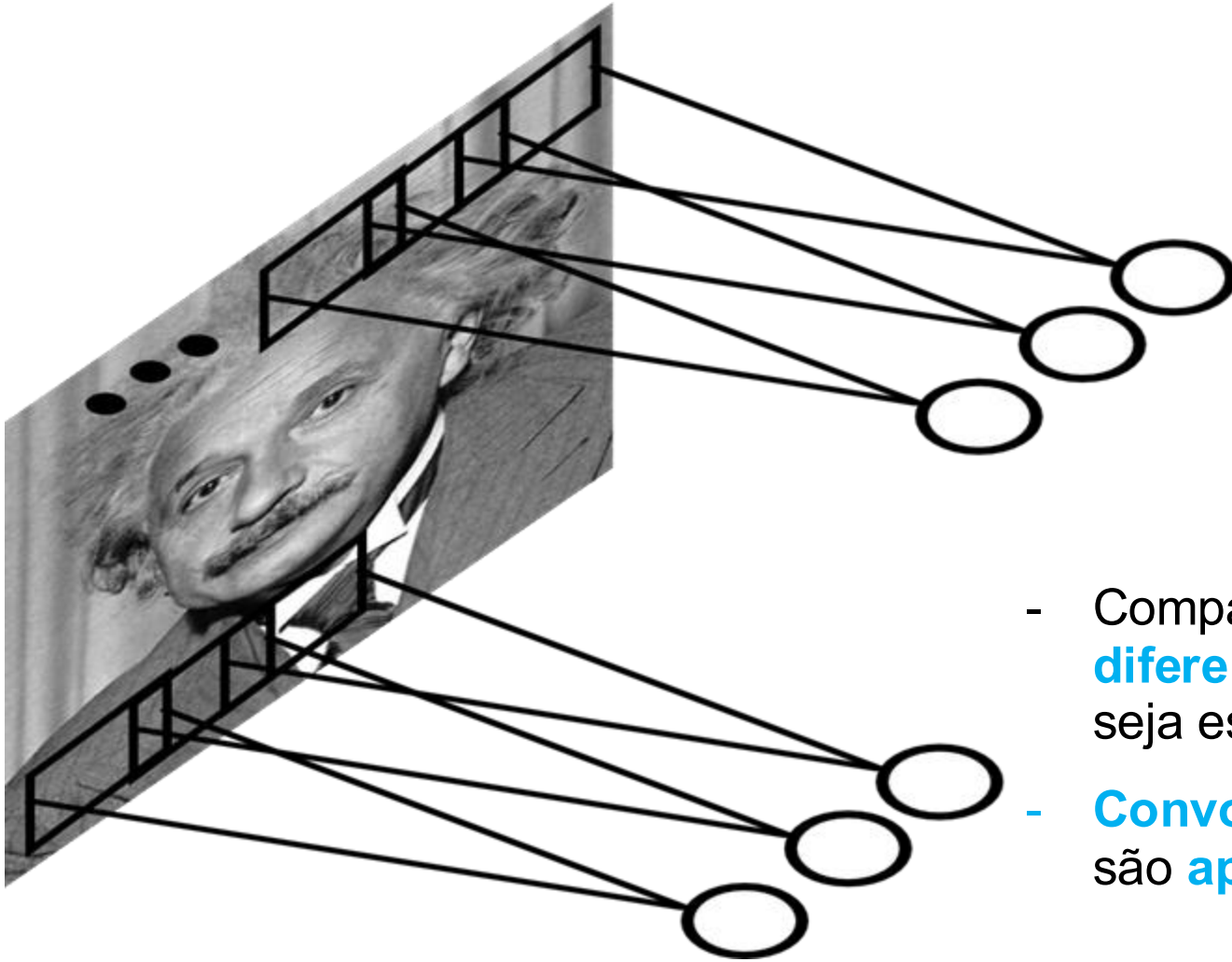
40K Neurônios

Filtro 10×10

➔ ~4M parâmetros !!!

- **ESTACIONARIEDADE** . . .
- As estatísticas são semelhantes em locais diferentes

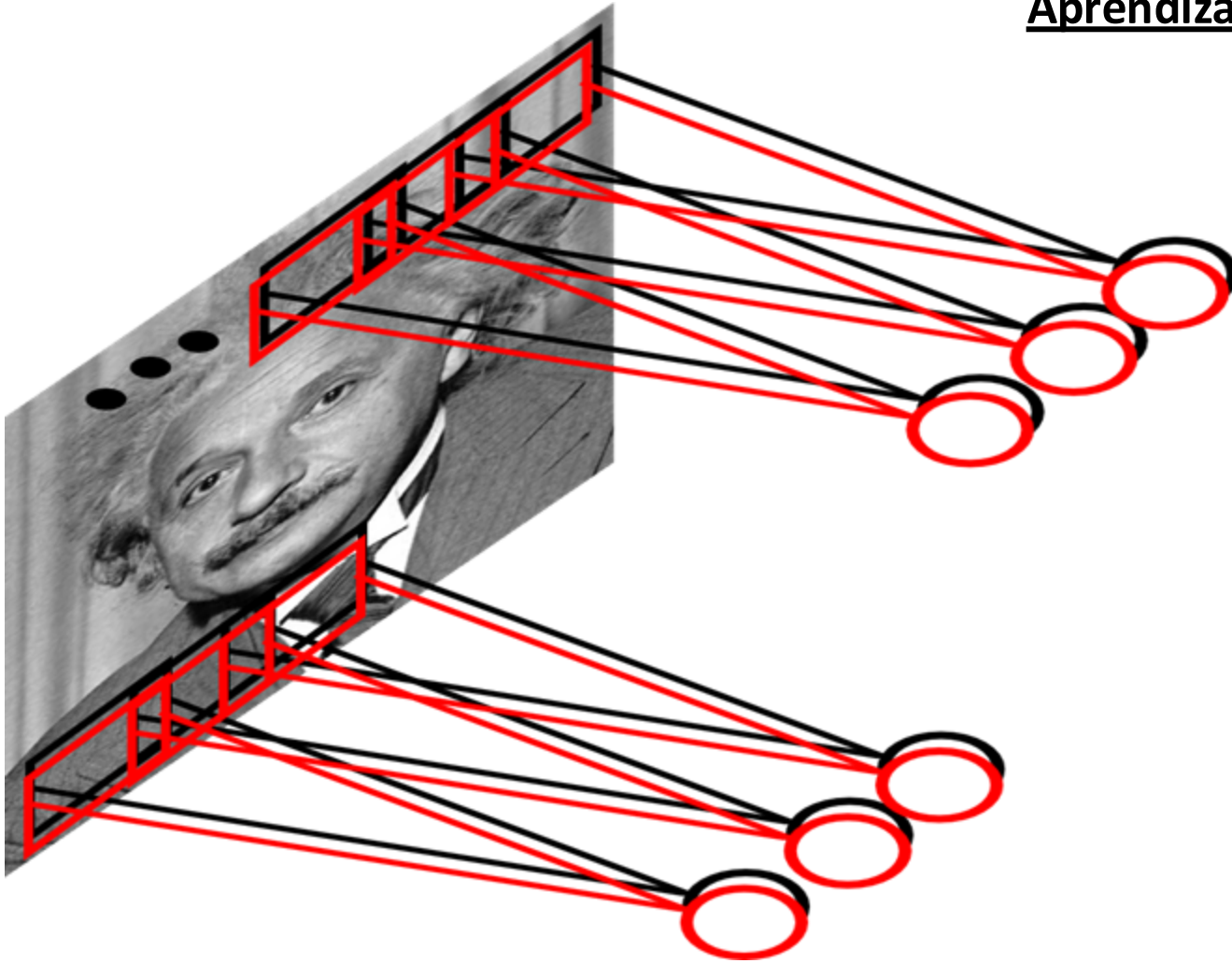
Camada Convolutiva



- Compartilhar os **mesmos parâmetros em diferentes locais** (supondo que a entrada seja estacionária)
- **Convoluções com filtros** (ou *kernels*) que são **aprendidos**

Camada Convolutucional

Aprendizado de **múltiplos filtros !!!**



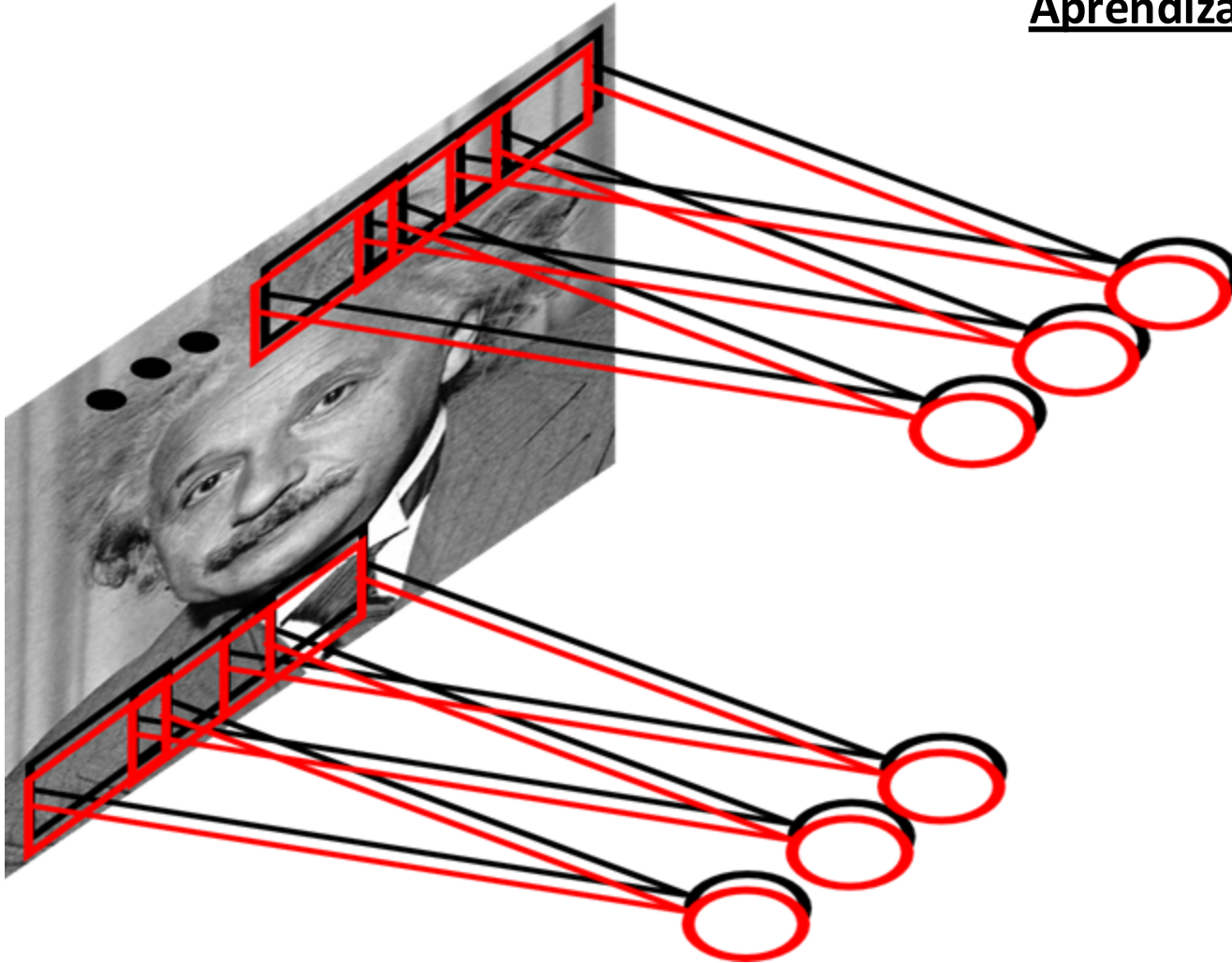
Camada Convolutiva

Aprendizado de **múltiplos filtros !!!**

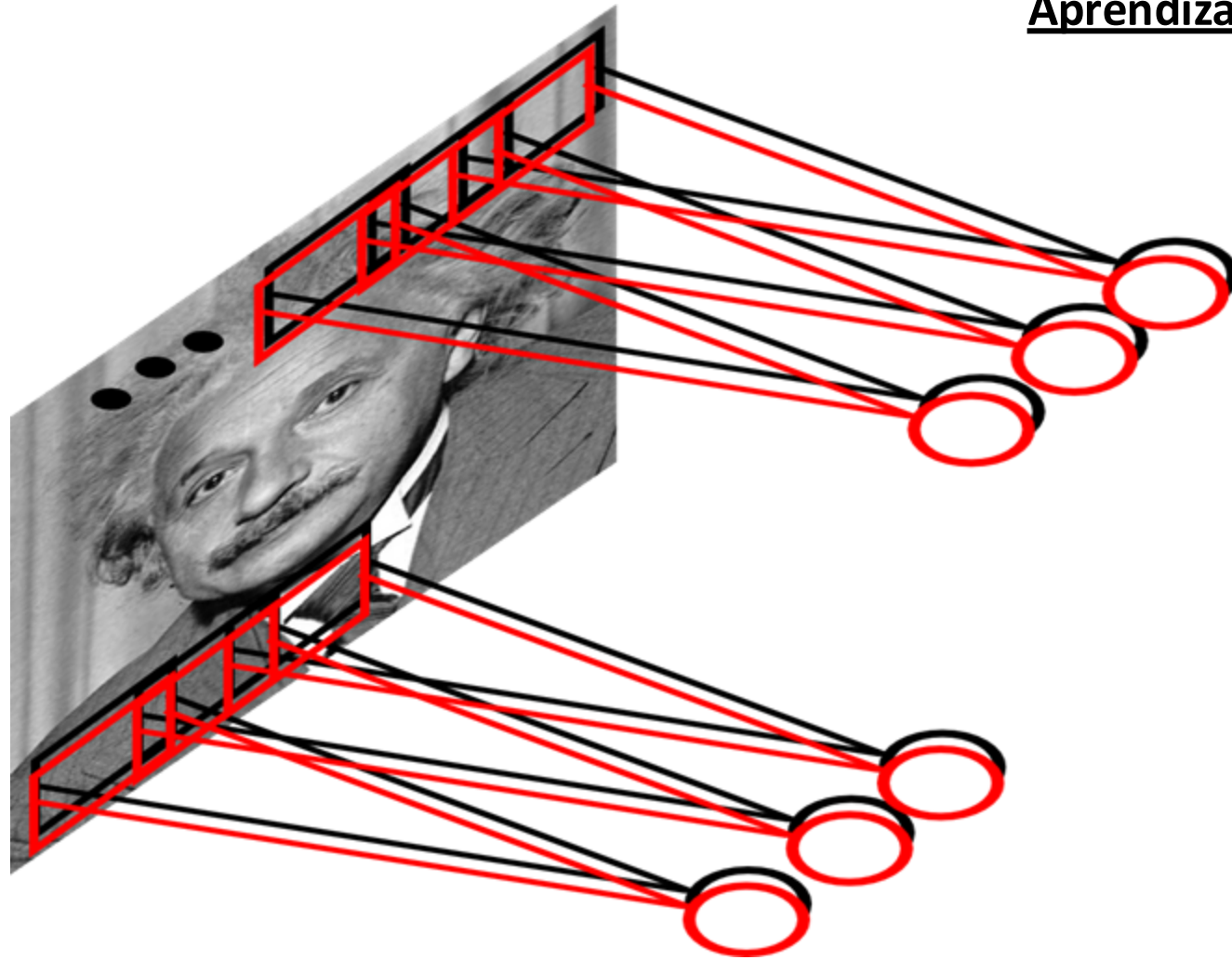
Exemplo: Imagem 200×200

100 Filtros

Filtro 10×10



Camada Convolutucional



Aprendizado de **múltiplos filtros !!!**

Exemplo: Imagem 200×200

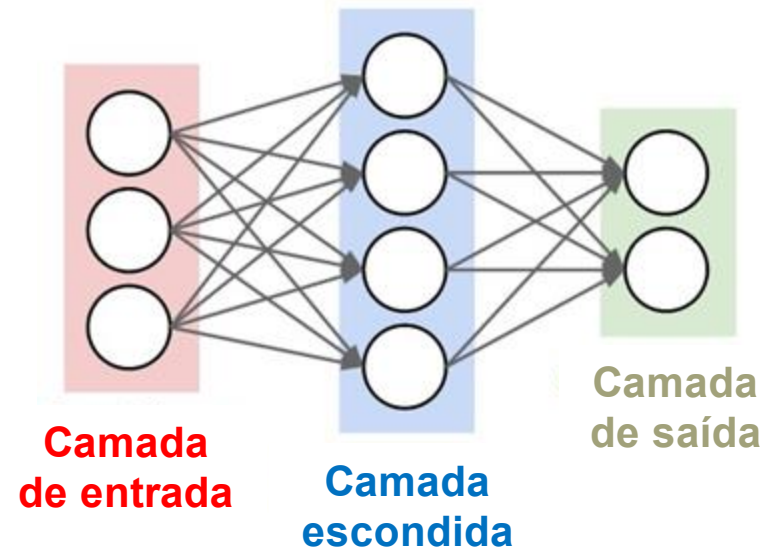
100 Filtros

Filtro 10×10

➡ **~10K parâmetros !!!**

Camada Convolutucional

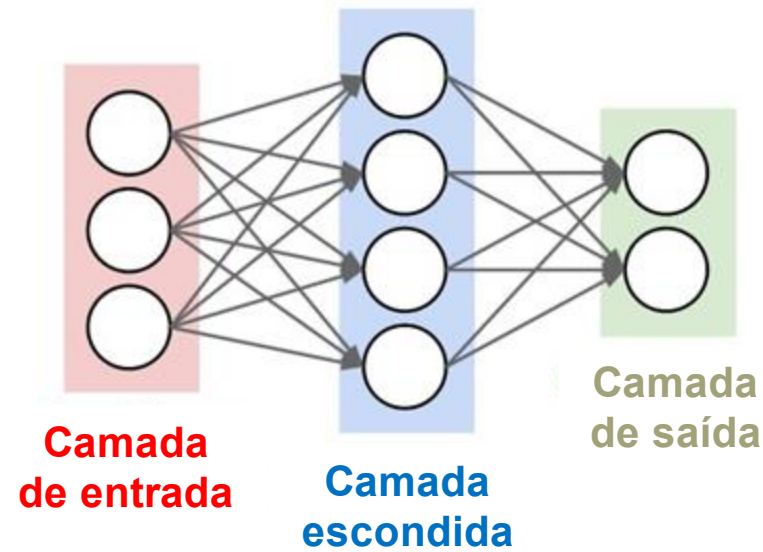
Antes:



Todos os filtros na mesma camada são os mesmos

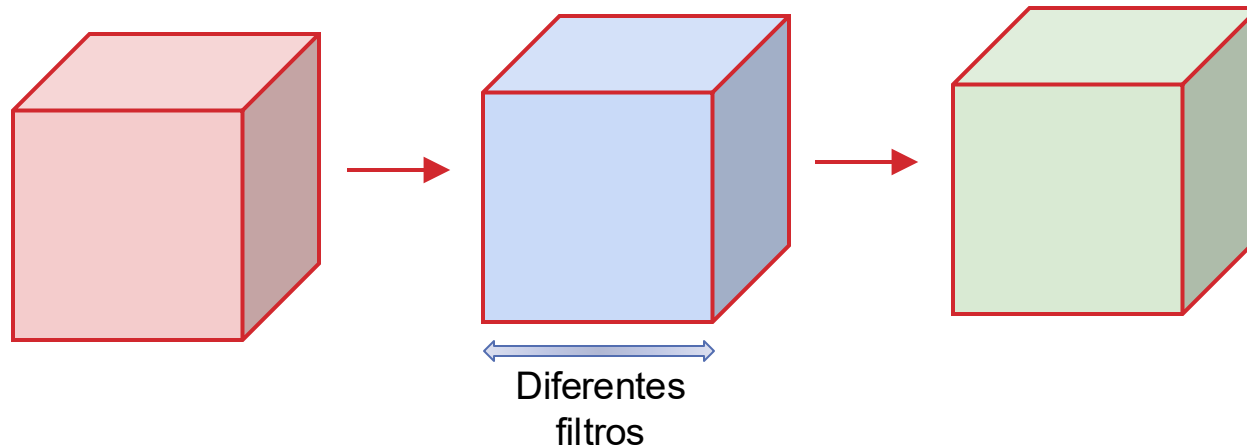
Camada Convolutucional

Antes:



Todos os filtros na mesma camada são os mesmos

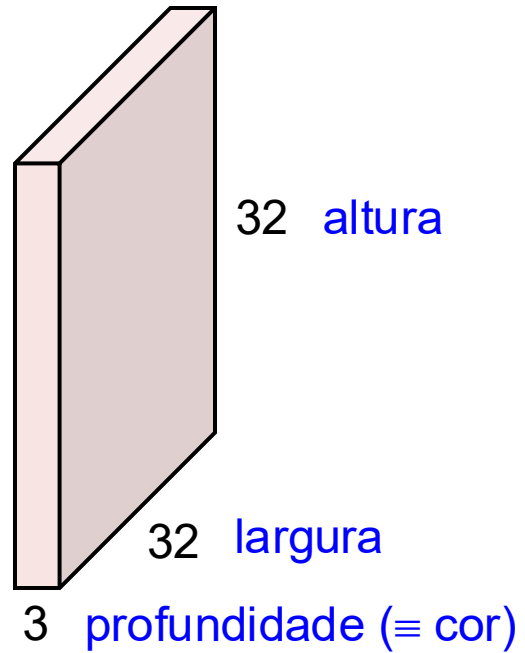
Agora:



A dimensão da profundidade representa filtros diferentes

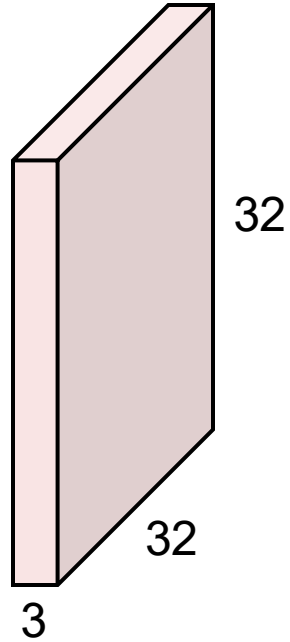
Camada Convolutacional

Imagem $32 \times 32 \times 3$



Camada Convolutacional

Imagem $32 \times 32 \times 3$

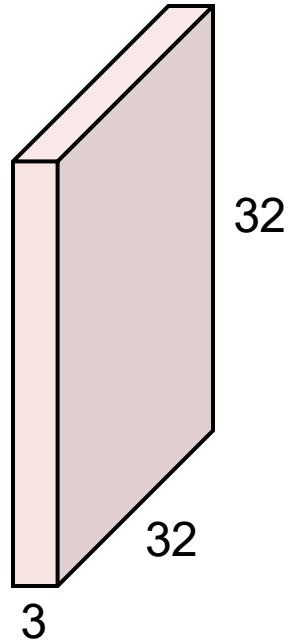


Filtro $5 \times 5 \times 3$



Camada Convolutacional

Imagem $32 \times 32 \times 3$

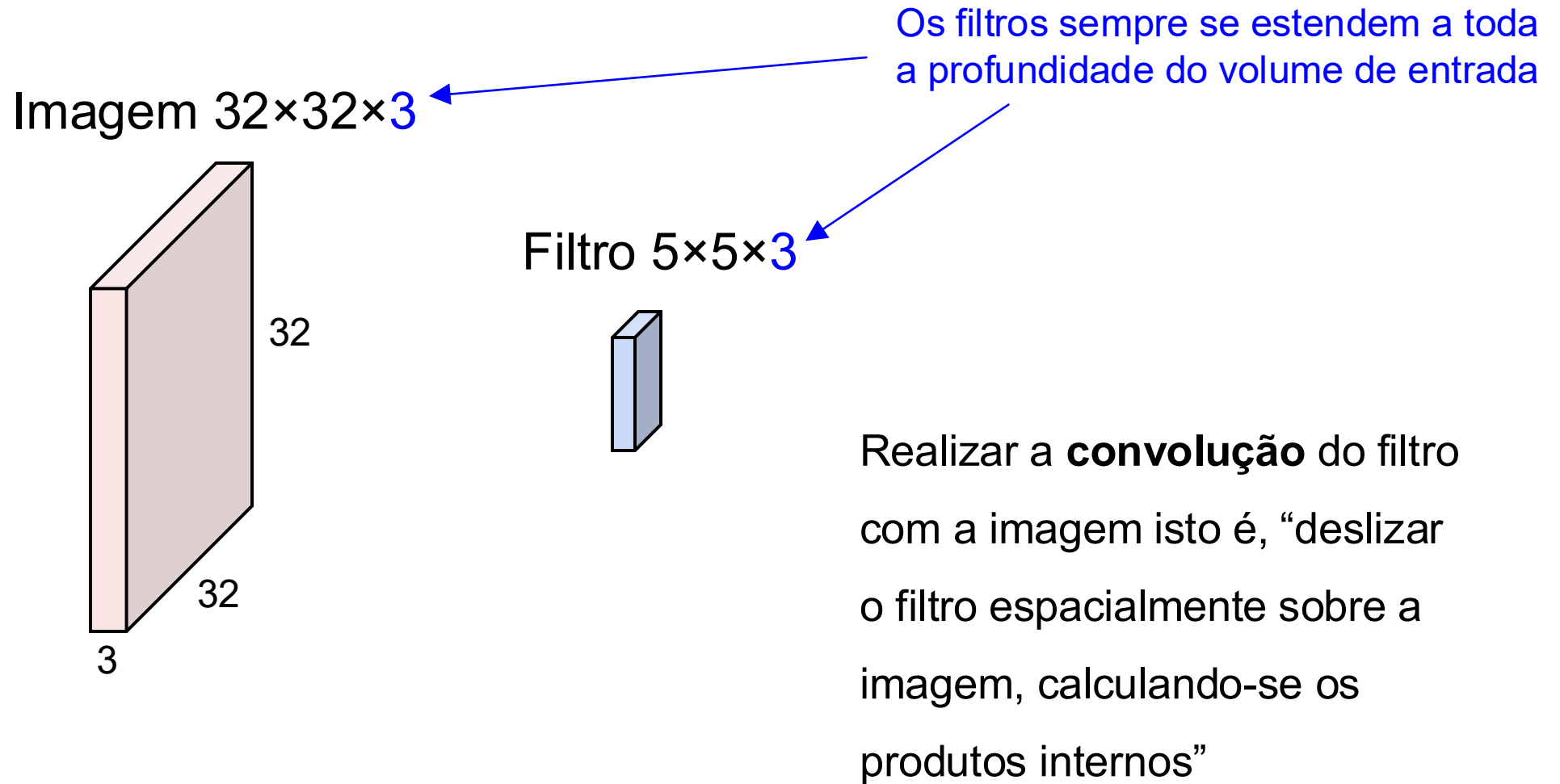


Filtro $5 \times 5 \times 3$

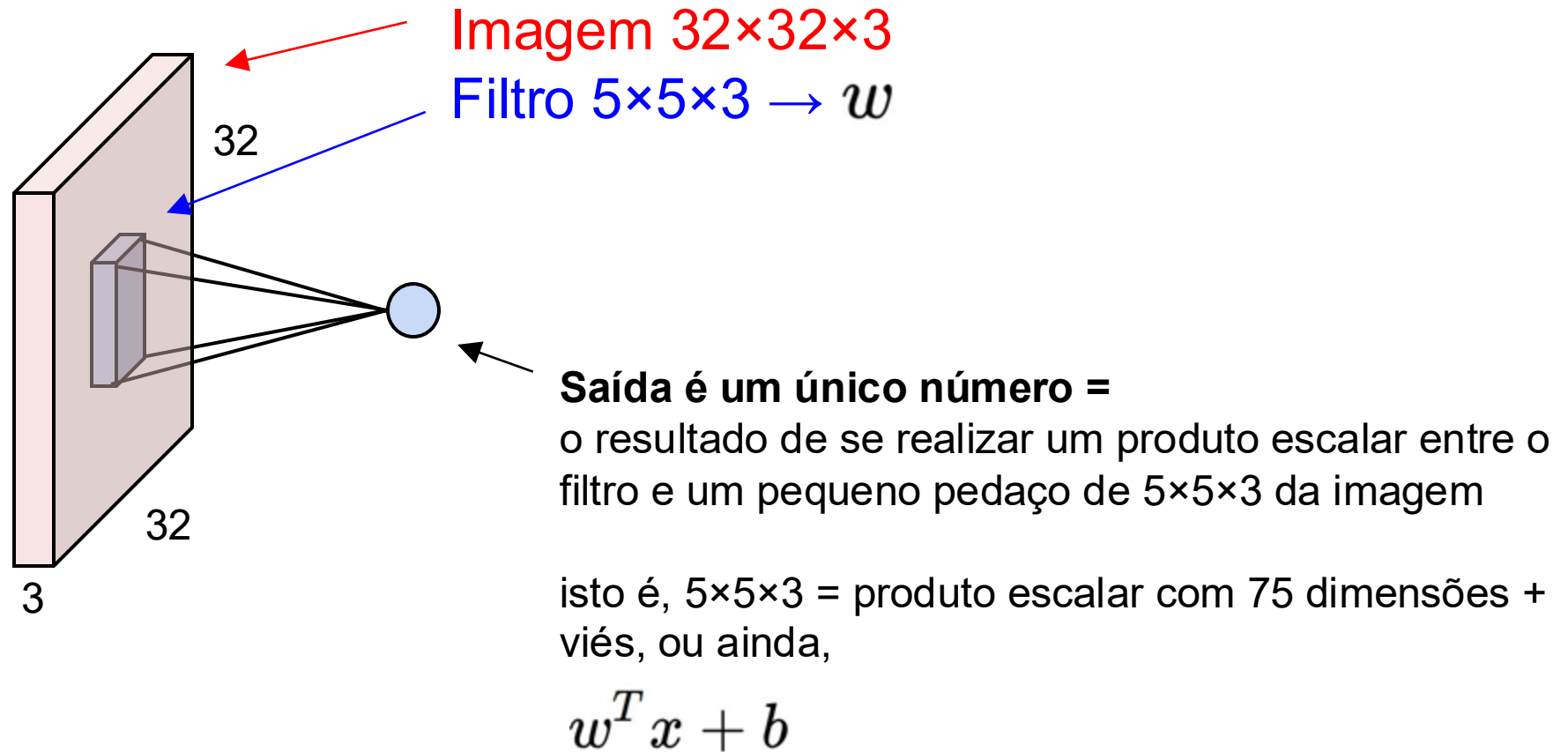


Realizar a **convolução** do filtro com a imagem isto é, “deslizar o filtro espacialmente sobre a imagem, calculando-se os produtos internos”

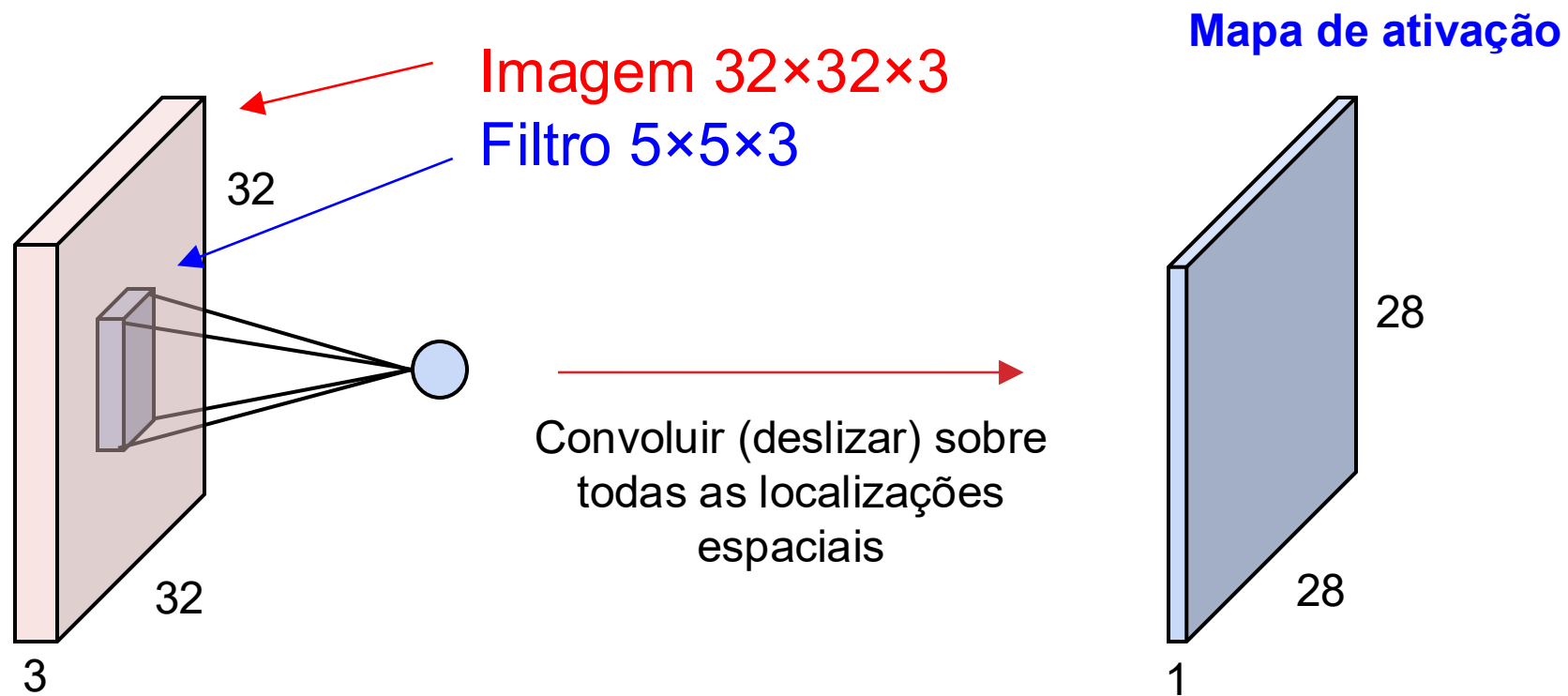
Camada Convolutiva



Camada Convolutiva

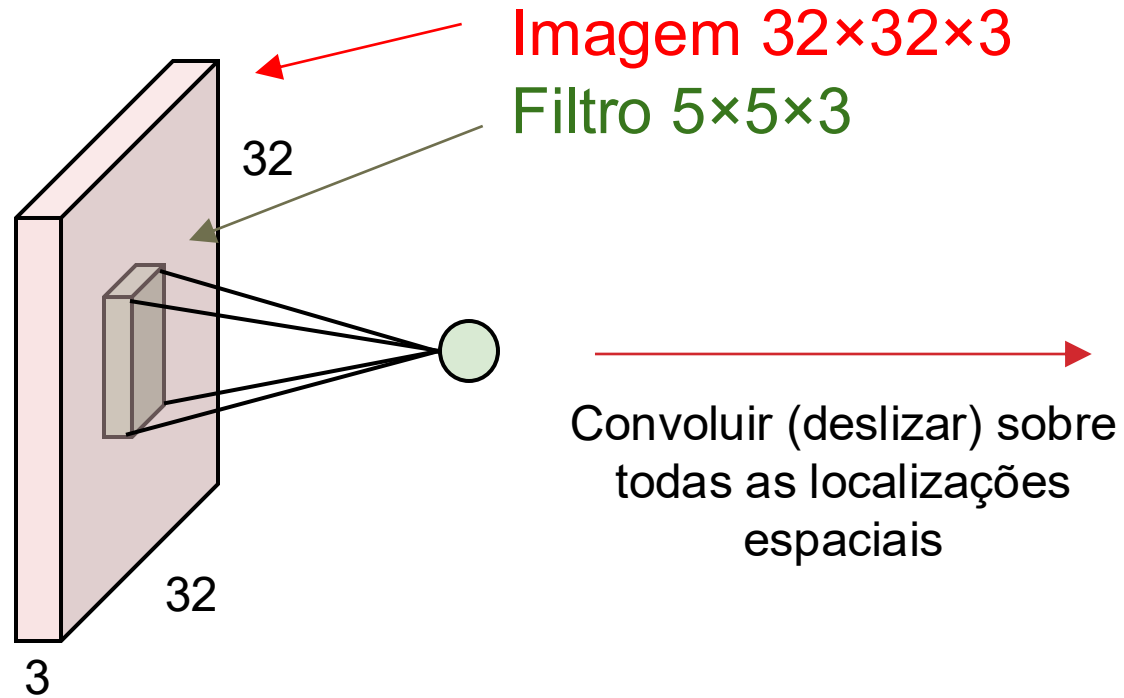


Camada Convolutiva

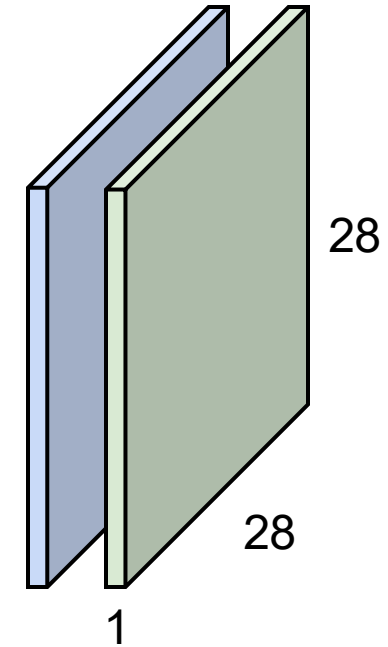


Camada Convolutiva

Considere um segundo filtro (em verde)

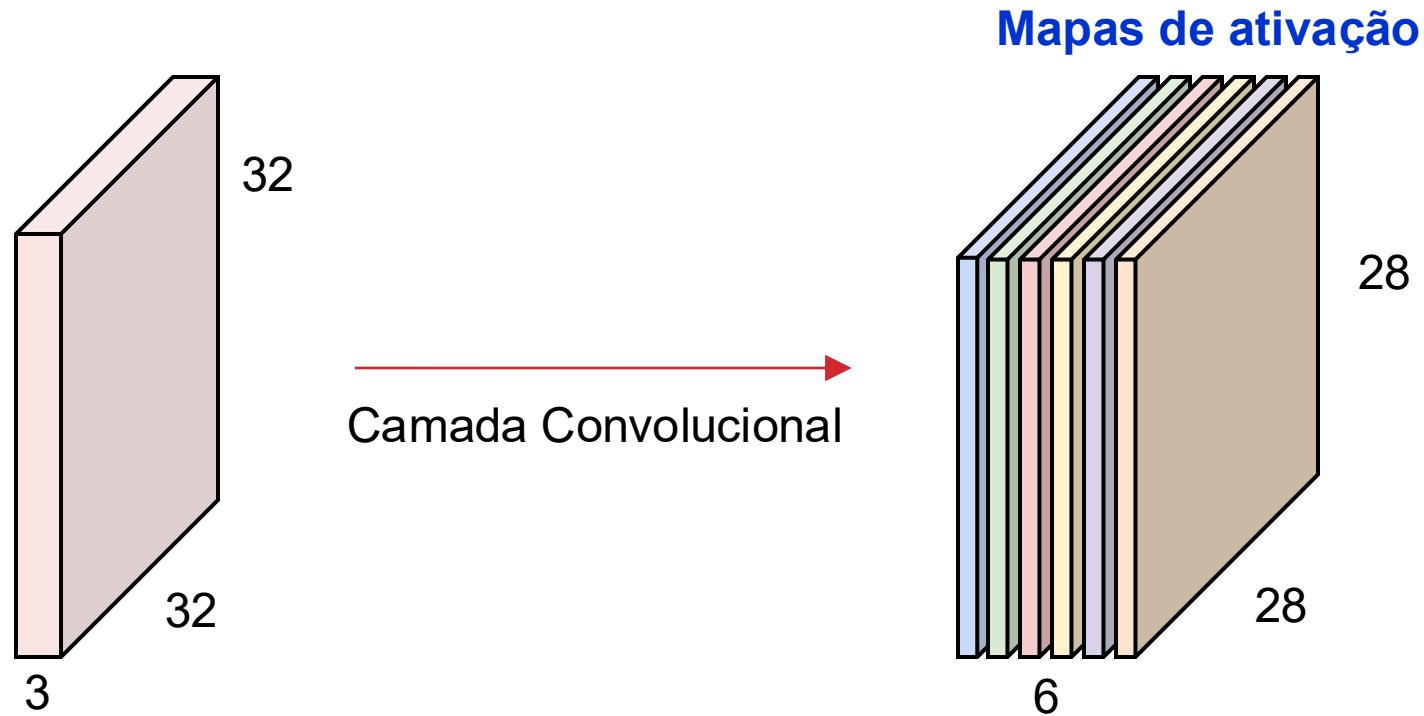


Mapas de ativação



Camada Convolutiva

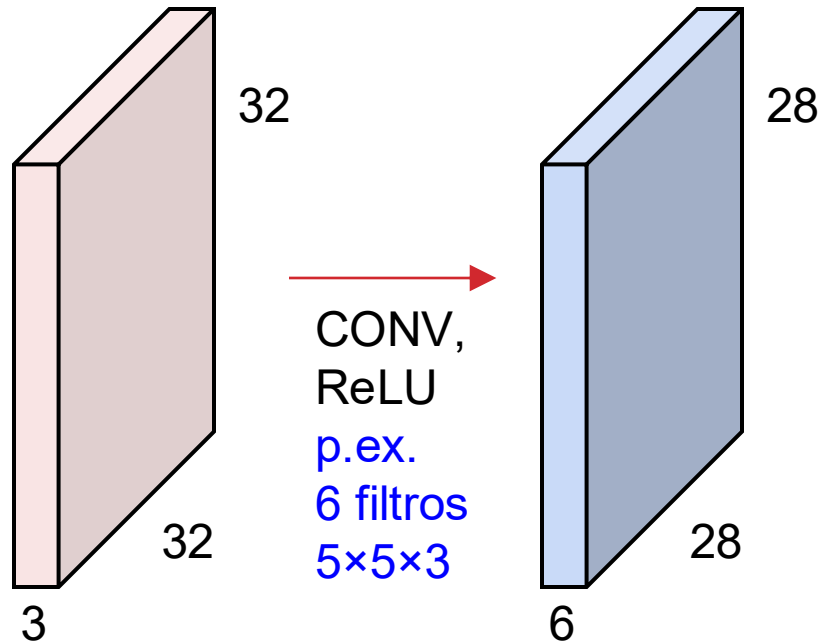
Por exemplo, se tivermos 6 filtros 5×5 , obtém-se 6 mapas de ativação distintos



Esses mapas podem ser agrupados (ou empilhados) de forma a produzir uma “nova imagem” de tamanho $28 \times 28 \times 6$!

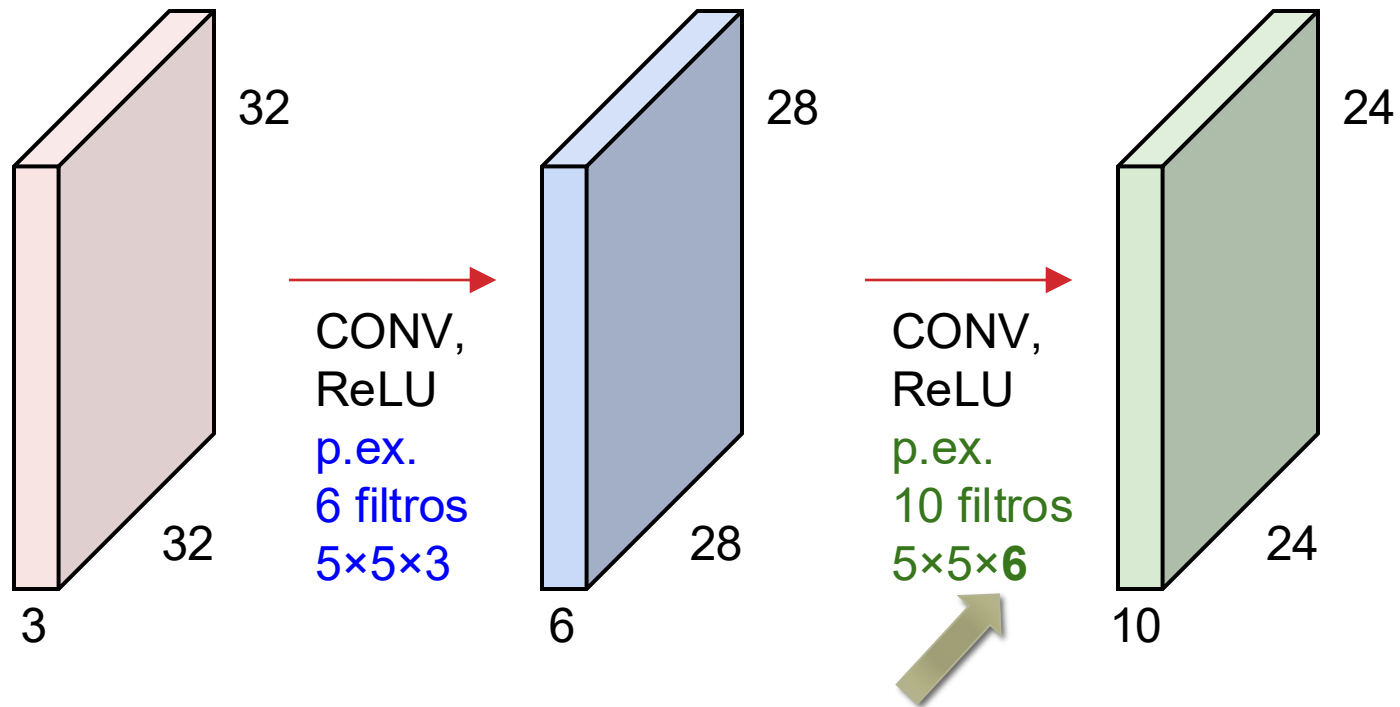
Camada Convolutiva

Uma rede convolucional (ConvNet) representa uma sequência de camadas convolucionais intercaladas com funções de ativação



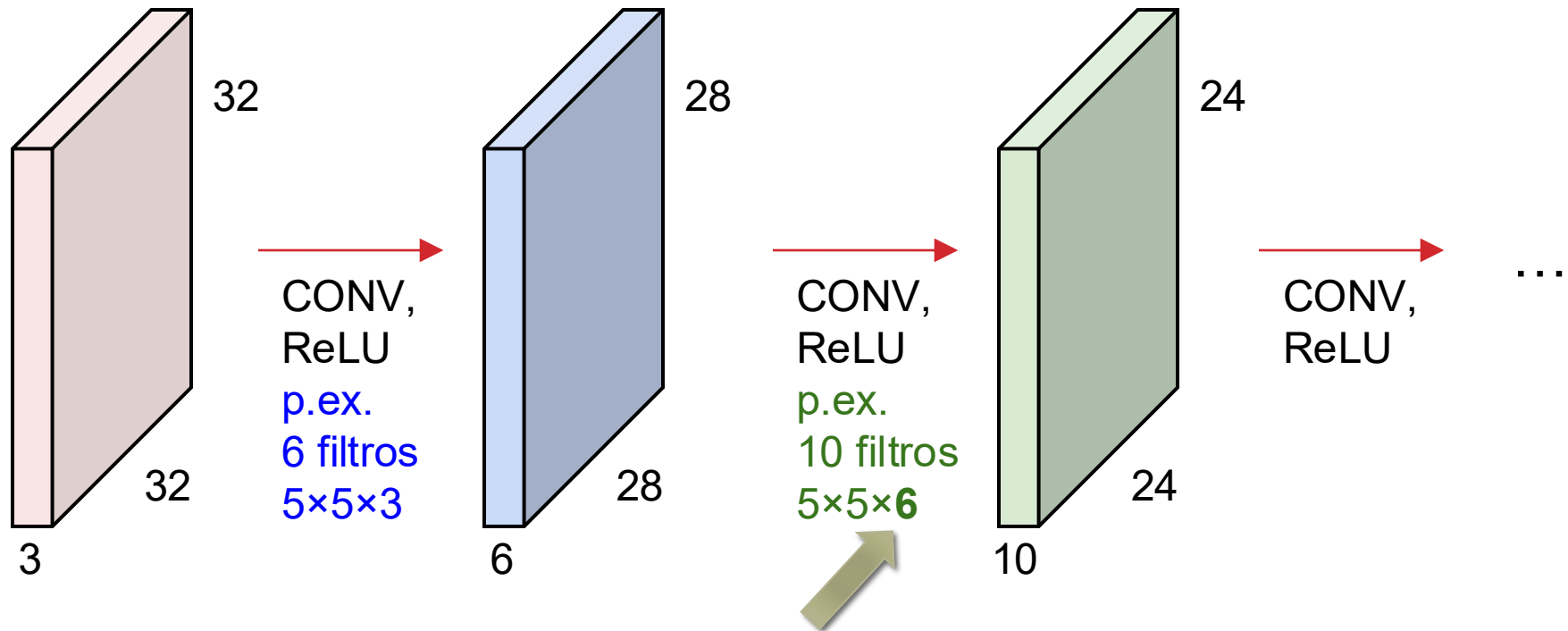
Camada Convolutiva

Uma rede convolucional (ConvNet) representa uma sequência de camadas convolucionais intercaladas com funções de ativação



Camada Convolucional

Uma rede convolucional (ConvNet) representa uma sequência de camadas convolucionais intercaladas com funções de ativação



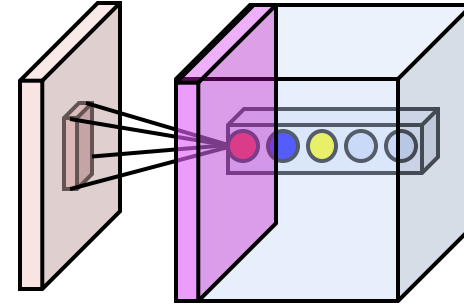
Camada Convolutiva

Entrada:



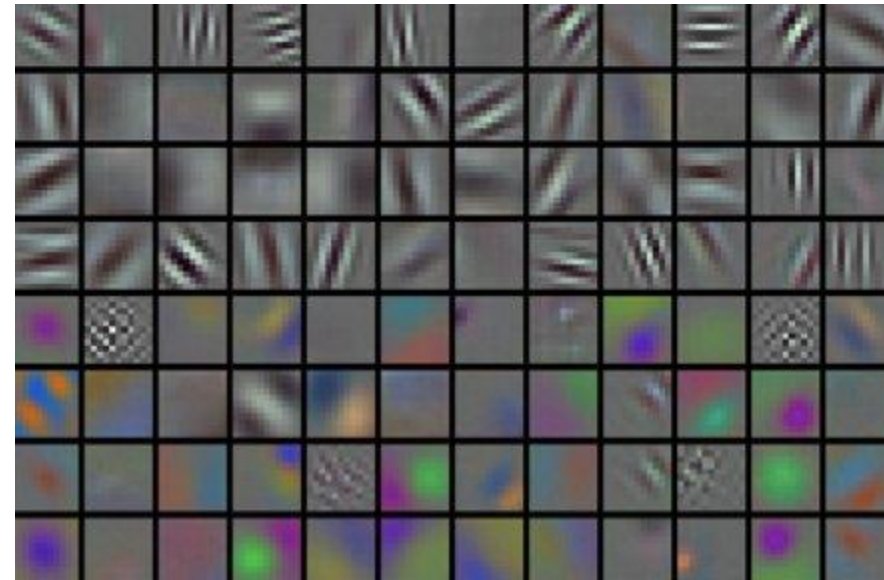
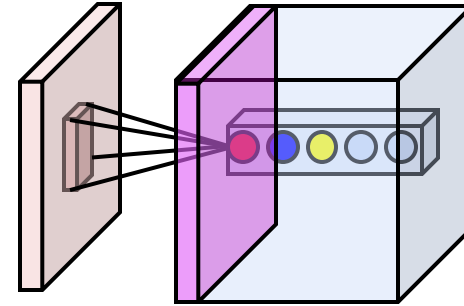
Camada Convolutiva

Entrada:



Camada Convolutucional

Entrada:

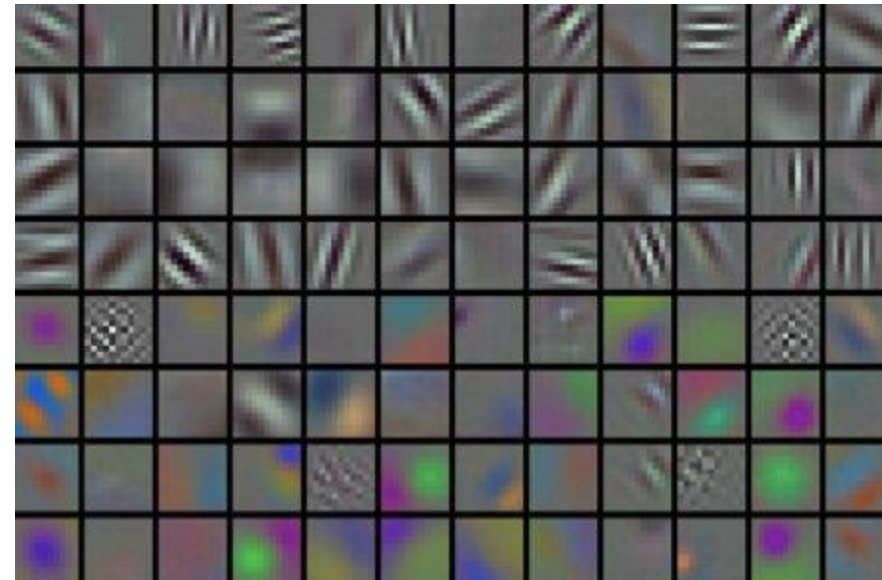
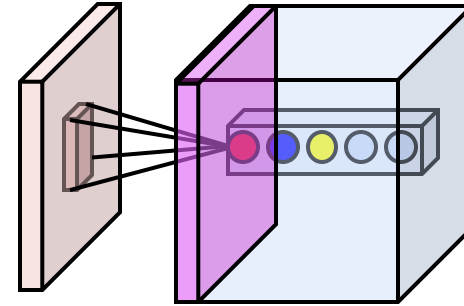
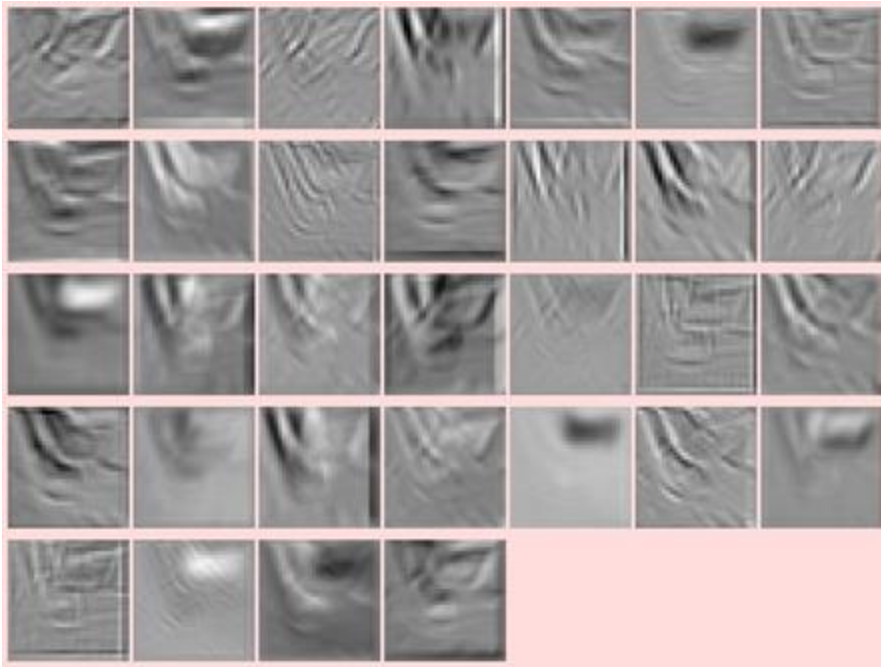


Camada Convolutucional

Entrada:



Ativações:

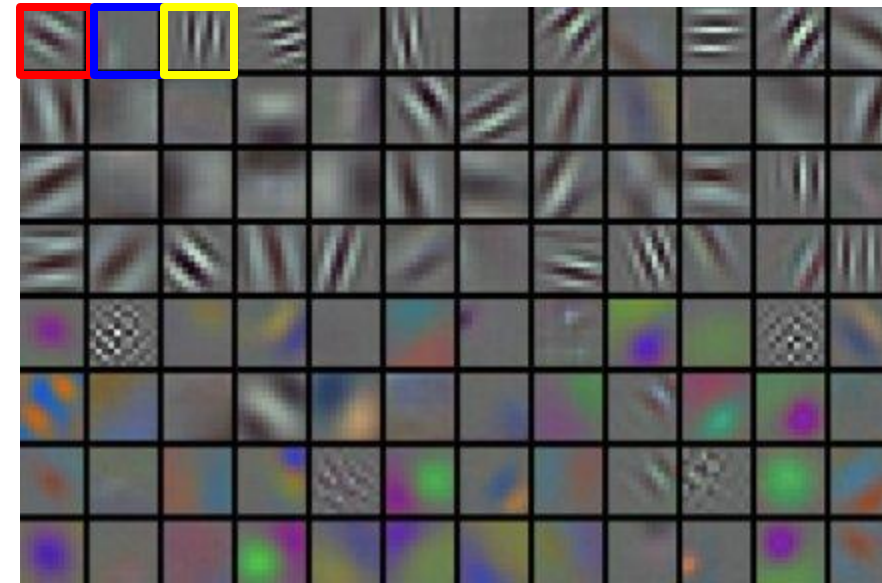
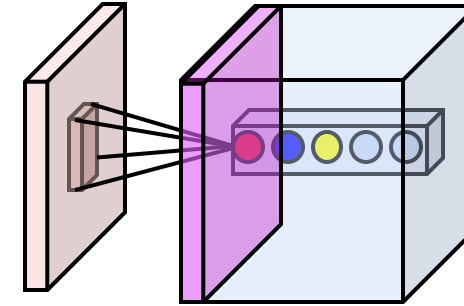
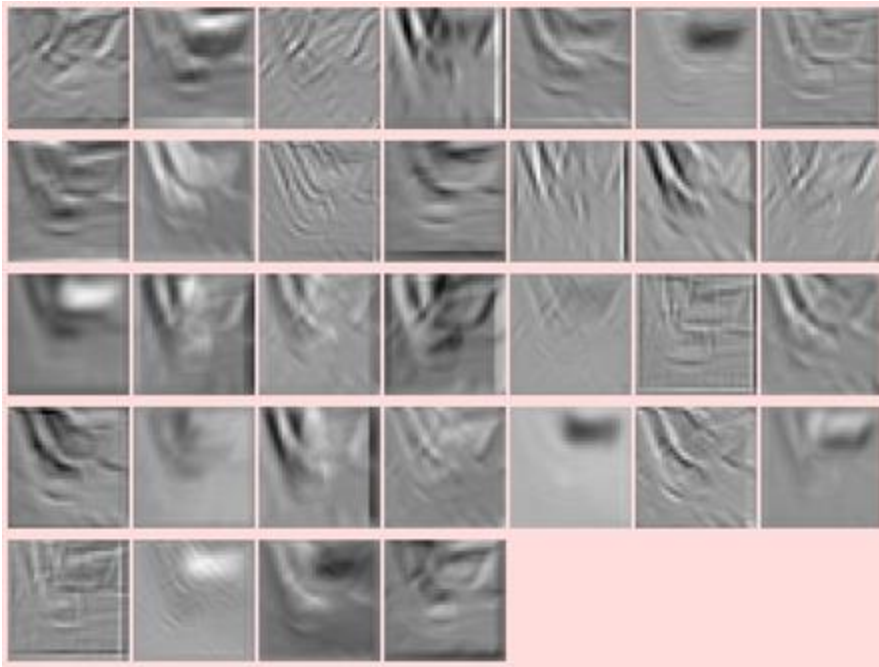


Camada Convolutucional

Entrada:



Ativações:

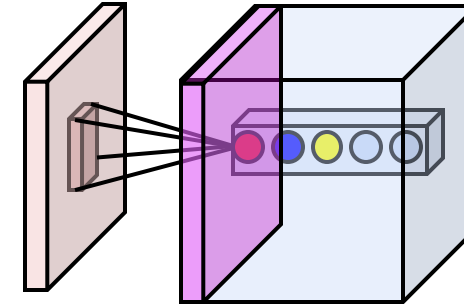


Camada Convolutucional

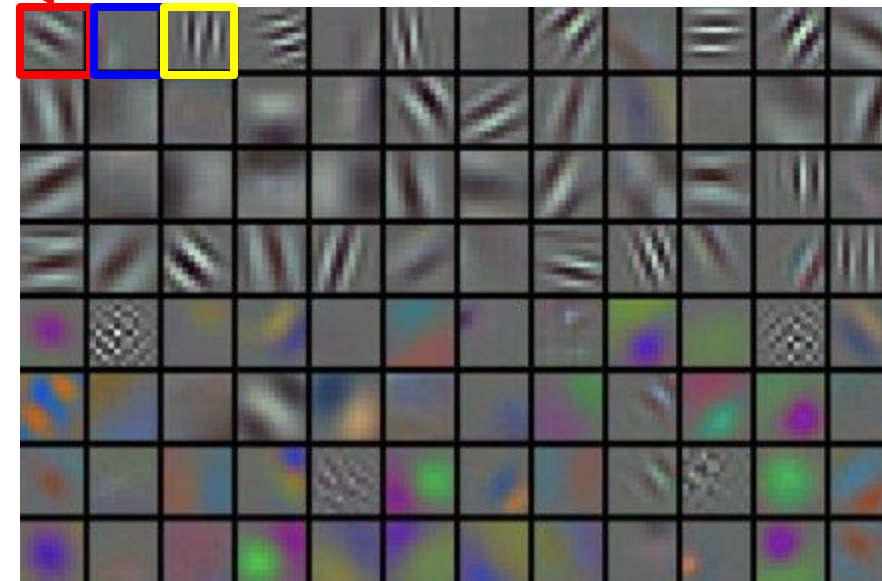
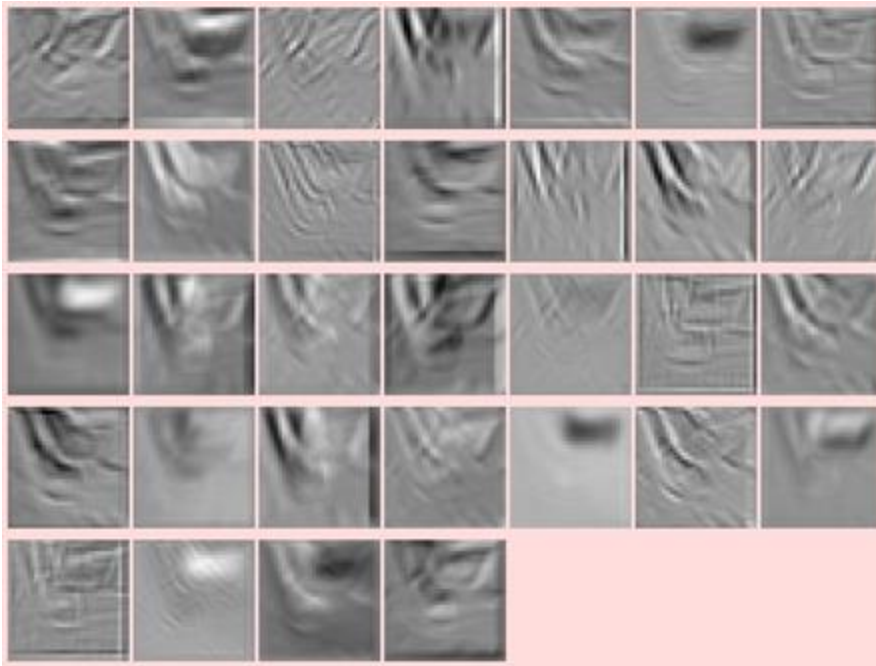
Entrada:



convoluir o primeiro **filtro** na entrada resulta na primeira fatia do volume de saída



Ativações:

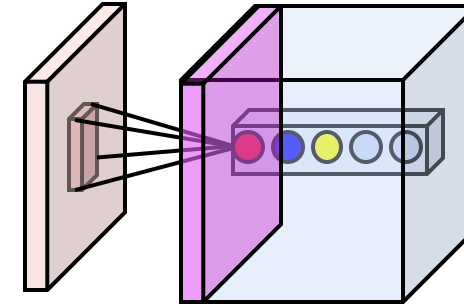


Camada Convolutucional

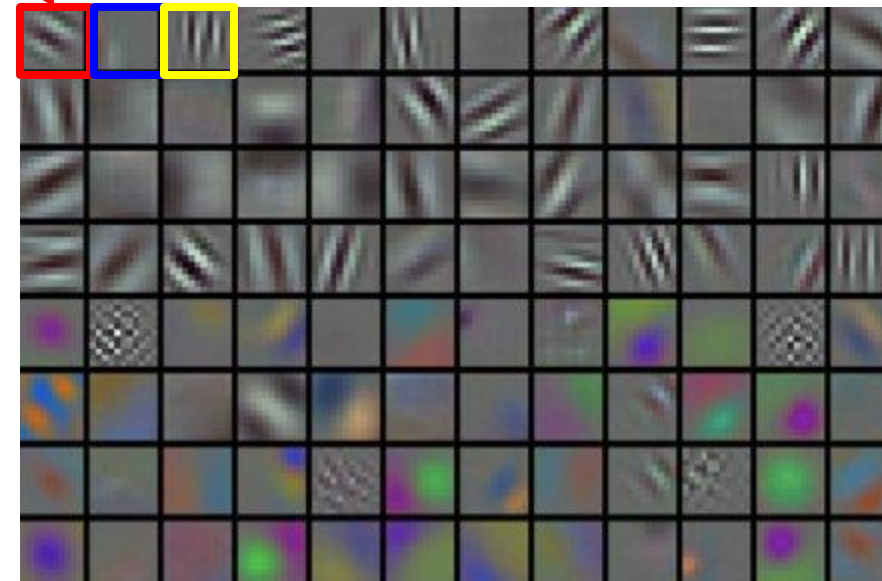
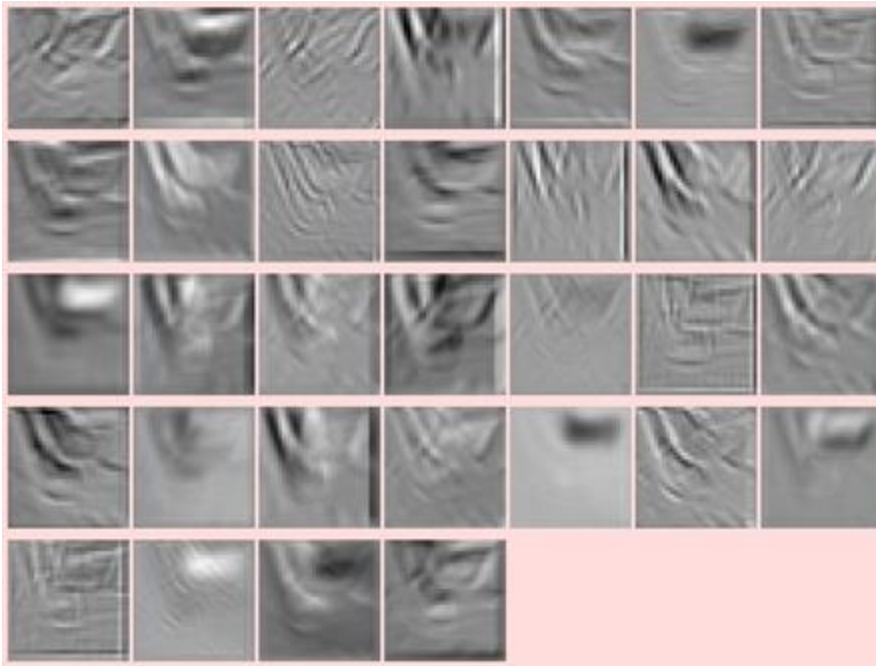
Entrada:



convoluir o primeiro **filtro** na **entrada** resulta na primeira fatia do volume de saída



Ativações:

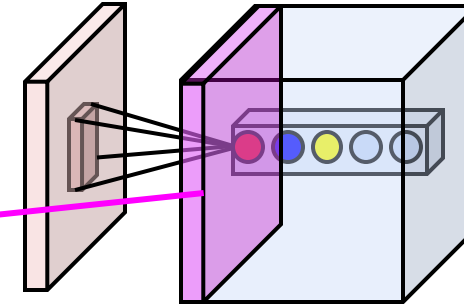


Camada Convolutucional

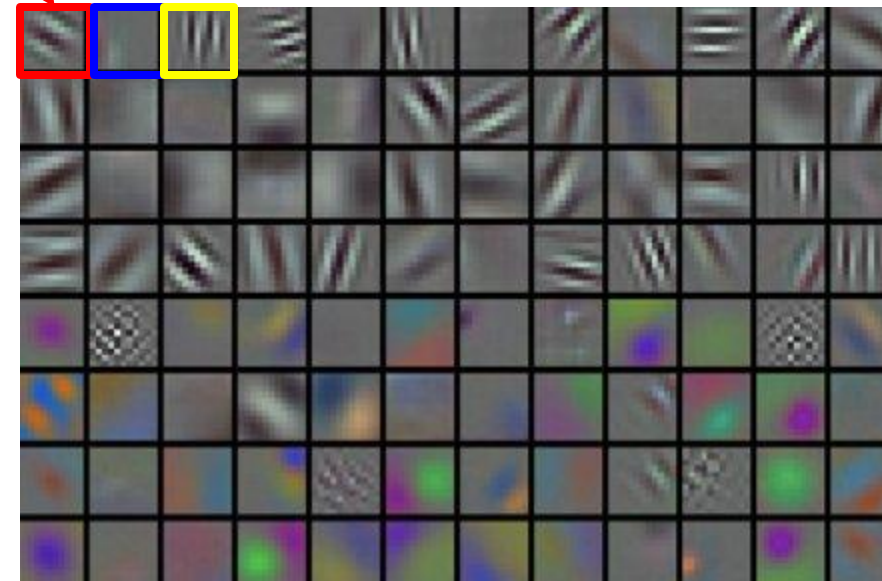
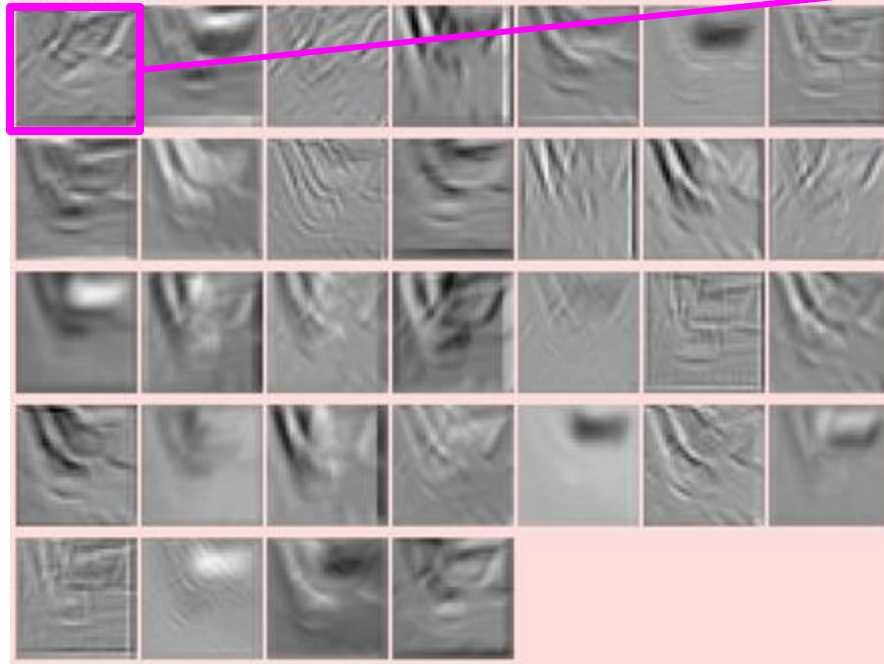
Entrada:



convoluir o primeiro filtro na entrada resulta na primeira fatia do volume de saída



Ativações:



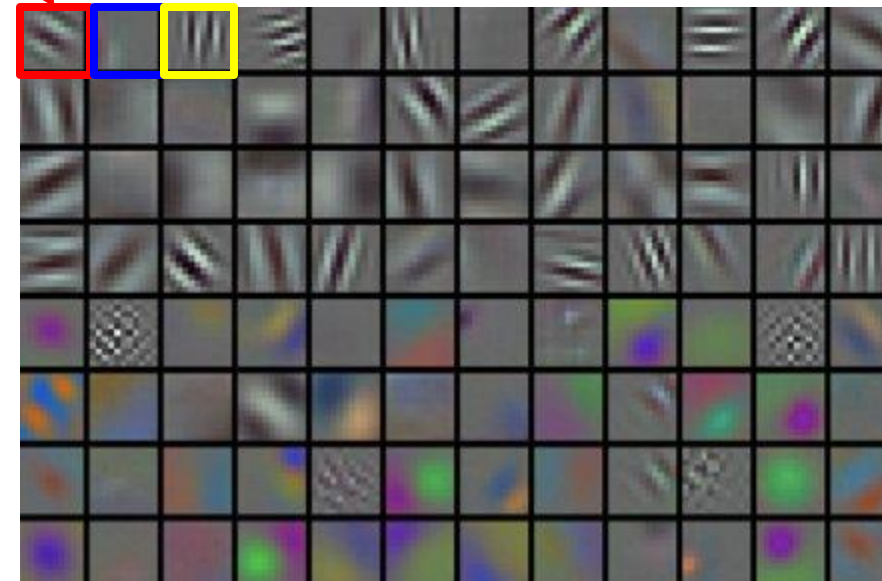
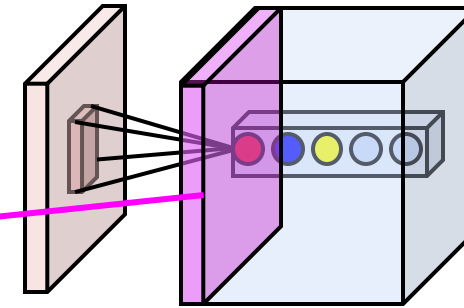
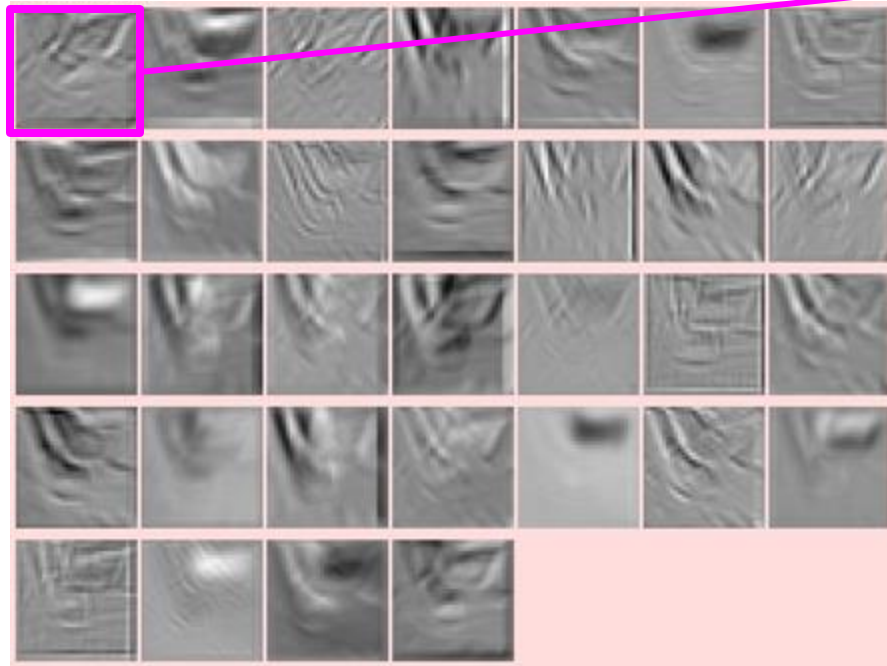
Camada Convolutucional

Entrada:

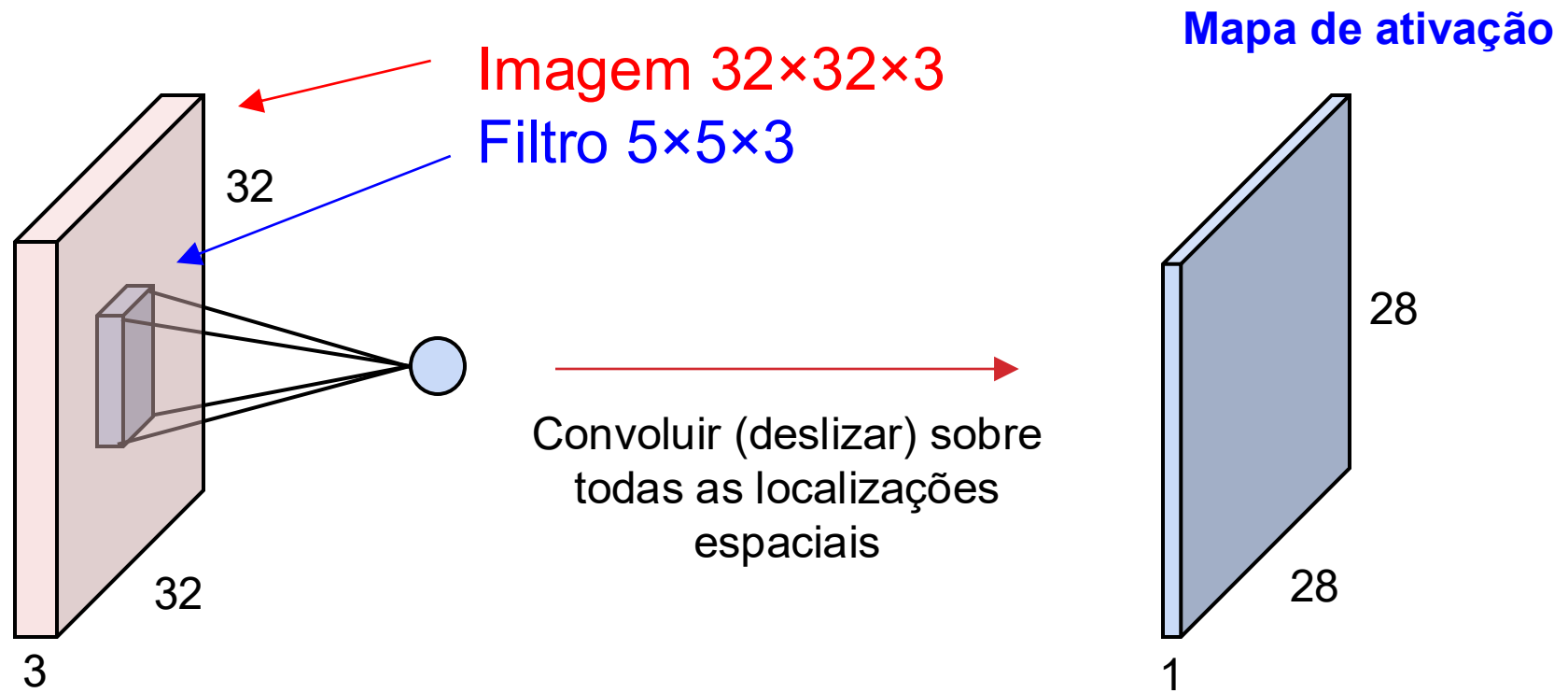


convoluir o primeiro filtro na entrada resulta na primeira fatia do volume de saída

Ativações:

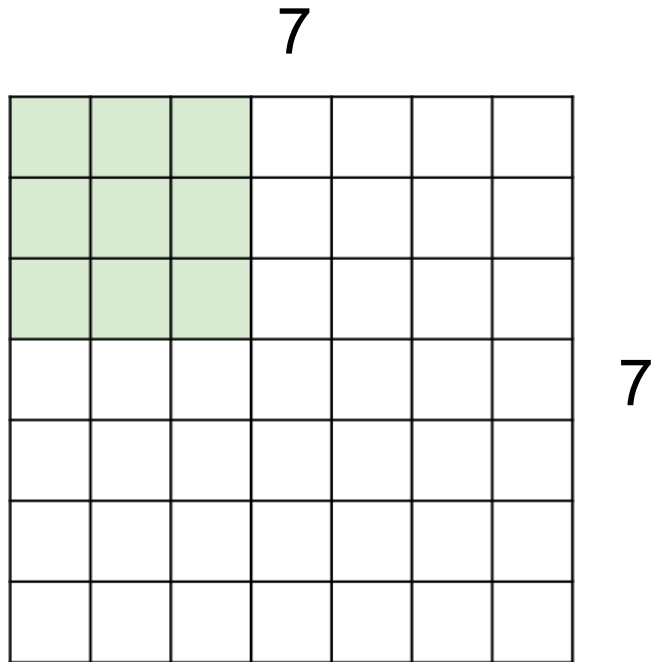


Camada Convolutiva



Camada Convolutiva – Tamanho do Passo

Um olhar mais atento às dimensões espaciais:

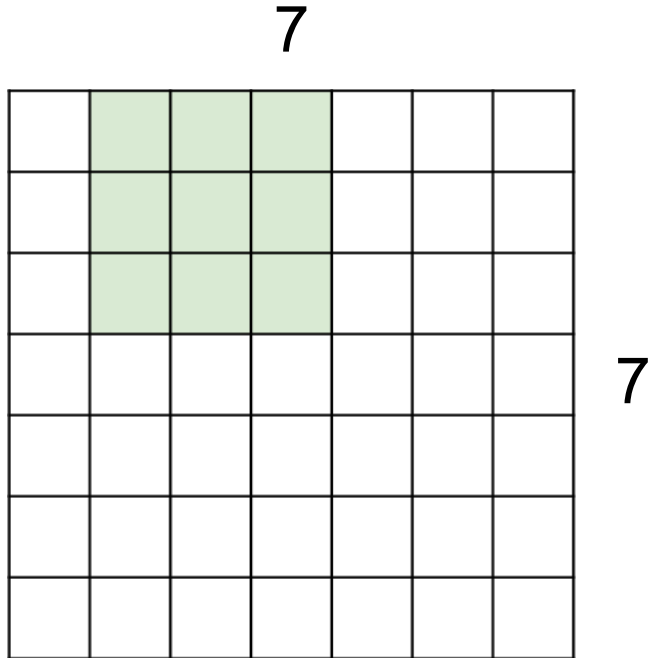


Entrada 7×7 (especialmente)

Assumindo um filtro 3×3 com **passo = 1**

Camada Convolutiva – Tamanho do Passo

Um olhar mais atento às dimensões espaciais:

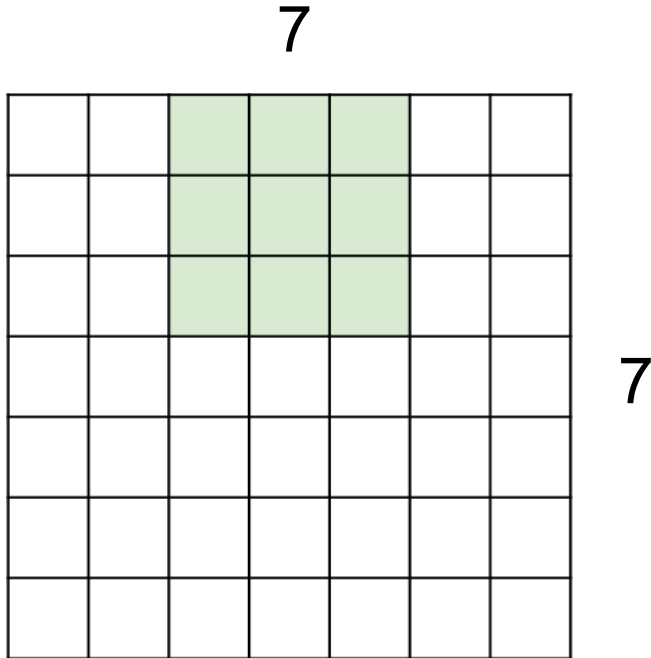


Entrada 7×7 (especialmente)

Assumindo um filtro 3×3 com **passo = 1**

Camada Convolutiva – Tamanho do Passo

Um olhar mais atento às dimensões espaciais:

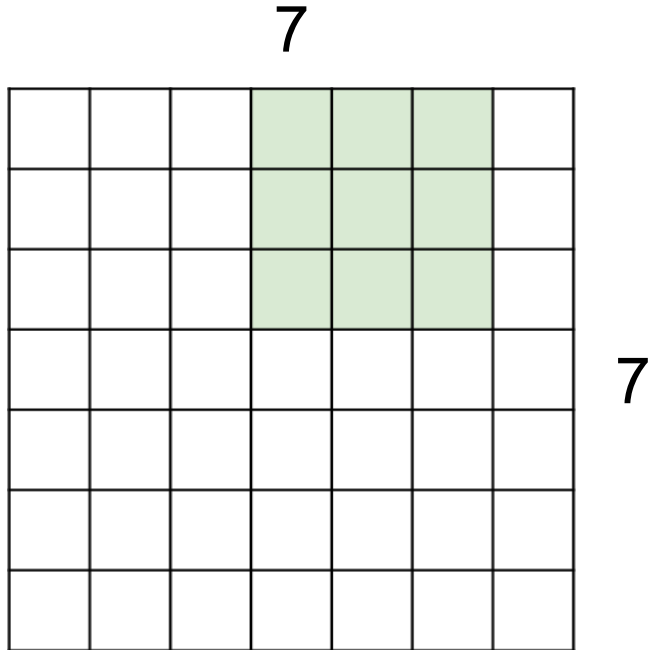


Entrada 7×7 (especialmente)

Assumindo um filtro 3×3 com **passo = 1**

Camada Convolutiva – Tamanho do Passo

Um olhar mais atento às dimensões espaciais:

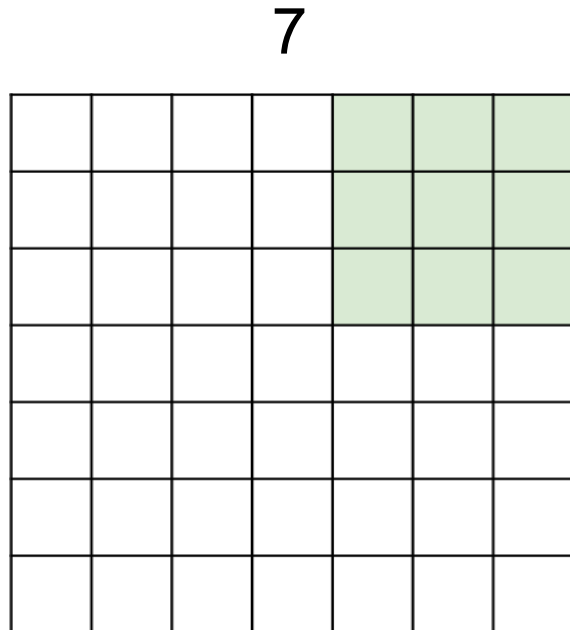


Entrada 7×7 (especialmente)

Assumindo um filtro 3×3 com **passo = 1**

Camada Convolutiva – Tamanho do Passo

Um olhar mais atento às dimensões espaciais:



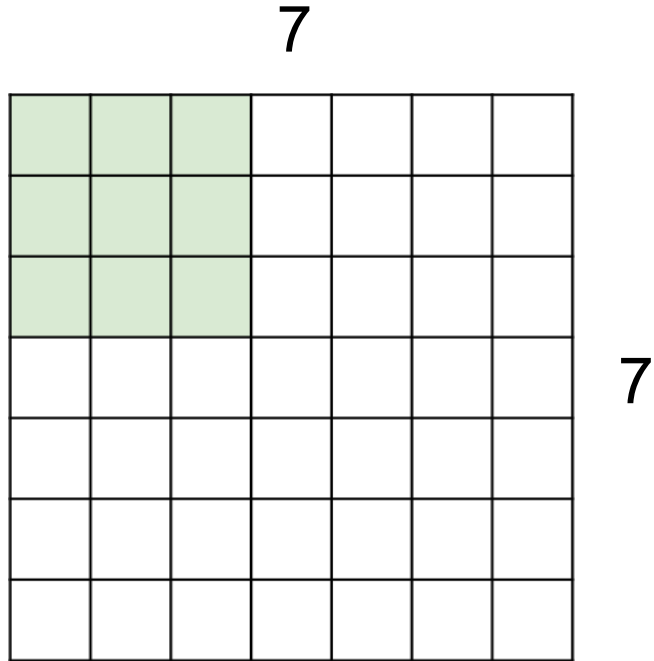
Entrada 7×7 (espacialmente)

Assumindo um filtro 3×3 com **passo = 1**

7 $\Rightarrow$ **Saída 5×5 (para passo = 1)**

Camada Convolutacional – Tamanho do Passo

Um olhar mais atento às dimensões espaciais:

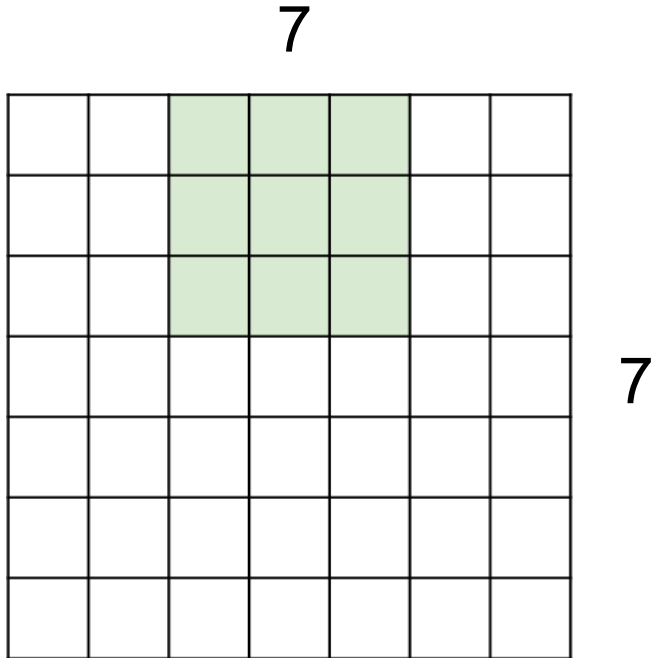


Entrada 7×7 (espacialmente)

Assumindo um filtro 3×3 com **passo = 2**

Camada Convolutiva – Tamanho do Passo

Um olhar mais atento às dimensões espaciais:

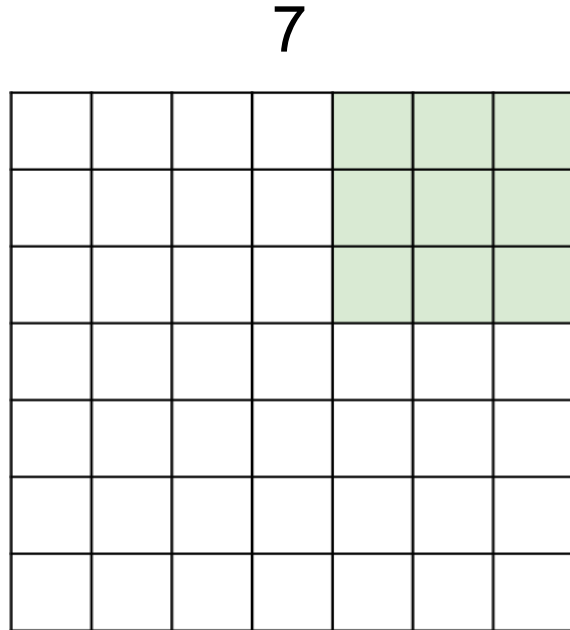


Entrada 7×7 (especialmente)

Assumindo um filtro 3×3 com **passo = 2**

Camada Convolutiva – Tamanho do Passo

Um olhar mais atento às dimensões espaciais:



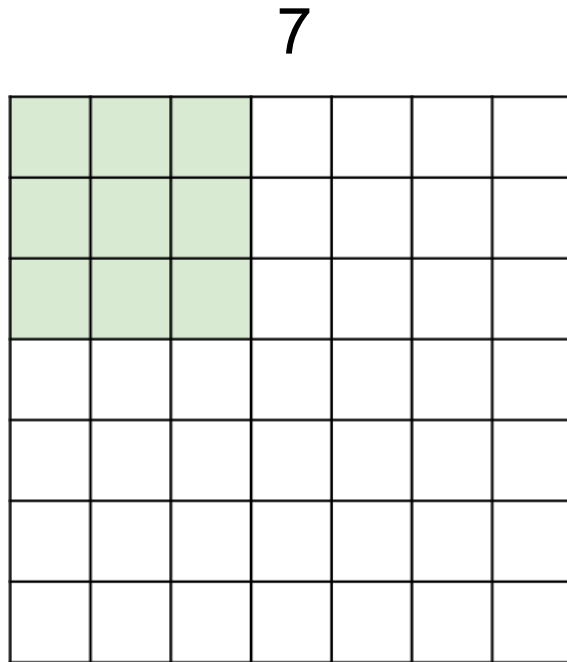
Entrada 7×7 (espacialmente)

Assumindo um filtro 3×3 com **passo = 2**

7 $\Rightarrow$ **Saída 3×3 (para passo = 2)**

Camada Convolutacional – Tamanho do Passo

Um olhar mais atento às dimensões espaciais:



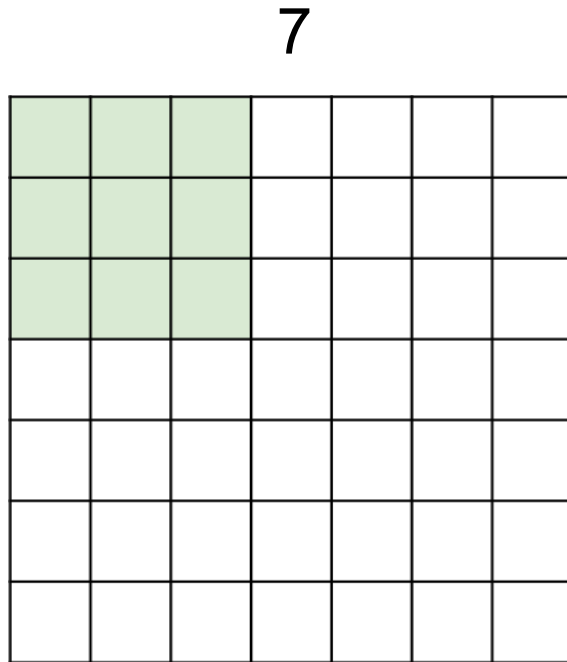
Entrada 7×7 (especialmente)

Assumindo um filtro 3×3 com **passo = 3**

Qual é a saída?

Camada Convolutacional – Tamanho do Passo

Um olhar mais atento às dimensões espaciais:



7

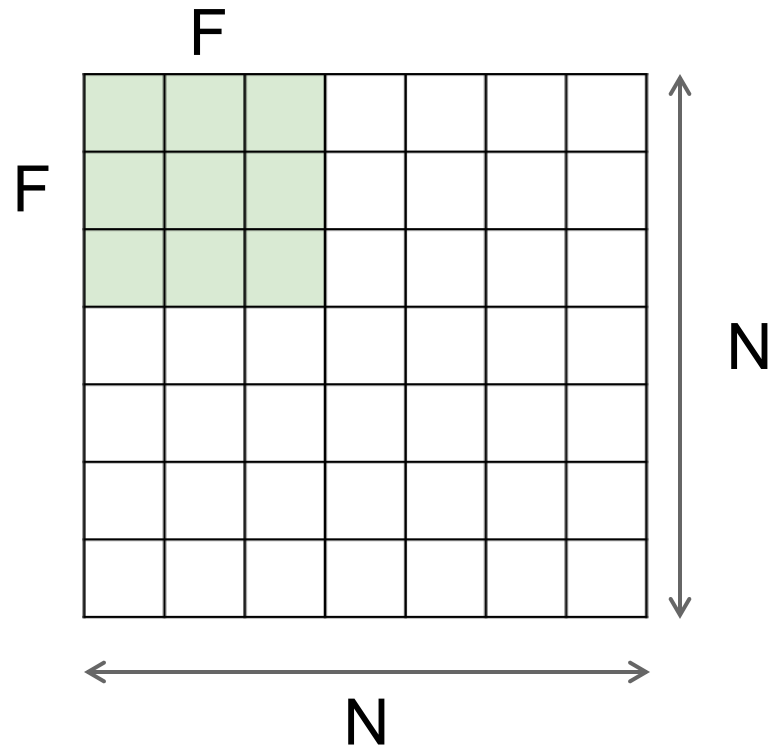
Entrada 7×7 (especialmente)

Assumindo um filtro 3×3 com **passo = 3**

Não se encaixa adequadamente!

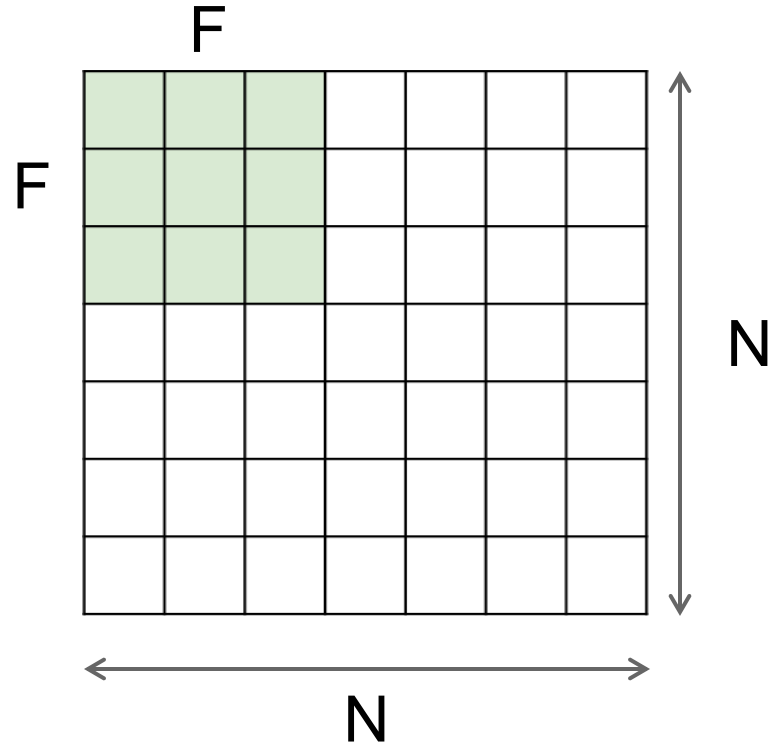
Não é possível aplicar o filtro 3×3 na entrada 7×7 com o passo 3

Camada Convolutacional – Tamanho do Passo



Tamanho da saída:
 $(N - F) / \text{passo} + 1$

Camada Convolutacional – Tamanho do Passo

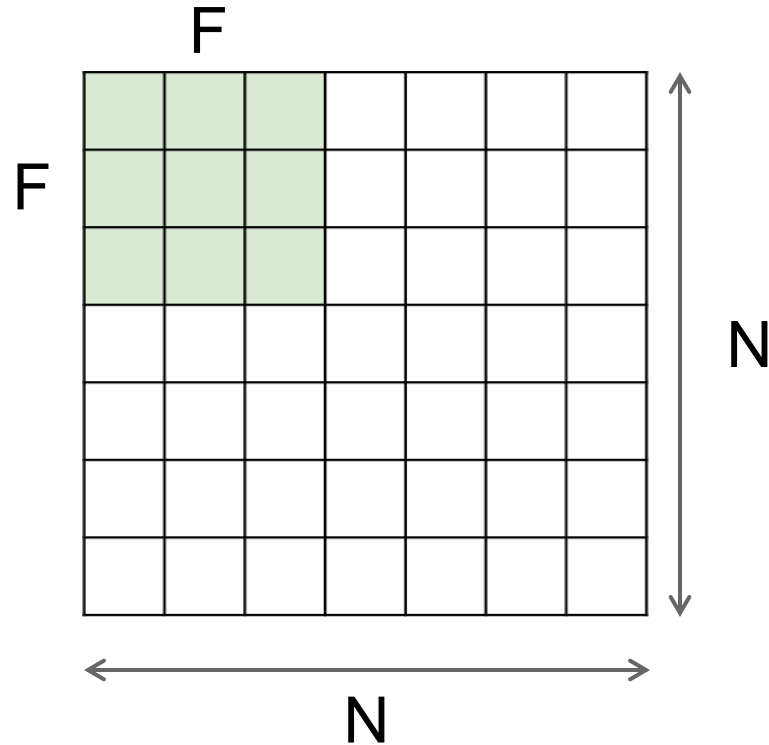


Tamanho da saída:

$$(N - F) / \text{passo} + 1$$

p.ex. $N = 7$, $F = 3$:

Camada Convolutacional – Tamanho do Passo



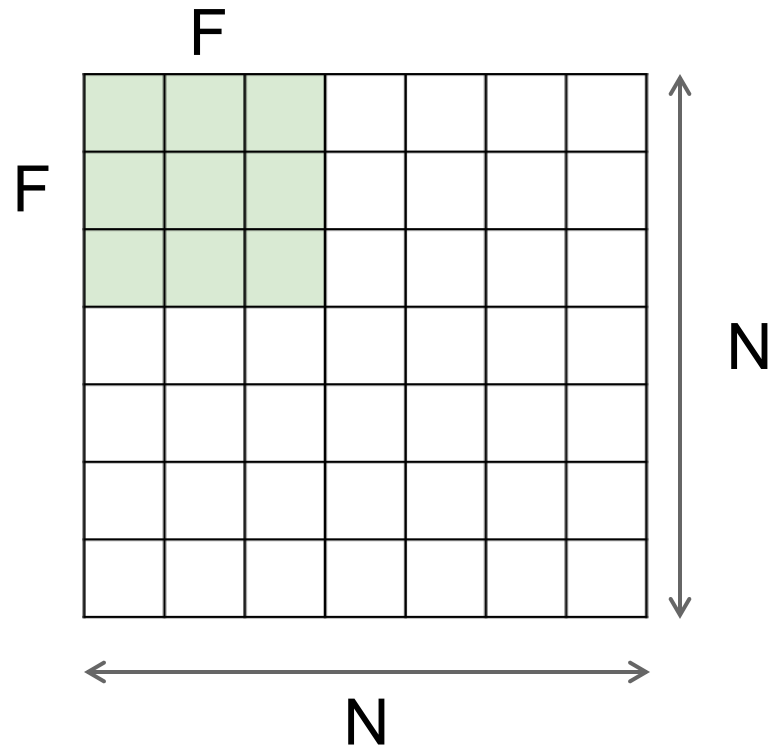
Tamanho da saída:

$$(N - F) / \text{passo} + 1$$

p.ex. $N = 7$, $F = 3$:

- passo 1 $\Rightarrow (7 - 3) / 1 + 1 = 5$

Camada Convolutacional – Tamanho do Passo



Tamanho da saída:

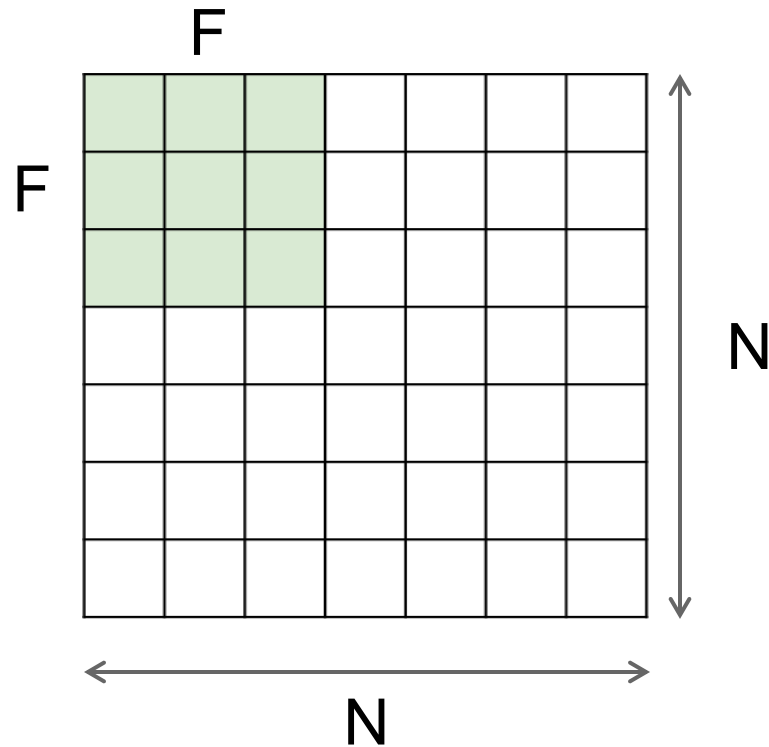
$$(N - F) / \text{passo} + 1$$

p.ex. $N = 7$, $F = 3$:

- passo 1 $\Rightarrow (7 - 3) / 1 + 1 = 5$

- passo 2 $\Rightarrow (7 - 3) / 2 + 1 = 3$

Camada Convolutacional – Tamanho do Passo



Tamanho da saída:

$$(N - F) / \text{passo} + 1$$

p.ex. $N = 7$, $F = 3$:

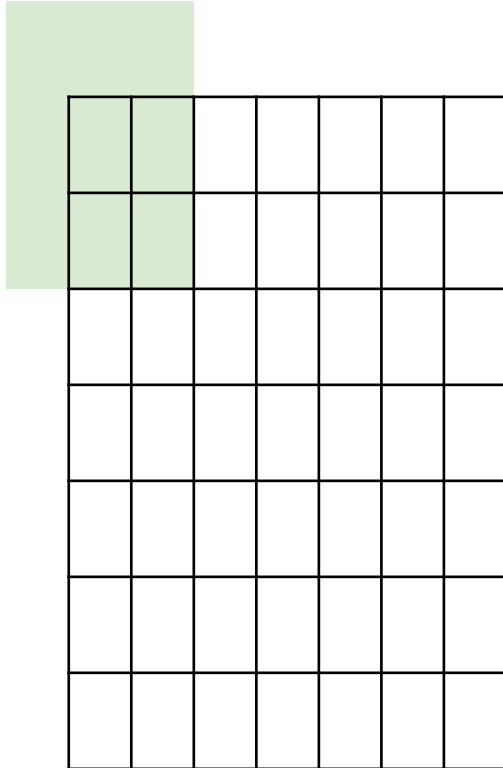
- passo 1 $\Rightarrow (7 - 3)/1 + 1 = 5$

- passo 2 $\Rightarrow (7 - 3)/2 + 1 = 3$

- passo 3 $\Rightarrow (7 - 3)/3 + 1 = 2,33 \text{ ☹️}$

Camada Convolutacional – Preenchimento

Problema de cálculo nas bordas



Entrada 7×7

Filtro 3×3 aplicado com **passo** = 1

Camada Convolutucional – Preenchimento

Problema de cálculo nas bordas

0	0	0	0	0	0			
0								
0								
0								
0								

Entrada 7×7

Filtro 3×3 aplicado com **passo = 1**

⇒ Na prática, é comum se acrescentar uma **borda de zeros** !

Camada Convolutiva – Preenchimento

Problema de cálculo nas bordas

0	0	0	0	0	0			
0								
0								
0								
0								

Entrada 7×7

Filtro 3×3 aplicado com **passo = 1**

Preenchimento (*padding*) com borda de tamanho 1

Qual é a saída?

⇒ Na prática, é comum se acrescentar uma **borda de zeros** !

Camada Convolutiva – Preenchimento

Problema de cálculo nas bordas

0	0	0	0	0	0			
0								
0								
0								
0								

Entrada 7×7

Filtro 3×3 aplicado com **passo = 1**

Preenchimento (*padding*) com borda de tamanho 1

Qual é a saída? $\Rightarrow$ **Saída 7×7**

$\Rightarrow$ Na prática, é comum se acrescentar uma **borda de zeros** !

Camada Convolutiva – Preenchimento

Problema de cálculo nas bordas

0	0	0	0	0	0			
0								
0								
0								
0								

Entrada 7×7

Filtro 3×3 aplicado com **passo = 1**

Preenchimento (*padding*) com borda de tamanho 1

Qual é a saída? ⇒ **Saída 7×7**

É comum, camadas convolucionais com passo 1, filtros de tamanho F e preenchimento **pad** = $(F - 1)/2$ zeros (para preservar as dimensões espaciais)

⇒ Na prática, é comum se acrescentar uma **borda de zeros** !

Camada Convolutiva – Preenchimento

Problema de cálculo nas bordas

0	0	0	0	0	0			
0								
0								
0								
0								

Entrada 7×7

Filtro 3×3 aplicado com **passo = 1**

Preenchimento (*padding*) com borda de tamanho 1

Qual é a saída? ⇒ **Saída 7×7**

É comum, camadas convolucionais com passo 1, filtros de tamanho F e preenchimento **pad = $(F - 1)/2$** zeros (para preservar as dimensões espaciais)

p.ex. $F = 3 \Rightarrow$ preenchimento com **1 zero**

$F = 5 \Rightarrow$ preenchimento com **2 zeros**

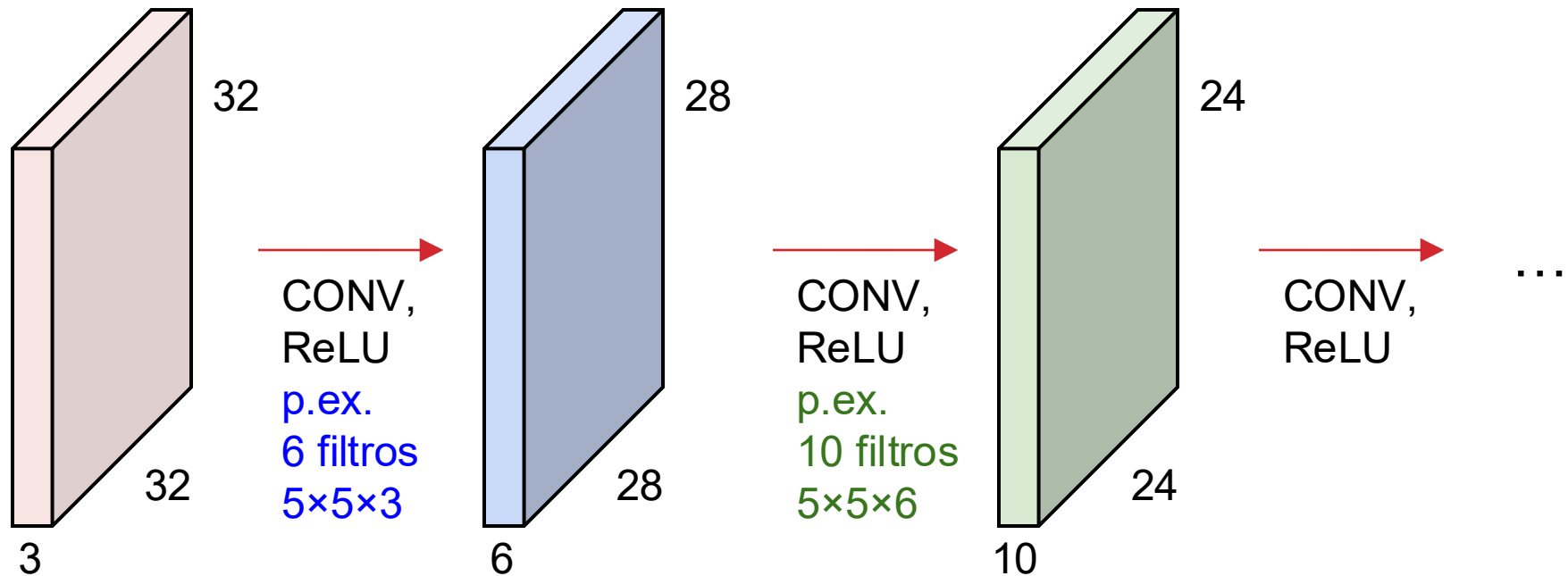
$F = 7 \Rightarrow$ preenchimento com **3 zeros**

⇒ Na prática, é comum se acrescentar uma **borda de zeros** !

Camada Convolutiva – Preenchimento

Lembre-se ...

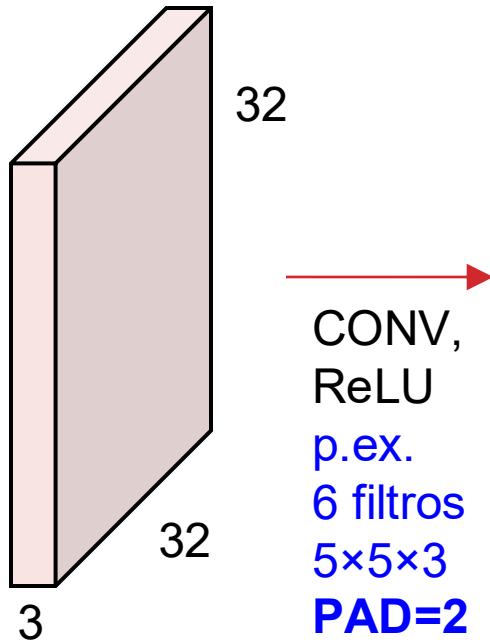
Por exemplo, uma entrada 32x32 convoluída repetidamente com filtros 5x5 reduz o volume espacialmente!
(32 → 28 → 24 ...) Encolher muito rápido não é bom e não funciona bem!



Camada Convolutiva – Preenchimento

Lembre-se ...

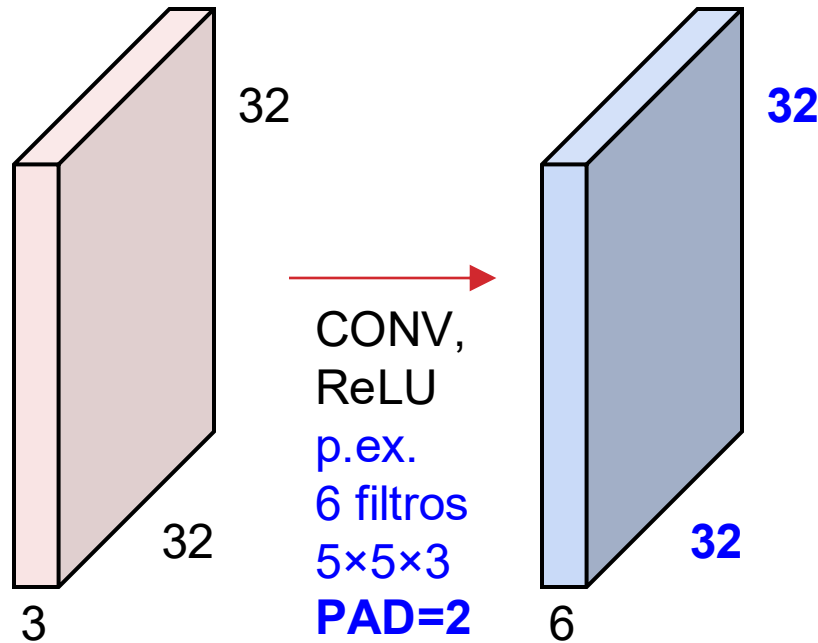
Por exemplo, uma entrada 32x32 convoluída repetidamente com filtros 5x5 reduz o volume espacialmente!
(32 → 28 → 24 ...) Encolher muito rápido não é bom e não funciona bem!



Camada Convolutiva – Preenchimento

Lembre-se ...

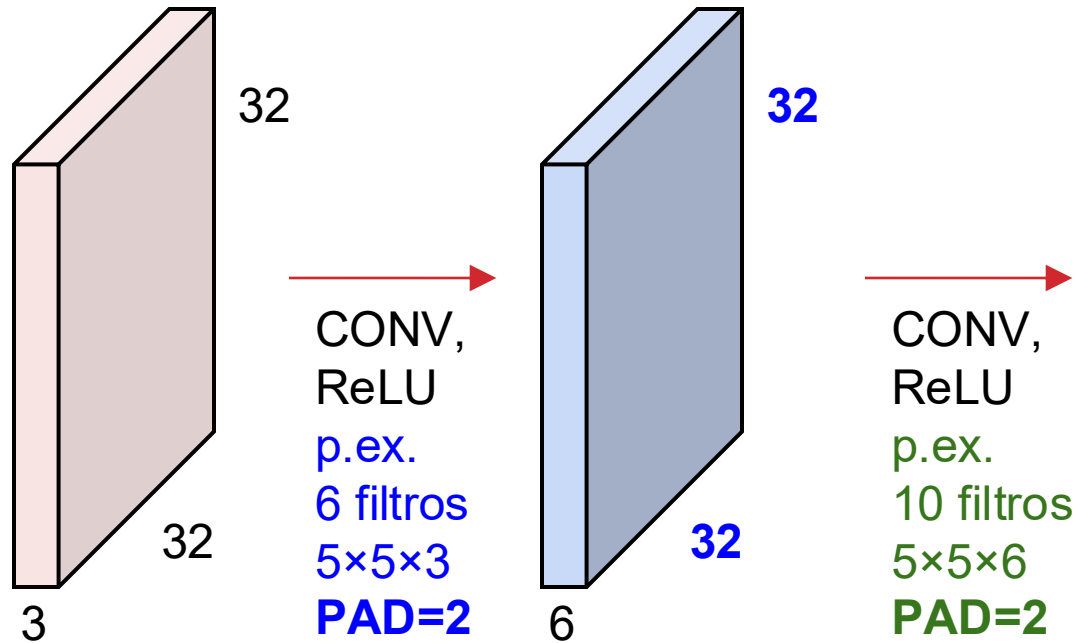
Por exemplo, uma entrada 32x32 convoluída repetidamente com filtros 5x5 reduz o volume espacialmente! (32 → 28 → 24 ...) Encolher muito rápido não é bom e não funciona bem!



Camada Convolutiva – Preenchimento

Lembre-se ...

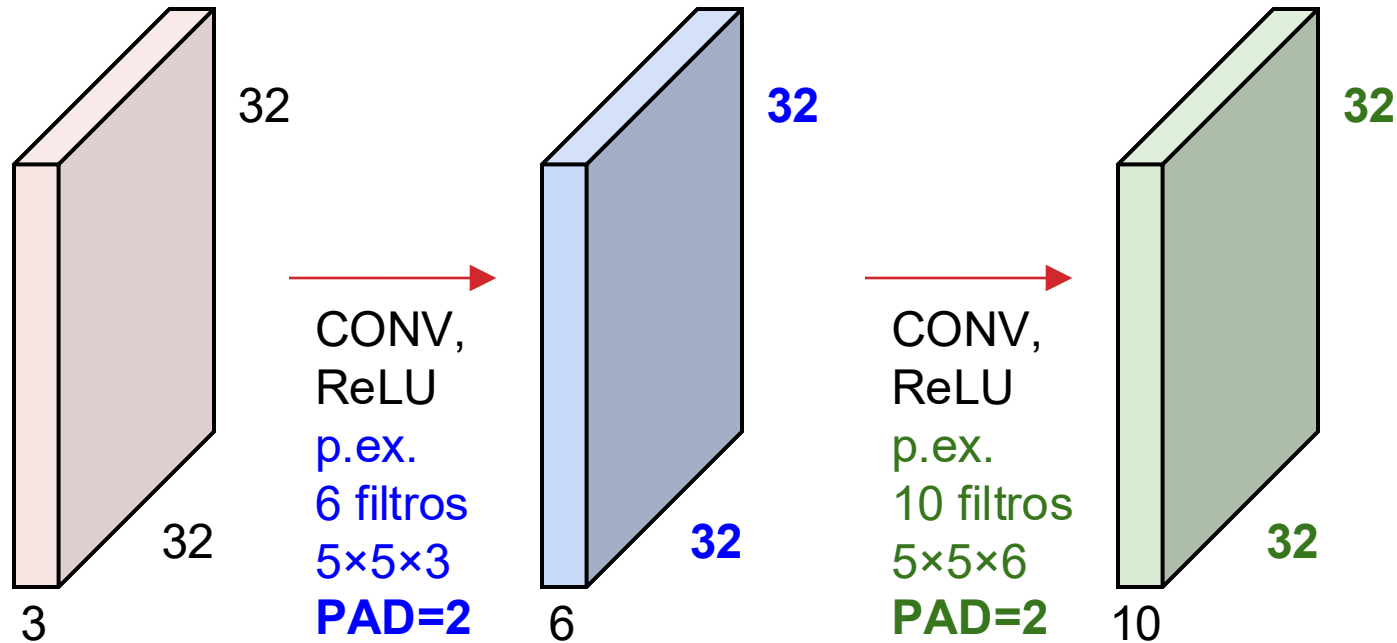
Por exemplo, uma entrada 32x32 convoluída repetidamente com filtros 5x5 reduz o volume espacialmente! (32 → 28 → 24 ...) Encolher muito rápido não é bom e não funciona bem!



Camada Convolutiva – Preenchimento

Lembre-se ...

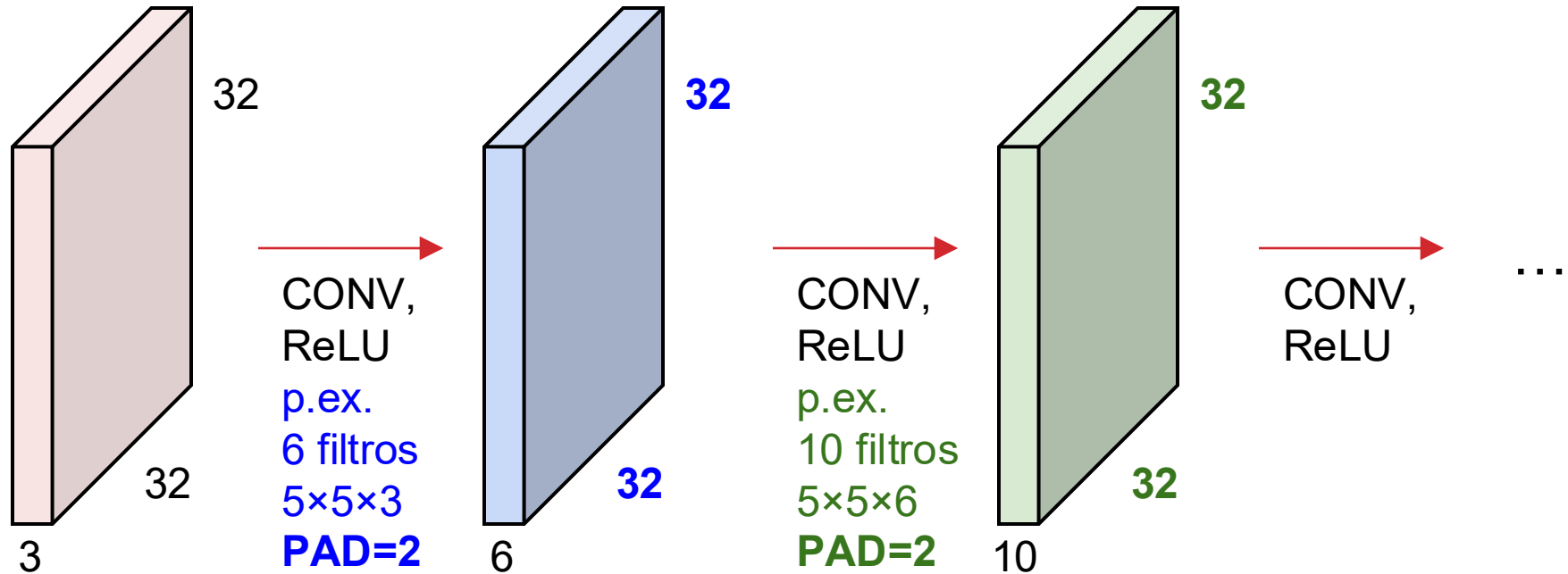
Por exemplo, uma entrada 32x32 convoluída repetidamente com filtros 5x5 reduz o volume espacialmente!
(32 → 28 → 24 ...) Encolher muito rápido não é bom e não funciona bem!



Camada Convolutacional – Preenchimento

Lembre-se ...

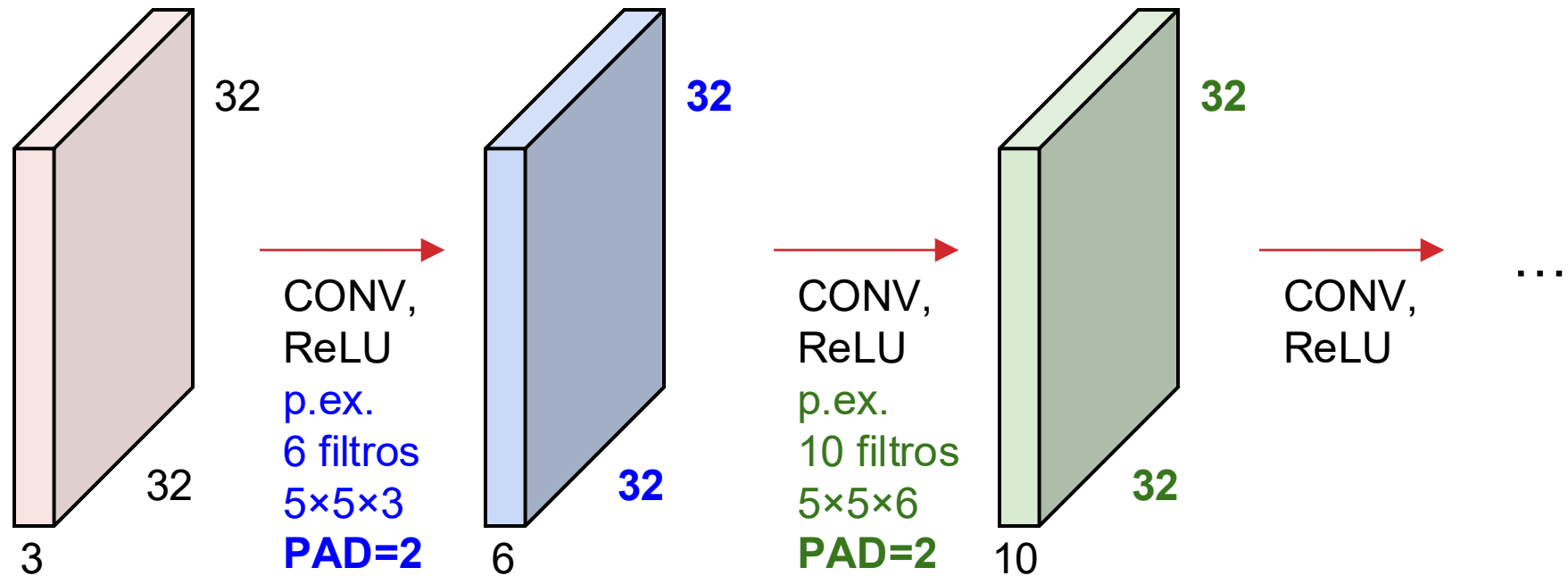
Por exemplo, uma entrada 32x32 convoluída repetidamente com filtros 5x5 reduz o volume espacialmente!
(32 → 28 → 24 ...) Encolher muito rápido não é bom e não funciona bem!



Camada Convolutiva – Preenchimento

Lembre-se ...

Por exemplo, uma entrada 32x32 convoluída repetidamente com filtros 5x5 reduz o volume espacialmente! (32 → 28 → 24 ...) Encolher muito rápido não é bom e não funciona bem!



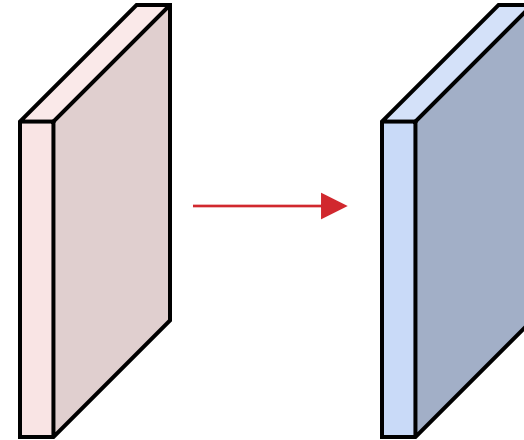
Preenchimento ajuda a minimizar o efeito da redução rápida de dimensionalidade espacial

Camada Convolutiva – Detalhes

Volume de entrada: **32×32×3**

10 filtros 5×5 com passo = 1, preenchimento = 2

Tamanho do volume de saída: ?



Camada Convolutacional – Detalhes

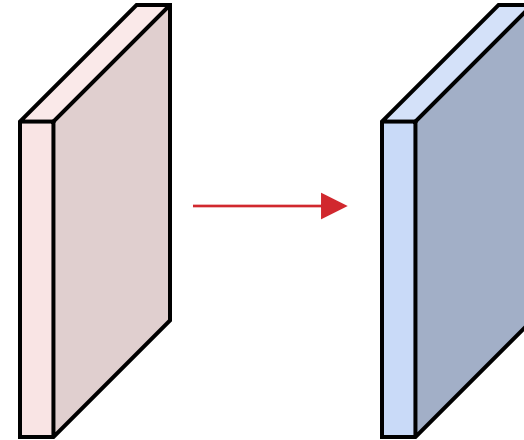
Volume de entrada: $32 \times 32 \times 3$

10 filtros 5×5 com passo = 1 , preenchimento = 2

Tamanho do volume de saída:

$$(32 + 2 \times 2 - 5) / 1 + 1 = 32 \text{ espacialmente,}$$

Portanto o volume de saída é $32 \times 32 \times \mathbf{10}$

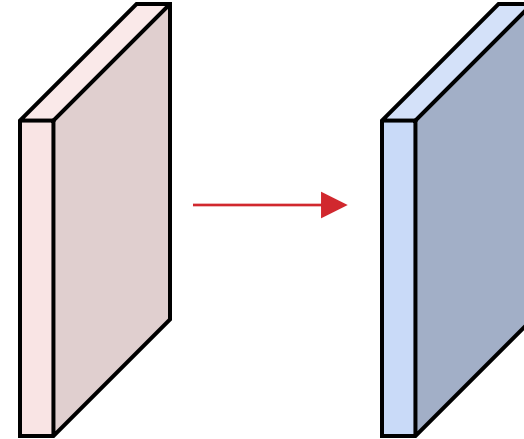


Camada Convolutiva – Detalhes

Volume de entrada: **32×32×3**

10 filtros 5×5 com passo = 1, preenchimento = 2

Número de parâmetros nessa camada: ?



Camada Convolutacional – Detalhes

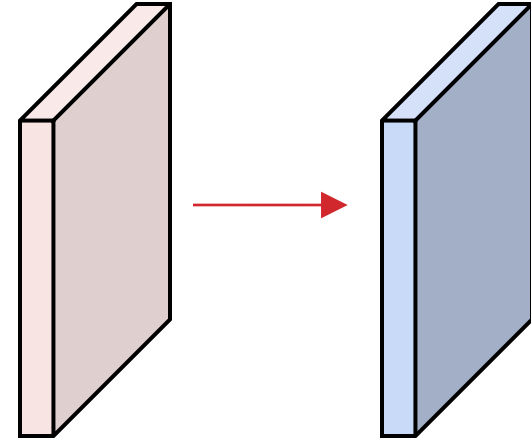
Volume de entrada: $32 \times 32 \times 3$

10 filtros 5×5 com passo = 1, preenchimento = 2

Número de parâmetros nessa camada:

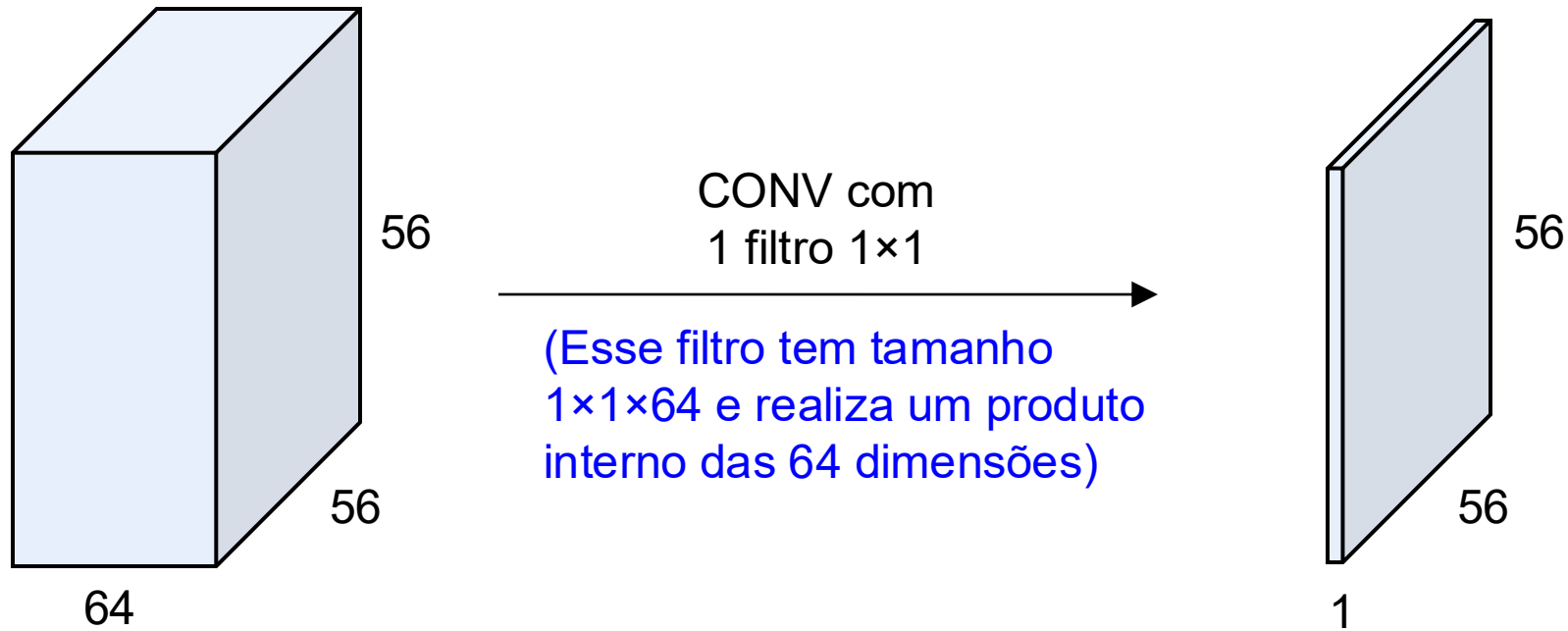
Cada filtro possui $5 \times 5 \times 3 + 1 = 76$ params (+1 para o viés)

Como são 10 filtros $\Rightarrow 76 \times 10 = 760$ params



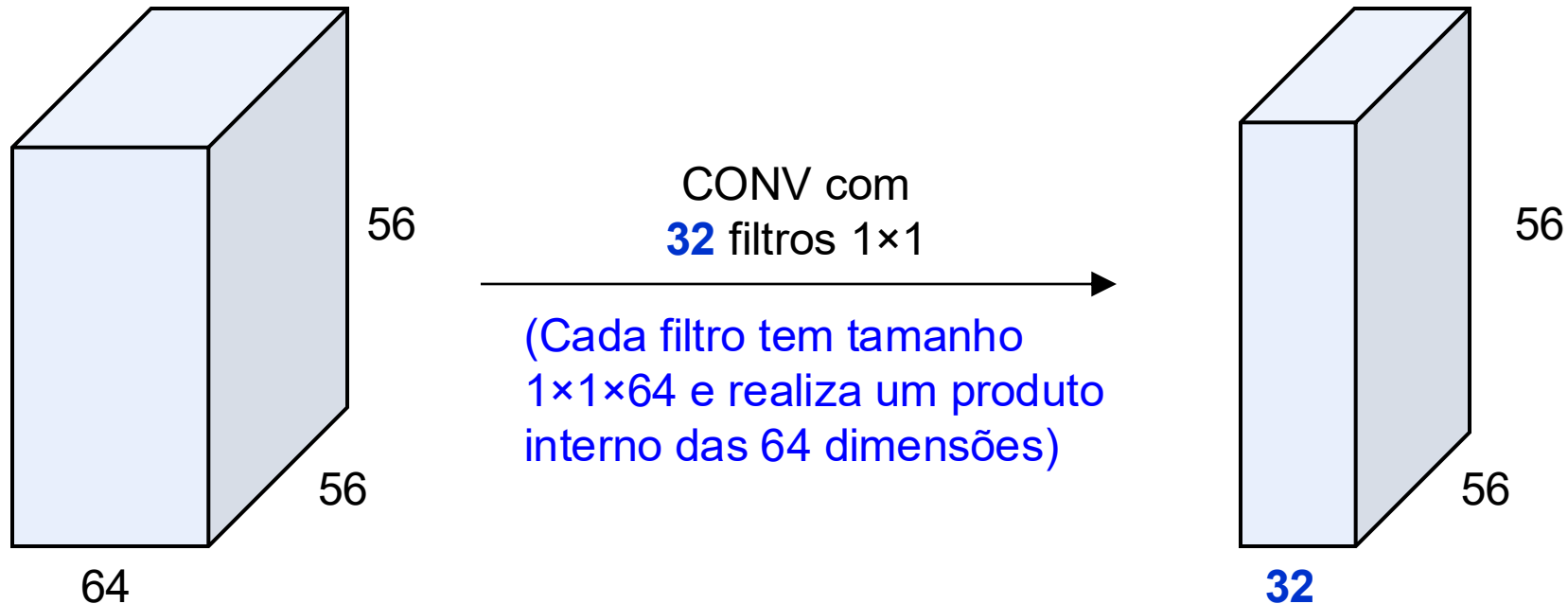
Camada Convolutiva

OBS: Camadas convolucionais 1×1 também fazem sentido!

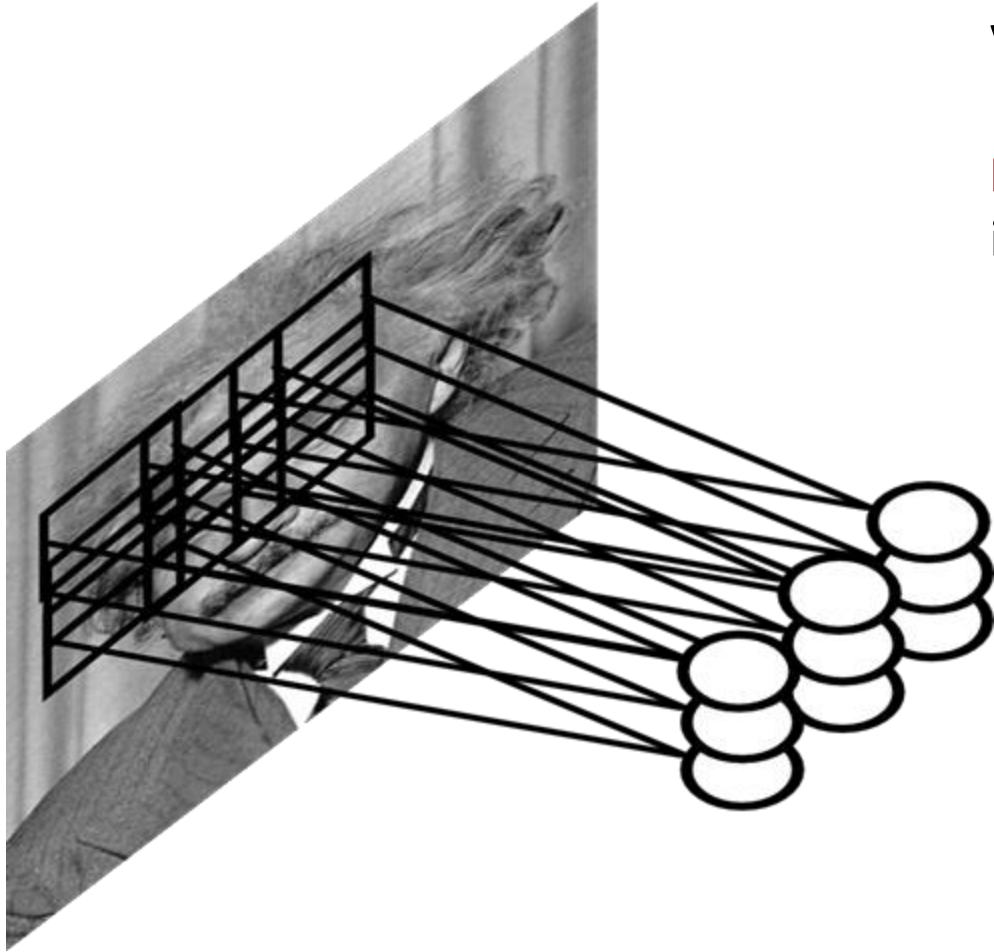


Camada Convolutiva

OBS: Camadas convolucionais 1×1 também fazem sentido!



Camada de Agrupamento (*Pooling*)



Vamos supor que o filtro seja um detector de “olhos”

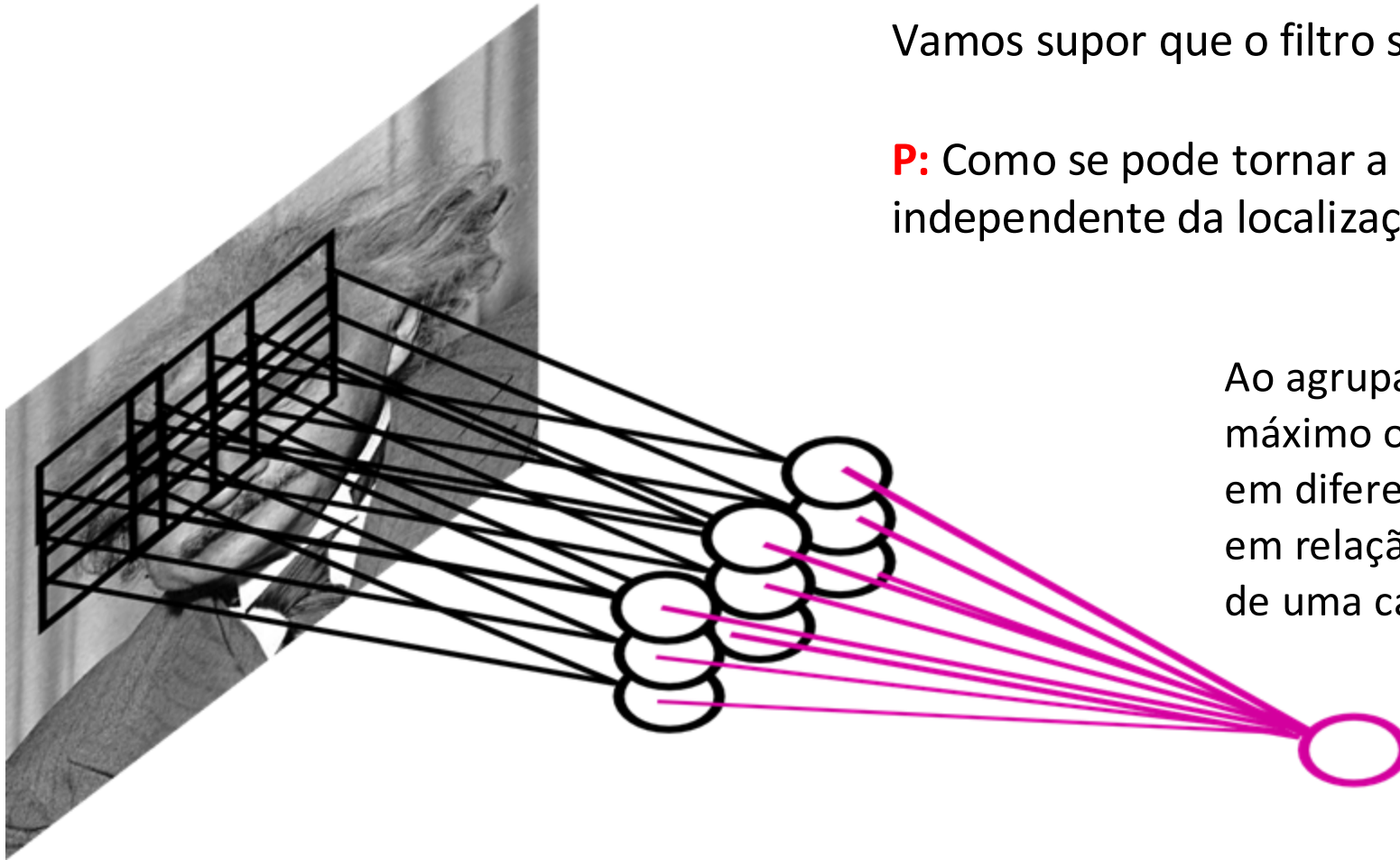
P: Como se pode tornar a detecção mais robusta e independente da localização exata do olho?

Camada de Agrupamento (*Pooling*)

Vamos supor que o filtro seja um detector de “olhos”

P: Como se pode tornar a detecção mais robusta e independente da localização exata do olho?

Ao agrupar (por exemplo, por meio do máximo ou média) as respostas do filtro em diferentes locais, ganha-se robustez em relação a localização espacial exata de uma característica

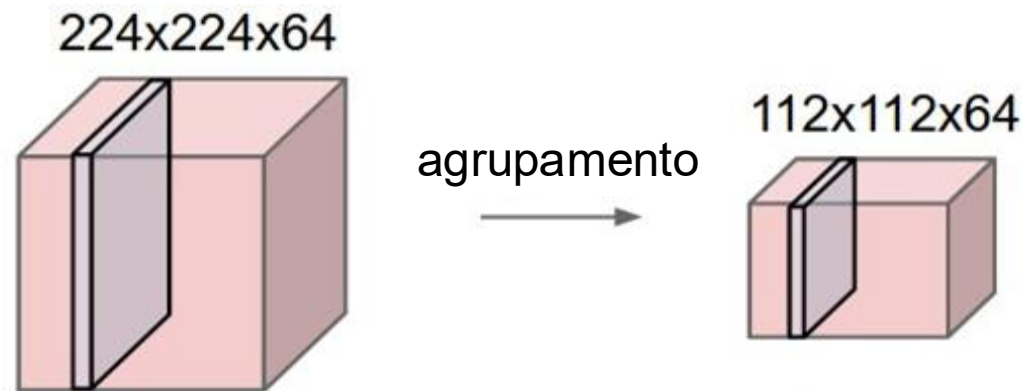


Camada de Agrupamento (*Pooling*)

- Torna as representações menores e mais gerenciáveis

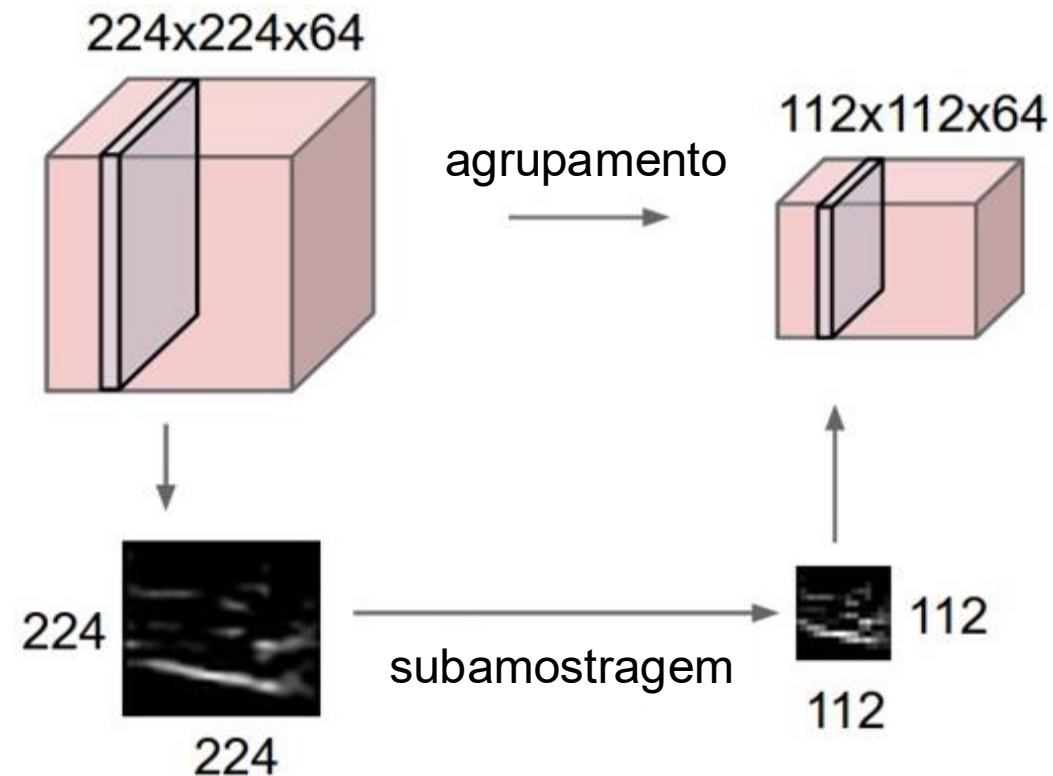
Camada de Agrupamento (*Pooling*)

- Torna as representações menores e mais gerenciáveis
- Opera sobre cada mapa de ativação de forma independente



Camada de Agrupamento (*Pooling*)

- Torna as representações menores e mais gerenciáveis
- Opera sobre cada mapa de ativação de forma independente



Camada de Agrupamento (*Pooling*)

Agrupamento utiliza uma operação para condensar as informações:

- Máximo
- Média
- L^2

Camada de Agrupamento (*Pooling*)

Agrupamento utiliza uma operação para condensar as informações:

- Máximo
- Média
- L^2

Exemplo de agrupamento pelo máximo (*max pooling*)

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

Um único mapa

Agrupamento pelo max
com filtros 2×2 e passo 2



Camada de Agrupamento (*Pooling*)

Agrupamento utiliza uma operação para condensar as informações:

- Máximo
- Média
- L^2

Exemplo de agrupamento pelo máximo (*max pooling*)

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

Um único mapa

Agrupamento pelo max
com filtros 2×2 e passo 2



Camada de Agrupamento (*Pooling*)

Agrupamento utiliza uma operação para condensar as informações:

- Máximo
- Média
- L^2

Exemplo de agrupamento pelo máximo (*max pooling*)

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

Um único mapa

Agrupamento pelo max
com filtros 2×2 e passo 2

6	

Camada de Agrupamento (*Pooling*)

Agrupamento utiliza uma operação para condensar as informações:

- Máximo
- Média
- L^2

Exemplo de agrupamento pelo máximo (*max pooling*)

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

Um único mapa

Agrupamento pelo max
com filtros 2×2 e passo 2

6	8

Camada de Agrupamento (*Pooling*)

Agrupamento utiliza uma operação para condensar as informações:

- Máximo
- Média
- L^2

Exemplo de agrupamento pelo máximo (*max pooling*)

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

Um único mapa

Agrupamento pelo max
com filtros 2×2 e passo 2

6	8
3	

Camada de Agrupamento (*Pooling*)

Agrupamento utiliza uma operação para condensar as informações:

- Máximo
- Média
- L^2

Exemplo de agrupamento pelo máximo (*max pooling*)

1	1	2	4
5	6	7	8
3	2	1	0
1	2	3	4

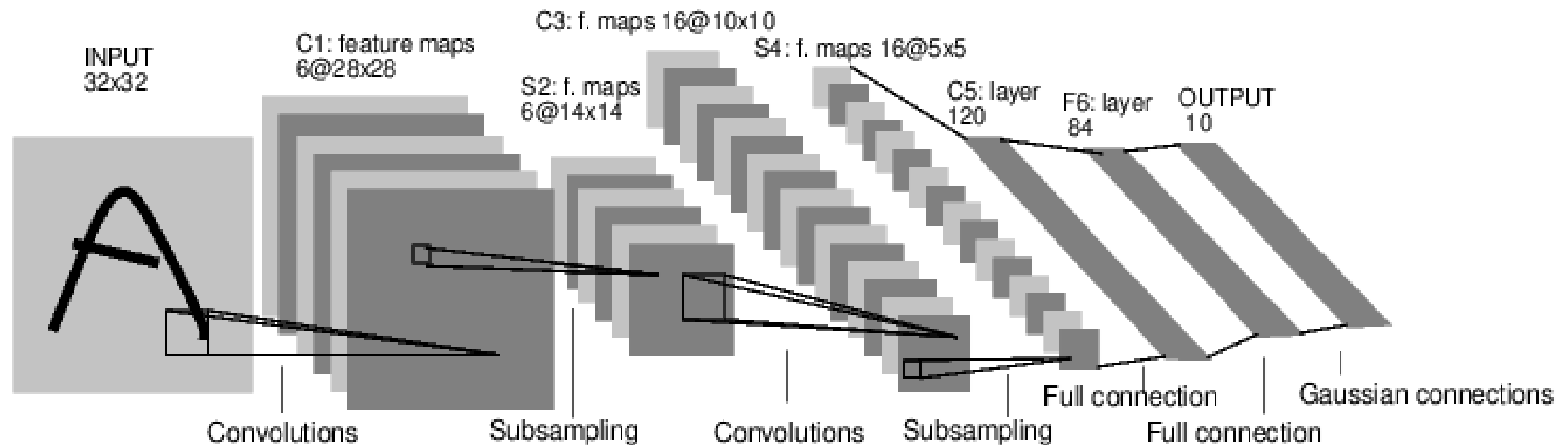
Um único mapa

Agrupamento pelo max
com filtros 2×2 e passo 2

6	8
3	4

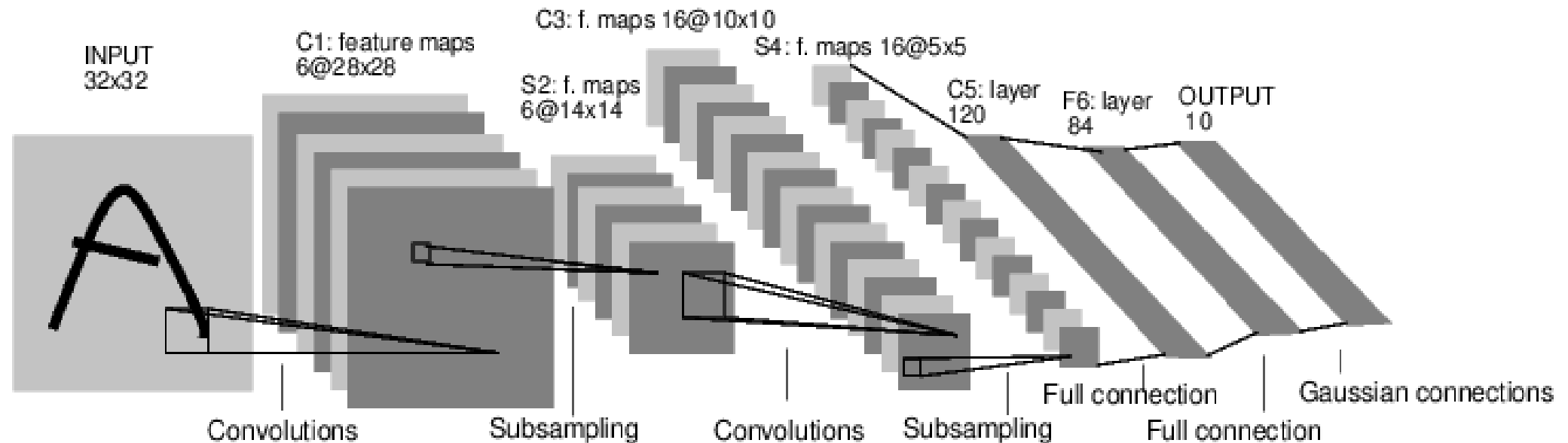
ConvNets – LeNet

[LeCun et al., 1998]



ConvNets – LeNet

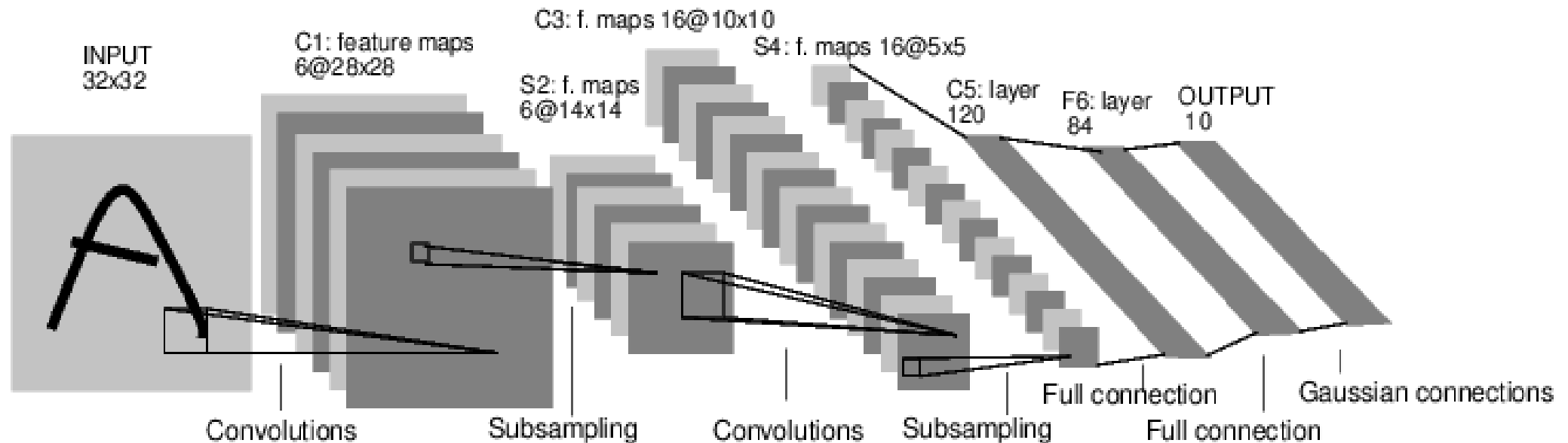
[LeCun et al., 1998]



- Filtros convolucionais 5×5 com passo 1

ConvNets – LeNet

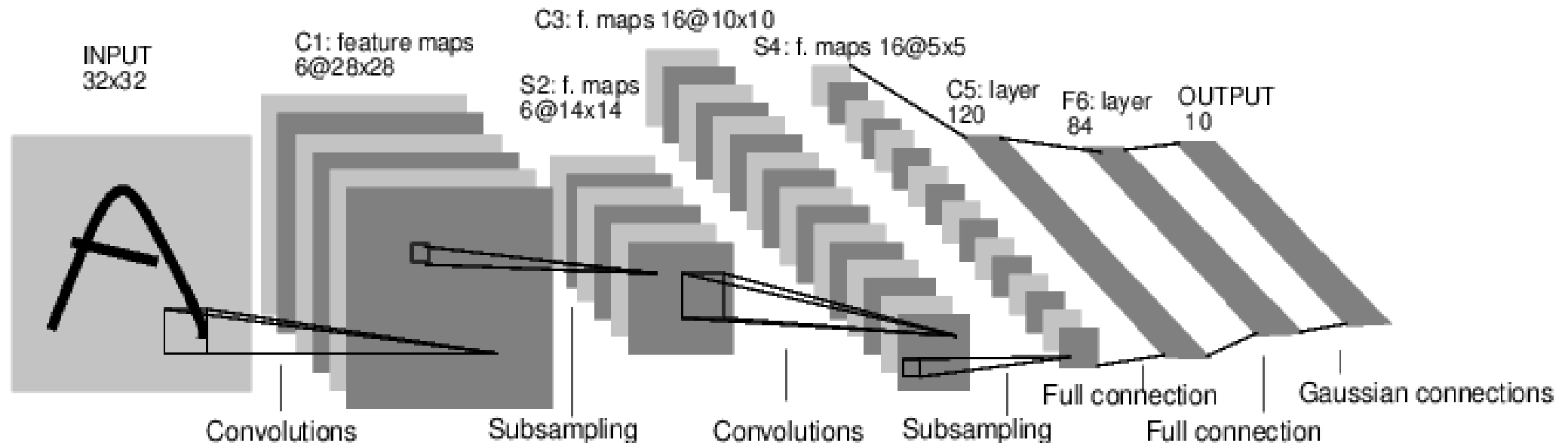
[LeCun et al., 1998]



- Filtros convolucionais 5×5 com passo 1
- Camadas de agrupamento (“subsampling”) com filtros 2×2 e passo 2

ConvNets – LeNet

[LeCun et al., 1998]

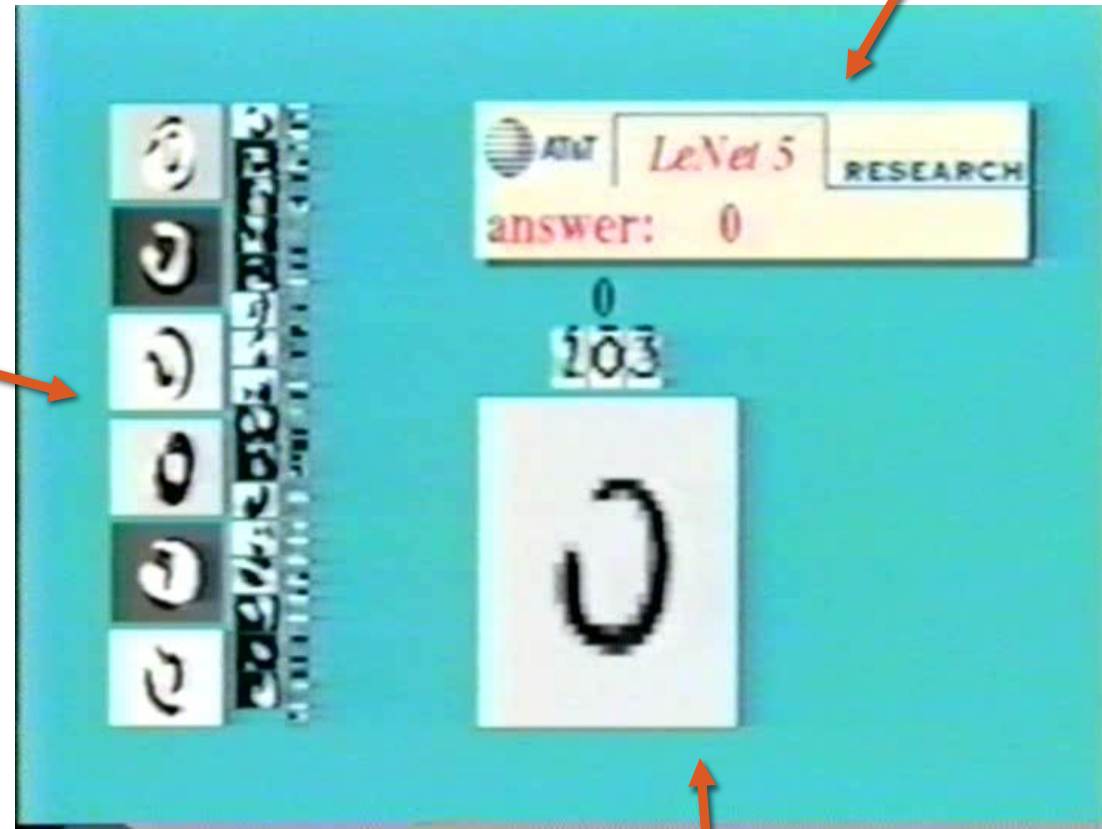


- Filtros convolucionais 5×5 com passo 1
- Camadas de agrupamento (“subsampling”) com filtros 2×2 e passo 2
- Arquitetura → [CONV-POOL-CONV-POOL-FC-FC]

ConvNets – LeNet

Classificação de Dígitos Manuscritos

Características
extraídas pelo
filtros

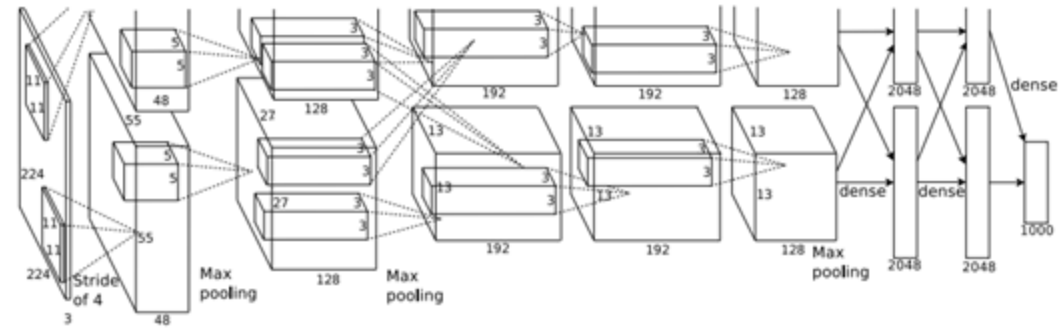


Entrada

Saída

ConvNets – AlexNet

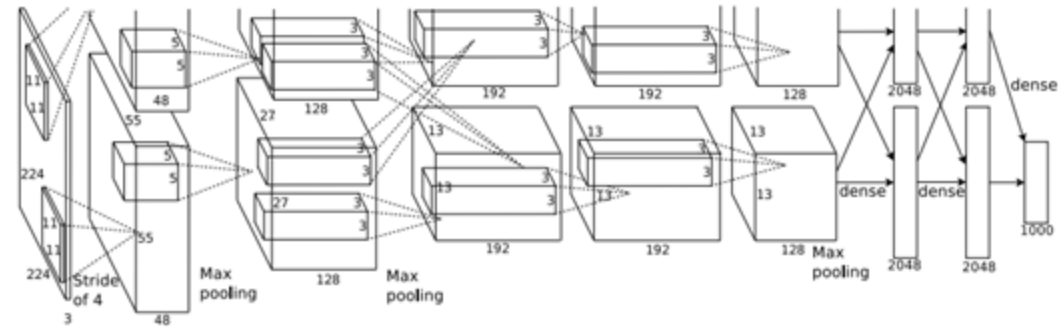
[Krizhevsky et al. 2012]



ConvNets – AlexNet

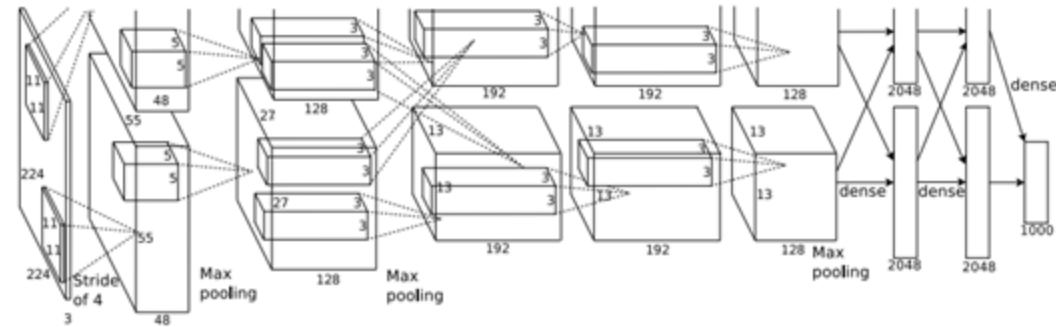
[Krizhevsky et al. 2012]

Entrada: imagem $227 \times 227 \times 3$



ConvNets – AlexNet

[Krizhevsky et al. 2012]

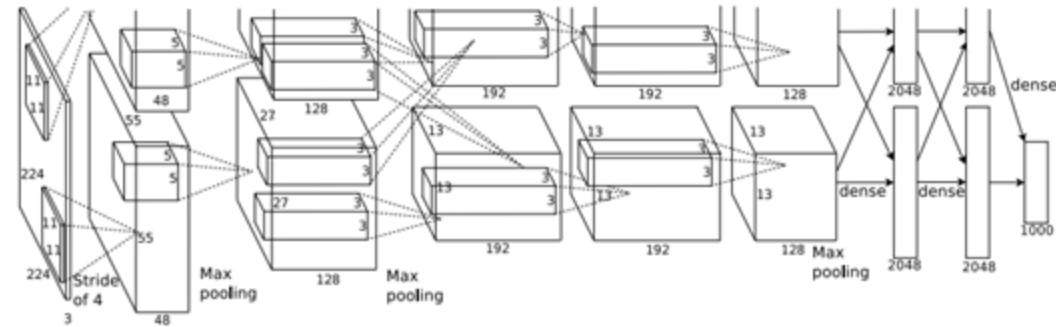


Entrada: imagem 227×227×3

Primeira camada (CONV1): 96 filtros 11×11 com passo 4

ConvNets – AlexNet

[Krizhevsky et al. 2012]



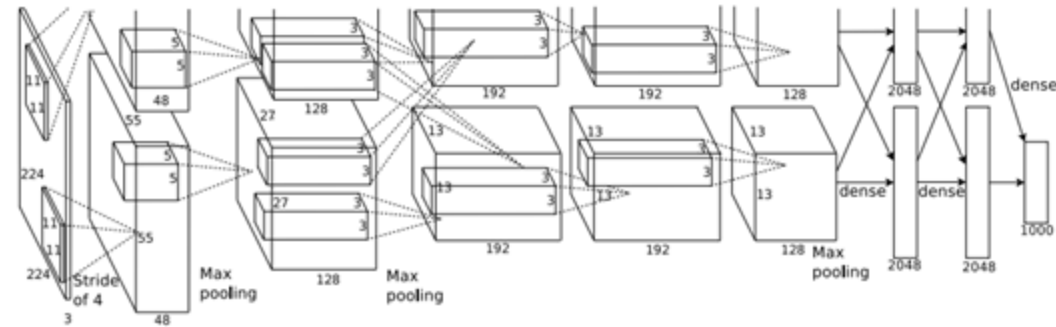
Entrada: imagem 227×227×3

Primeira camada (CONV1): 96 filtros 11×11 com passo 4

P: Qual é o tamanho do volume de saída? Dica: $(227-11)/4+1 = 55$

ConvNets – AlexNet

[Krizhevsky et al. 2012]



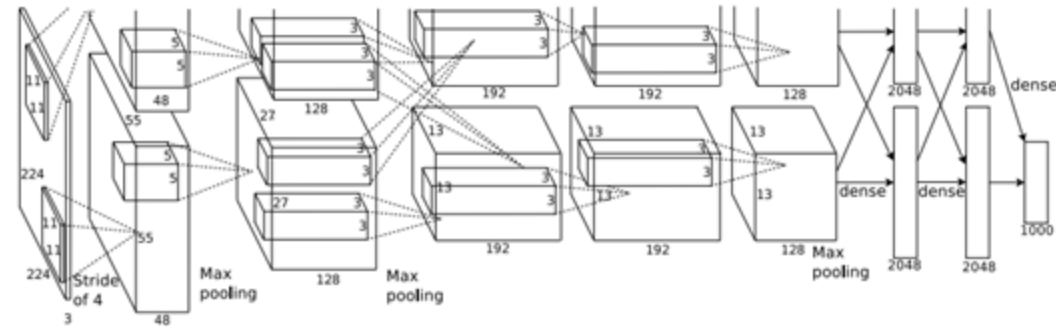
Entrada: imagem $227 \times 227 \times 3$

Primeira camada (CONV1): 96 filtros 11×11 com passo 4

Tamanho do volume de saída = **$55 \times 55 \times 96$**

ConvNets – AlexNet

[Krizhevsky et al. 2012]



Entrada: imagem $227 \times 227 \times 3$

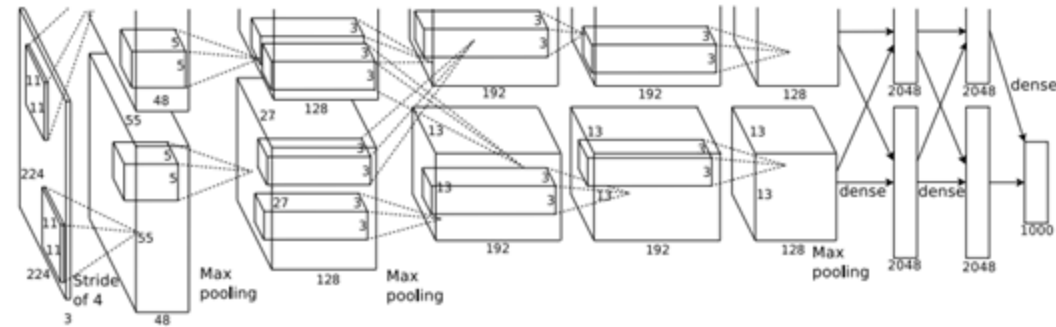
Primeira camada (CONV1): 96 filtros 11×11 com passo 4

Tamanho do volume de saída = **$55 \times 55 \times 96$**

P: Qual é o número total de parâmetros nessa camada?

ConvNets – AlexNet

[Krizhevsky et al. 2012]



Entrada: imagem $227 \times 227 \times 3$

Primeira camada (CONV1): 96 filtros 11×11 com passo 4

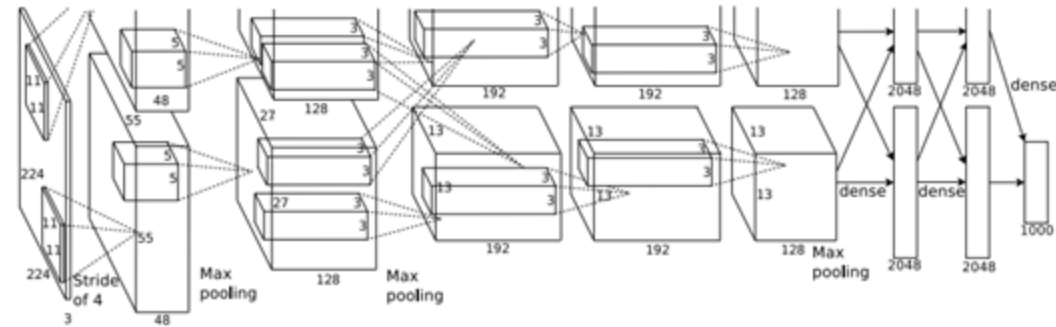
Tamanho do volume de saída = **$55 \times 55 \times 96$**

Número de parâmetros = $(11 \times 11 \times 3 + 1) \times 96 \approx$ **35K**

ConvNets – AlexNet

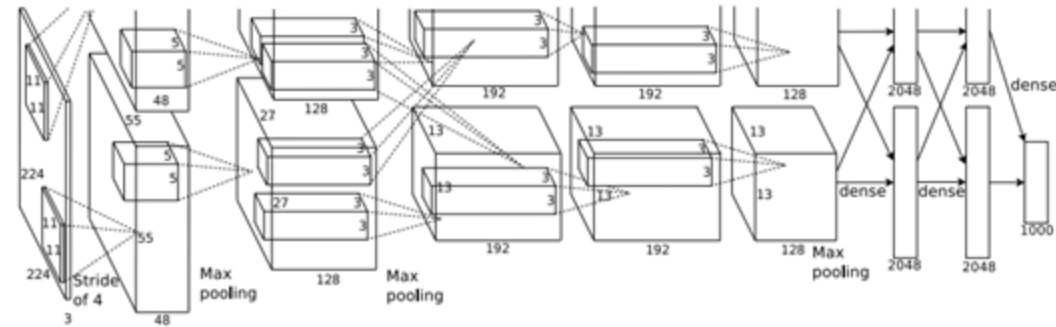
[Krizhevsky et al. 2012]

Entrada: imagem $227 \times 227 \times 3$
Depois de CONV1: $55 \times 55 \times 96$



ConvNets – AlexNet

[Krizhevsky et al. 2012]



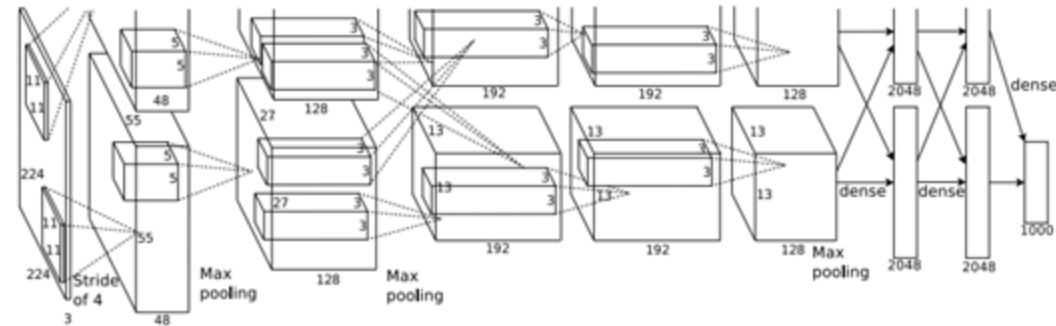
Entrada: imagem $227 \times 227 \times 3$

Depois de CONV1: $55 \times 55 \times 96$

Segunda camada (POOL1): 96 filtros 3×3 com passo 2

ConvNets – AlexNet

[Krizhevsky et al. 2012]



Entrada: imagem 227×227×3

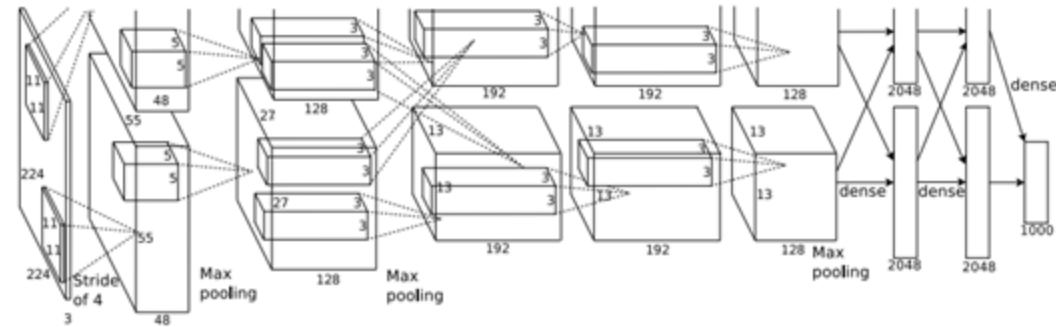
Depois de CONV1: 55×55×96

Segunda camada (POOL1): 96 filtros 3×3 com passo 2

P: Qual é o tamanho do volume de saída? Dica: $(55-3)/2+1 = 27$

ConvNets – AlexNet

[Krizhevsky et al. 2012]



Entrada: imagem $227 \times 227 \times 3$

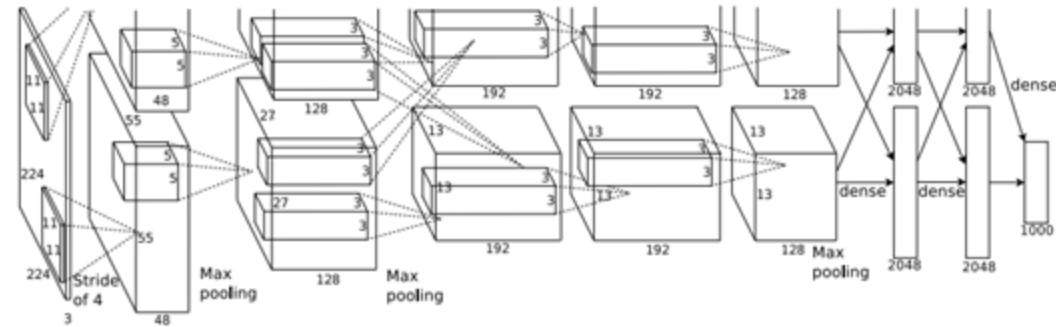
Depois de CONV1: $55 \times 55 \times 96$

Segunda camada (POOL1): 96 filtros 3×3 com passo 2

Tamanho do volume de saída = **$27 \times 27 \times 96$**

ConvNets – AlexNet

[Krizhevsky et al. 2012]



Entrada: imagem $227 \times 227 \times 3$

Depois de CONV1: $55 \times 55 \times 96$

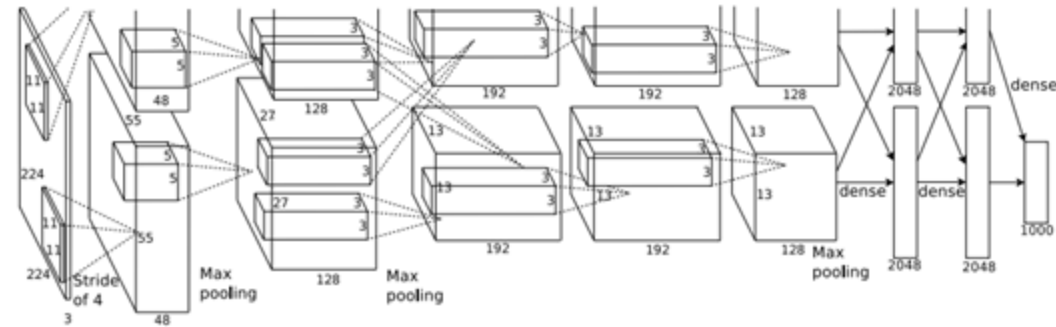
Segunda camada (POOL1): 96 filtros 3×3 com passo 2

Tamanho do volume de saída = **$27 \times 27 \times 96$**

P: Qual é o número total de parâmetros nessa camada?

ConvNets – AlexNet

[Krizhevsky et al. 2012]



Entrada: imagem $227 \times 227 \times 3$

Depois de CONV1: $55 \times 55 \times 96$

Segunda camada (POOL1): 96 filtros 3×3 com passo 2

Tamanho do volume de saída = **$[27 \times 27 \times 96]$**

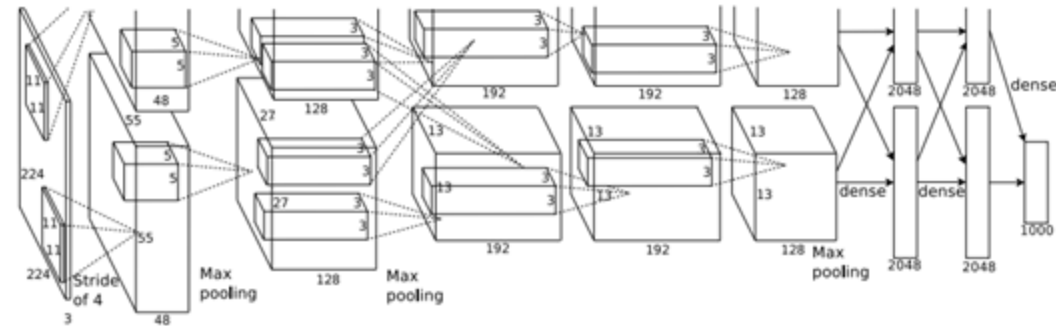
Número de parâmetros = **0** !

ConvNets – AlexNet

[Krizhevsky et al. 2012]

Entrada: imagem $227 \times 227 \times 3$
Depois de CONV1: $55 \times 55 \times 96$
Depois de POOL1: $27 \times 27 \times 96$

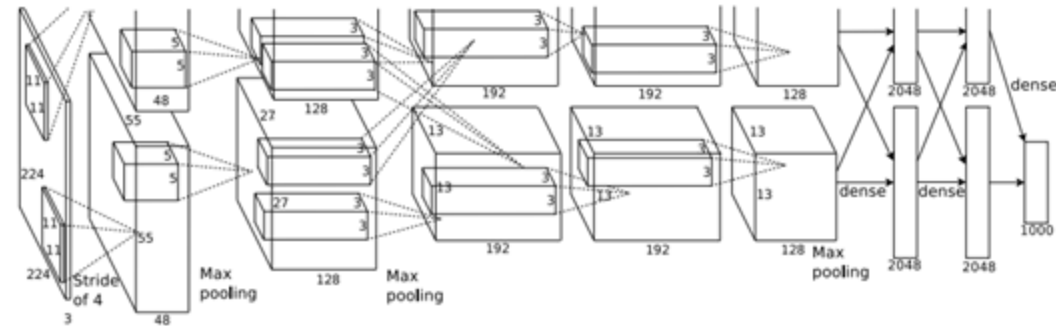
⋮



ConvNets – AlexNet

[Krizhevsky et al. 2012]

Arquitetura da AlexNet:

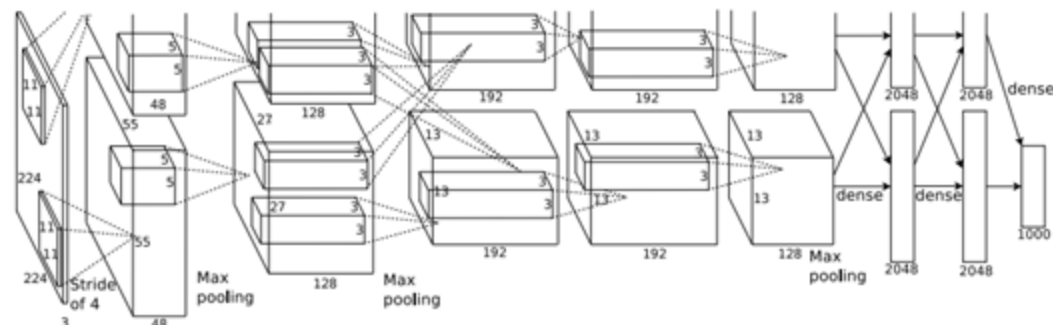


- [227×227×3] Entrada
- [55×55×96] CONV1: 96 filtros 11×11 passo 4, pad 0
- [27×27×96] MAX POOL1: 96 filtros 3×3 passo 2
- [27×27×96] NORM1: camada de normalização
- [27×27×256] CONV2: 256 filtros 5×5 passo 1, pad 2
- [13×13×256] MAX POOL2: 256 filtros 3×3 passo 2
- [13×13×256] NORM2: camada de normalização
- [13×13×384] CONV3: 384 filtros 3×3 passo 1, pad 1
- [13×13×384] CONV4: 384 filtros 3×3 passo 1, pad 1
- [13×13×256] CONV5: 256 filtros 3×3 passo 1, pad 1
- [6×6×256] MAX POOL3: filtros 3×3 passo 2
- [4096] FC6: 4096 neurônios
- [4096] FC7: 4096 neurônios
- [1000] FC8: 1000 neurônios (“scores” de classe)

ConvNets – AlexNet

[Krizhevsky et al. 2012]

Arquitetura da AlexNet:



[227×227×3] Entrada
[55×55×96] **CONV1**: 96 filtros 11×11 passo 4, pad 0
[27×27×96] **MAX POOL1**: 96 filtros 3×3 passo 2
[27×27×96] **NORM1**: camada de normalização
[27×27×256] **CONV2**: 256 filtros 5×5 passo 1, pad 2
[13×13×256] **MAX POOL2**: 256 filtros 3×3 passo 2
[13×13×256] **NORM2**: camada de normalização
[13×13×384] **CONV3**: 384 filtros 3×3 passo 1, pad 1
[13×13×384] **CONV4**: 384 filtros 3×3 passo 1, pad 1
[13×13×256] **CONV5**: 256 filtros 3×3 passo 1, pad 1
[6×6×256] **MAX POOL3**: filtros 3×3 passo 2
[4096] **FC6**: 4096 neurônios
[4096] **FC7**: 4096 neurônios
[1000] **FC8**: 1000 neurônios (“scores” de classe)

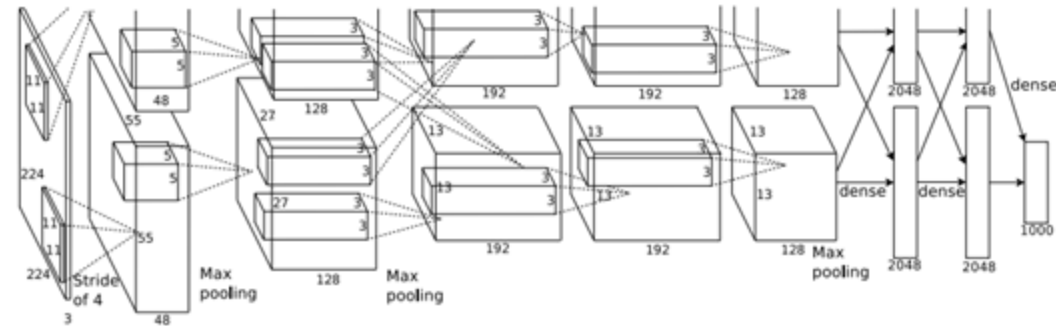
Detalhes:

- Primeiro uso de ReLU

ConvNets – AlexNet

[Krizhevsky et al. 2012]

Arquitetura da AlexNet:



- [227×227×3] Entrada
- [55×55×96] **CONV1**: 96 filtros 11×11 passo 4, pad 0
- [27×27×96] **MAX POOL1**: 96 filtros 3×3 passo 2
- [27×27×96] **NORM1**: camada de normalização
- [27×27×256] **CONV2**: 256 filtros 5×5 passo 1, pad 2
- [13×13×256] **MAX POOL2**: 256 filtros 3×3 passo 2
- [13×13×256] **NORM2**: camada de normalização
- [13×13×384] **CONV3**: 384 filtros 3×3 passo 1, pad 1
- [13×13×384] **CONV4**: 384 filtros 3×3 passo 1, pad 1
- [13×13×256] **CONV5**: 256 filtros 3×3 passo 1, pad 1
- [6×6×256] **MAX POOL3**: filtros 3×3 passo 2
- [4096] **FC6**: 4096 neurônios
- [4096] **FC7**: 4096 neurônios
- [1000] **FC8**: 1000 neurônios (“scores” de classe)

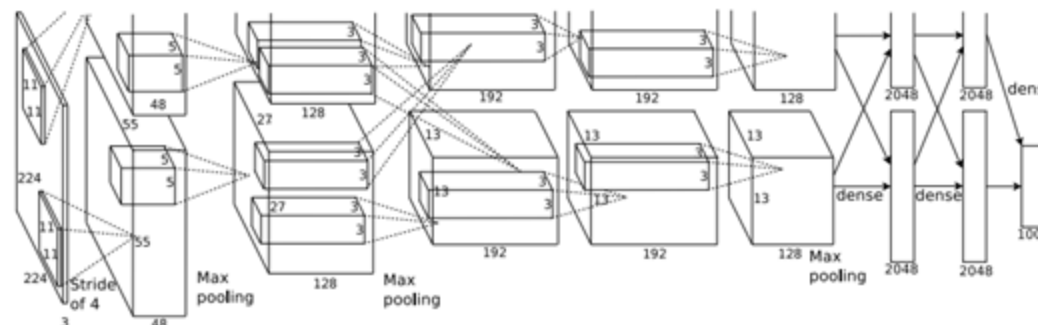
Detalhes:

- Primeiro uso de ReLU
- Uso de camada de normalização (desuso)

ConvNets – AlexNet

[Krizhevsky et al. 2012]

Arquitetura da AlexNet:



[227×227×3] Entrada
[55×55×96] **CONV1**: 96 filtros 11×11 passo 4, pad 0
[27×27×96] **MAX POOL1**: 96 filtros 3×3 passo 2
[27×27×96] **NORM1**: camada de normalização
[27×27×256] **CONV2**: 256 filtros 5×5 passo 1, pad 2
[13×13×256] **MAX POOL2**: 256 filtros 3×3 passo 2
[13×13×256] **NORM2**: camada de normalização
[13×13×384] **CONV3**: 384 filtros 3×3 passo 1, pad 1
[13×13×384] **CONV4**: 384 filtros 3×3 passo 1, pad 1
[13×13×256] **CONV5**: 256 filtros 3×3 passo 1, pad 1
[6×6×256] **MAX POOL3**: filtros 3×3 passo 2
[4096] **FC6**: 4096 neurônios
[4096] **FC7**: 4096 neurônios
[1000] **FC8**: 1000 neurônios (“scores” de classe)

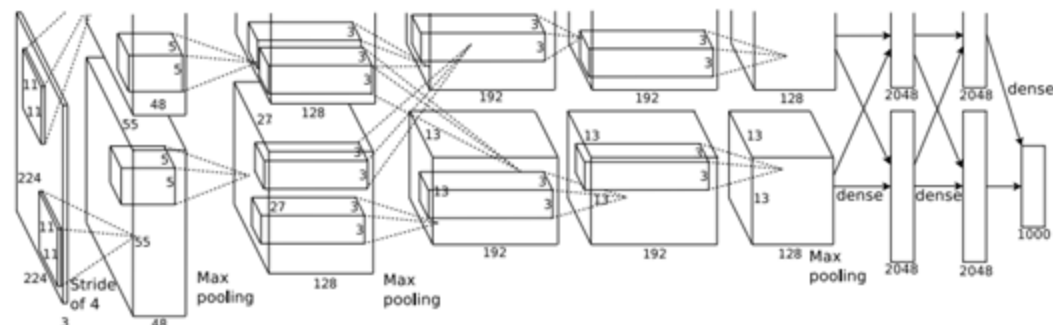
Detalhes:

- Primeiro uso de ReLU
- Uso de camada de normalização (desuso)
- Grande uso de “data augmentation”

ConvNets – AlexNet

[Krizhevsky et al. 2012]

Arquitetura da AlexNet:



[227×227×3] Entrada
[55×55×96] **CONV1**: 96 filtros 11×11 passo 4, pad 0
[27×27×96] **MAX POOL1**: 96 filtros 3×3 passo 2
[27×27×96] **NORM1**: camada de normalização
[27×27×256] **CONV2**: 256 filtros 5×5 passo 1, pad 2
[13×13×256] **MAX POOL2**: 256 filtros 3×3 passo 2
[13×13×256] **NORM2**: camada de normalização
[13×13×384] **CONV3**: 384 filtros 3×3 passo 1, pad 1
[13×13×384] **CONV4**: 384 filtros 3×3 passo 1, pad 1
[13×13×256] **CONV5**: 256 filtros 3×3 passo 1, pad 1
[6×6×256] **MAX POOL3**: filtros 3×3 passo 2
[4096] **FC6**: 4096 neurônios
[4096] **FC7**: 4096 neurônios
[1000] **FC8**: 1000 neurônios (“scores” de classe)

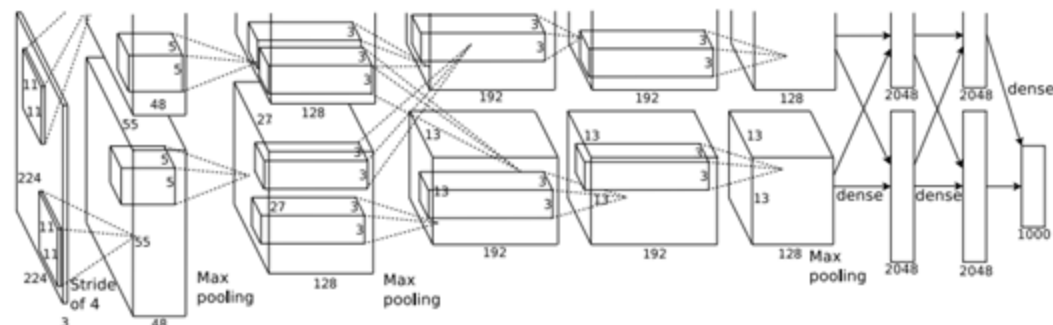
Detalhes:

- Primeiro uso de ReLU
- Uso de camada de normalização (desuso)
- Grande uso de “data augmentation”
- Tamanho de *minibatch* = 128

ConvNets – AlexNet

[Krizhevsky et al. 2012]

Arquitetura da AlexNet:



[227×227×3] Entrada
[55×55×96] **CONV1**: 96 filtros 11×11 passo 4, pad 0
[27×27×96] **MAX POOL1**: 96 filtros 3×3 passo 2
[27×27×96] **NORM1**: camada de normalização
[27×27×256] **CONV2**: 256 filtros 5×5 passo 1, pad 2
[13×13×256] **MAX POOL2**: 256 filtros 3×3 passo 2
[13×13×256] **NORM2**: camada de normalização
[13×13×384] **CONV3**: 384 filtros 3×3 passo 1, pad 1
[13×13×384] **CONV4**: 384 filtros 3×3 passo 1, pad 1
[13×13×256] **CONV5**: 256 filtros 3×3 passo 1, pad 1
[6×6×256] **MAX POOL3**: filtros 3×3 passo 2
[4096] **FC6**: 4096 neurônios
[4096] **FC7**: 4096 neurônios
[1000] **FC8**: 1000 neurônios (“scores” de classe)

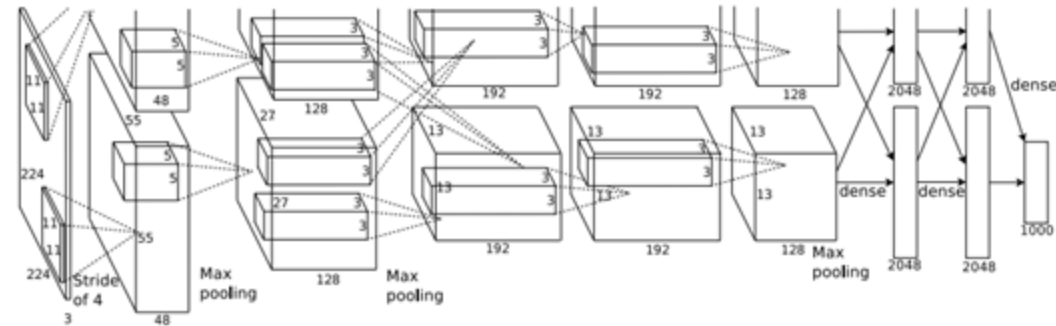
Detalhes:

- Primeiro uso de ReLU
- Uso de camada de normalização (desuso)
- Grande uso de “data augmentation”
- Tamanho de *minibatch* = 128
- *SGD+Momentum* com constante de 0,9

ConvNets – AlexNet

[Krizhevsky et al. 2012]

Arquitetura da AlexNet:



[227×227×3] Entrada
[55×55×96] **CONV1**: 96 filtros 11×11 passo 4, pad 0
[27×27×96] **MAX POOL1**: 96 filtros 3×3 passo 2
[27×27×96] **NORM1**: camada de normalização
[27×27×256] **CONV2**: 256 filtros 5×5 passo 1, pad 2
[13×13×256] **MAX POOL2**: 256 filtros 3×3 passo 2
[13×13×256] **NORM2**: camada de normalização
[13×13×384] **CONV3**: 384 filtros 3×3 passo 1, pad 1
[13×13×384] **CONV4**: 384 filtros 3×3 passo 1, pad 1
[13×13×256] **CONV5**: 256 filtros 3×3 passo 1, pad 1
[6×6×256] **MAX POOL3**: filtros 3×3 passo 2
[4096] **FC6**: 4096 neurônios
[4096] **FC7**: 4096 neurônios
[1000] **FC8**: 1000 neurônios (“scores” de classe)

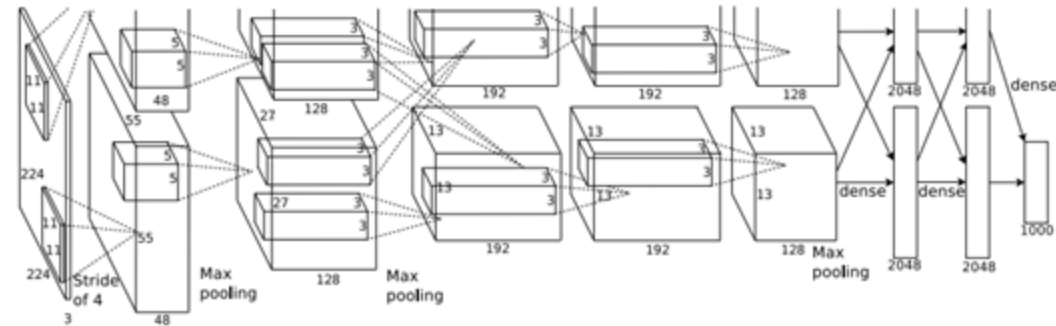
Detalhes:

- Primeiro uso de ReLU
- Uso de camada de normalização (desuso)
- Grande uso de “data augmentation”
- Tamanho de *minibatch* = 128
- *SGD+Momentum* com constante de 0,9
- Tx. Aprendizado = 10^{-2} e reduzida manualmente em “plateaus”

ConvNets – AlexNet

[Krizhevsky et al. 2012]

Arquitetura da AlexNet:



[227×227×3] Entrada
[55×55×96] **CONV1**: 96 filtros 11×11 passo 4, pad 0
[27×27×96] **MAX POOL1**: 96 filtros 3×3 passo 2
[27×27×96] **NORM1**: camada de normalização
[27×27×256] **CONV2**: 256 filtros 5×5 passo 1, pad 2
[13×13×256] **MAX POOL2**: 256 filtros 3×3 passo 2
[13×13×256] **NORM2**: camada de normalização
[13×13×384] **CONV3**: 384 filtros 3×3 passo 1, pad 1
[13×13×384] **CONV4**: 384 filtros 3×3 passo 1, pad 1
[13×13×256] **CONV5**: 256 filtros 3×3 passo 1, pad 1
[6×6×256] **MAX POOL3**: filtros 3×3 passo 2
[4096] **FC6**: 4096 neurônios
[4096] **FC7**: 4096 neurônios
[1000] **FC8**: 1000 neurônios (“scores” de classe)

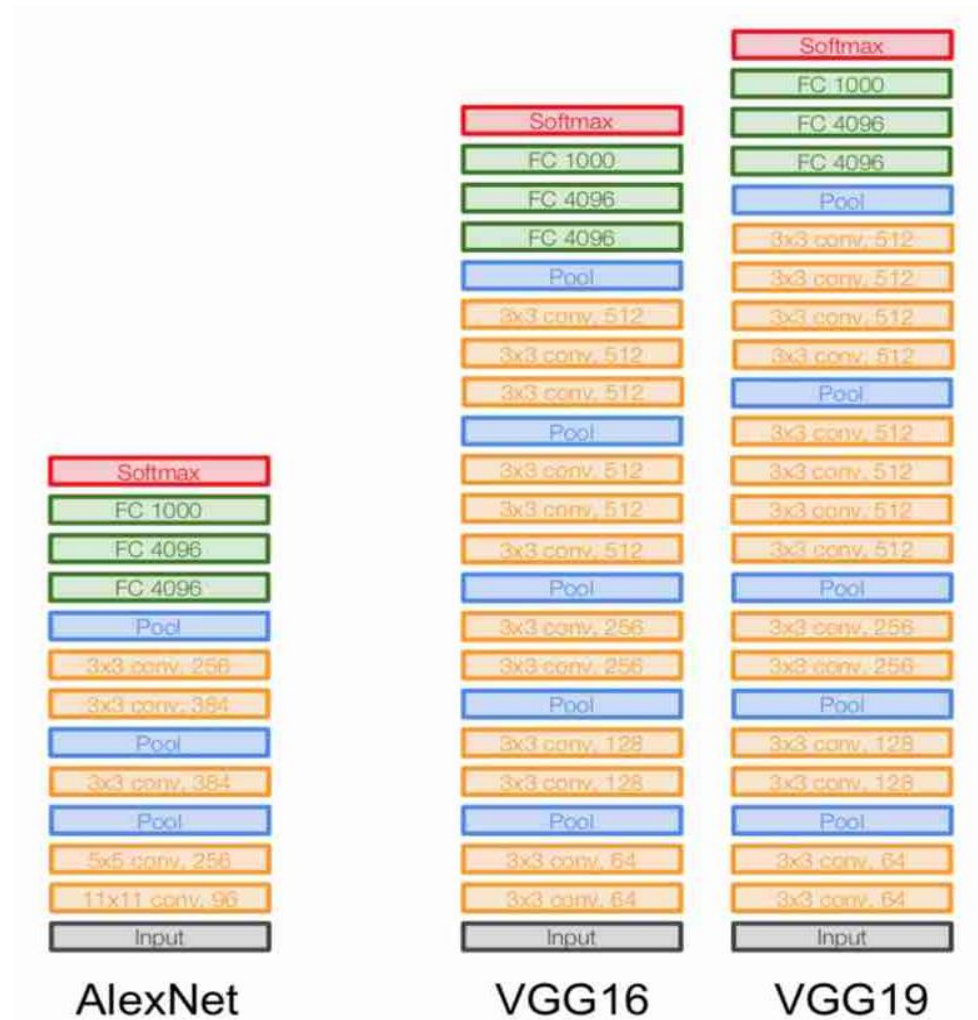
Detalhes:

- Primeiro uso de ReLU
- Uso de camada de normalização (desuso)
- Grande uso de “data augmentation”
- Tamanho de *minibatch* = 128
- SGD+*Momentum* com constante de 0,9
- Tx. Aprendizado = 10^{-2} e reduzida manualmente em “plateaus”
- “Ensemble” de 7 CNNs: 18,2% → 15,4%

ConvNets – VGG

[Simonyan and Zisserman, 2014]

Uso de filtros menores em redes mais profundas



ConvNets – VGG

Uso de filtros menores em redes mais profundas

[Simonyan and Zisserman, 2014]



ConvNets – VGG

[Simonyan and Zisserman, 2014]

Uso de filtros menores em redes mais profundas

Apenas CONV 3×3 passo 1, pad 1
e
MAX POOL 2×2 passo 2



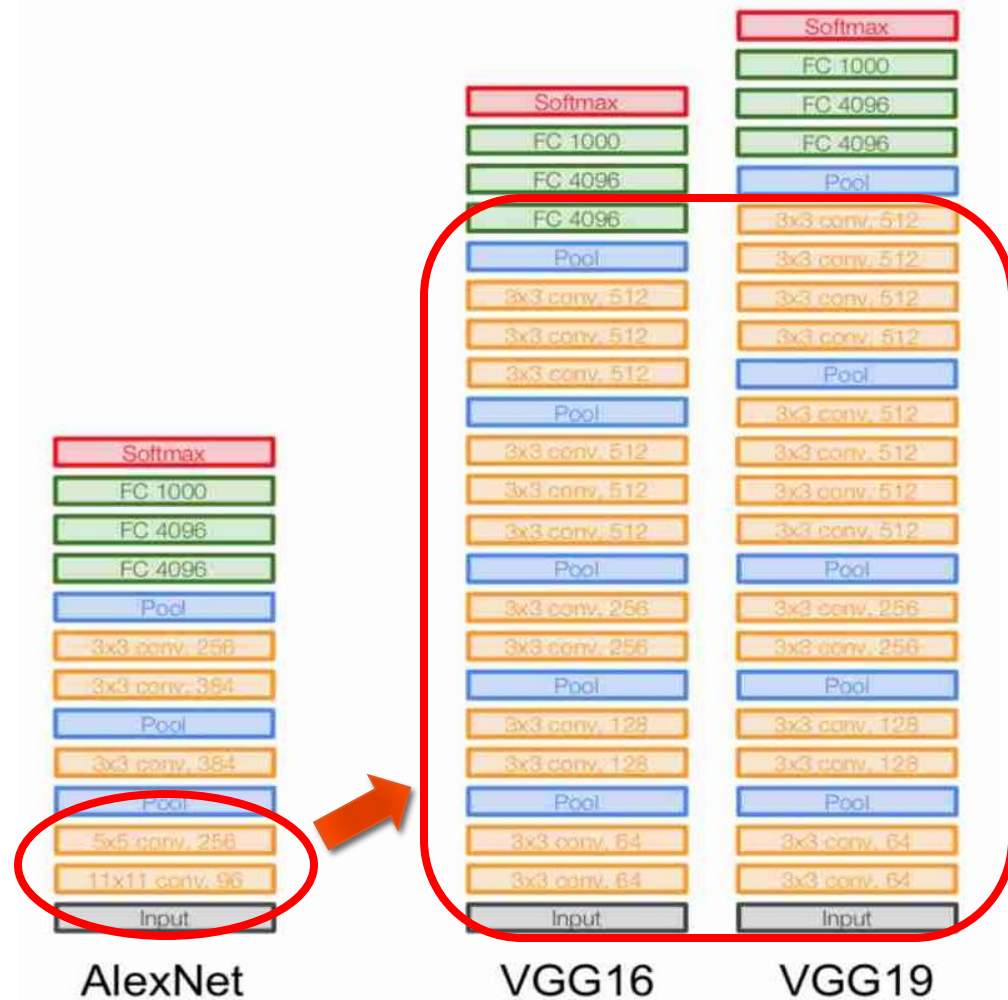
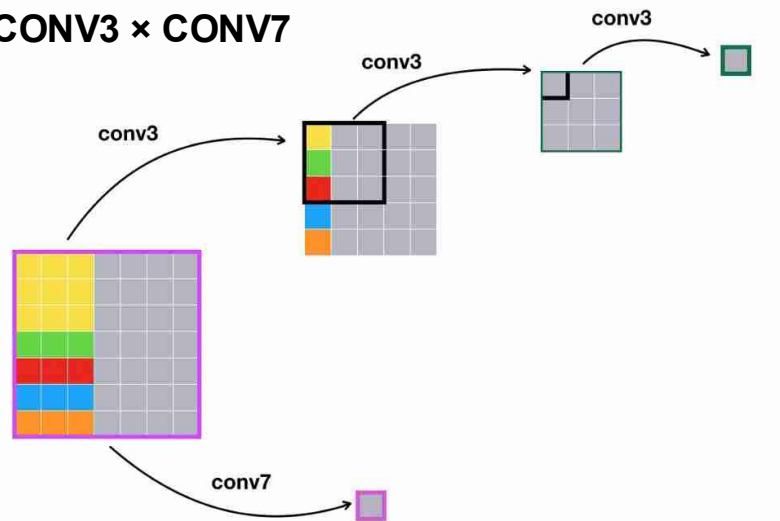
ConvNets – VGG

[Simonyan and Zisserman, 2014]

Uso de filtros menores em redes mais profundas

Apenas CONV 3×3 passo 1, pad 1
e
MAX POOL 2×2 passo 2

CONV3 × CONV7



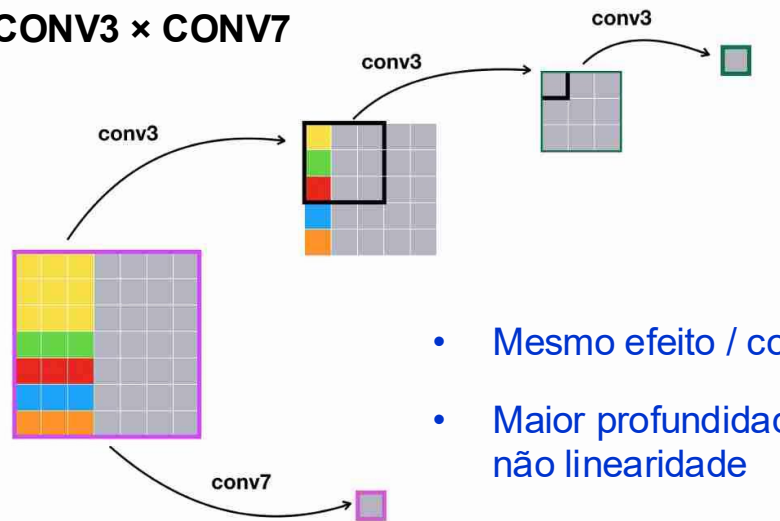
ConvNets – VGG

[Simonyan and Zisserman, 2014]

Uso de filtros menores em redes mais profundas

Apenas CONV 3×3 passo 1, pad 1
e
MAX POOL 2×2 passo 2

CONV3 × CONV7



- Mesmo efeito / cobertura
- Maior profundidade $\Rightarrow$ mais não linearidade
- Menor número de parâmetros



ConvNets – VGG

[Simonyan and Zisserman, 2014]

Uso de filtros menores em redes mais profundas

Apenas CONV 3×3 passo 1, pad 1
e
MAX POOL 2×2 passo 2

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: Number of parameters (in millions).

Network	A, A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

ConvNets – VGG

[Simonyan and Zisserman, 2014]

Uso de filtros menores em redes mais profundas

Apenas CONV 3×3 passo 1, pad 1
e
MAX POOL 2×2 passo 2

Melhor modelo

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: Number of parameters (in millions).

Network	A, A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

ConvNets – VGG

[Simonyan and Zisserman, 2014]

Uso de filtros menores em redes mais profundas

Apenas CONV 3×3 passo 1, pad 1
e
MAX POOL 2×2 passo 2

Melhor modelo

Erro (top 5) = 11,2% - ILSVRC 2013



Erro (top 5) = 7,3%

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: Number of parameters (in millions).

Network	A, A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

ConvNets – VGG

[Simonyan and Zisserman, 2014]

Uso de filtros menores em redes mais profundas

Apenas CONV 3×3 passo 1, pad 1
e
MAX POOL 2×2 passo 2

Melhor modelo

Erro (top 5) = 11,2% - ILSVRC 2013



Erro (top 5) = 7,3%

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: Number of parameters (in millions)

Network	A, A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

138M

ConvNets – VGG

ENTRADA: [224x224x3] **memória:** $224*224*3=150K$ **params:** 0 (sem contar vieses)

CONV3-64: [224x224x64] **memória:** $224*224*64=3.2M$ **params:** $(3*3*3)*64 = 1.728$

CONV3-64: [224x224x64] **memória:** $224*224*64=3.2M$ **params:** $(3*3*64)*64 = 36.864$

POOL2: [112x112x64] **memória:** $112*112*64=800K$ **params:** 0

CONV3-128: [112x112x128] **memória:** $112*112*128=1.6M$ **params:** $(3*3*64)*128 = 73.728$

CONV3-128: [112x112x128] **memória:** $112*112*128=1.6M$ **params:** $(3*3*128)*128 = 147.456$

POOL2: [56x56x128] **memória:** $56*56*128=400K$ **params:** 0

CONV3-256: [56x56x256] **memória:** $56*56*256=800K$ **params:** $(3*3*128)*256 = 294.912$

CONV3-256: [56x56x256] **memória:** $56*56*256=800K$ **params:** $(3*3*256)*256 = 589.824$

CONV3-256: [56x56x256] **memória:** $56*56*256=800K$ **params:** $(3*3*256)*256 = 589.824$

POOL2: [28x28x256] **memória:** $28*28*256=200K$ **params:** 0

CONV3-512: [28x28x512] **memória:** $28*28*512=400K$ **params:** $(3*3*256)*512 = 1.179.648$

CONV3-512: [28x28x512] **memória:** $28*28*512=400K$ **params:** $(3*3*512)*512 = 2.359.296$

CONV3-512: [28x28x512] **memória:** $28*28*512=400K$ **params:** $(3*3*512)*512 = 2.359.296$

POOL2: [14x14x512] **memória:** $14*14*512=100K$ **params:** 0

CONV3-512: [14x14x512] **memória:** $14*14*512=100K$ **params:** $(3*3*512)*512 = 2.359.296$

CONV3-512: [14x14x512] **memória:** $14*14*512=100K$ **params:** $(3*3*512)*512 = 2.359.296$

CONV3-512: [14x14x512] **memória:** $14*14*512=100K$ **params:** $(3*3*512)*512 = 2.359.296$

POOL2: [7x7x512] **memória:** $7*7*512=25K$ **params:** 0

FC: [1x1x4096] **memória:** 4096 **params:** $7*7*512*4096 = 102.760.448$

FC: [1x1x4096] **memória:** 4096 **params:** $4096*4096 = 16.777.216$

FC: [1x1x1000] **memória:** 1000 **params:** $4096*1000 = 4.096.000$

[Simonyan and Zisserman, 2014]

ConvNet Configuration			
B	C	D	E
13 weight layers	16 weight layers	16 weight layers	19 weight layers
Input (224 × 224 RGB image)			
conv3-64	conv3-64	conv3-64	conv3-64
conv3-64	conv3-64	conv3-64	conv3-64
maxpool			
conv3-128	conv3-128	conv3-128	conv3-128
conv3-128	conv3-128	conv3-128	conv3-128
maxpool			
conv3-256	conv3-256	conv3-256	conv3-256
conv3-256	conv3-256	conv3-256	conv3-256
	conv1-256	conv3-256	conv3-256
maxpool			
conv3-512	conv3-512	conv3-512	conv3-512
conv3-512	conv3-512	conv3-512	conv3-512
	conv1-512	conv3-512	conv3-512
maxpool			
conv3-512	conv3-512	conv3-512	conv3-512
conv3-512	conv3-512	conv3-512	conv3-512
	conv1-512	conv3-512	conv3-512
maxpool			
FC-4096			
FC-4096			
FC-1000			
soft-max			

ConvNets – VGG

[Simonyan and Zisserman, 2014]

ENTRADA: [224x224x3] **memória:** $224*224*3=150K$ **params:** 0 (sem contar vieses)

CONV3-64: [224x224x64] **memória:** $224*224*64=3.2M$ **params:** $(3*3*3)*64 = 1.728$

CONV3-64: [224x224x64] **memória:** $224*224*64=3.2M$ **params:** $(3*3*64)*64 = 36.864$

POOL2: [112x112x64] **memória:** $112*112*64=800K$ **params:** 0

CONV3-128: [112x112x128] **memória:** $112*112*128=1.6M$ **params:** $(3*3*64)*128 = 73.728$

CONV3-128: [112x112x128] **memória:** $112*112*128=1.6M$ **params:** $(3*3*128)*128 = 147.456$

POOL2: [56x56x128] **memória:** $56*56*128=400K$ **params:** 0

CONV3-256: [56x56x256] **memória:** $56*56*256=800K$ **params:** $(3*3*128)*256 = 294.912$

CONV3-256: [56x56x256] **memória:** $56*56*256=800K$ **params:** $(3*3*256)*256 = 589.824$

CONV3-256: [56x56x256] **memória:** $56*56*256=800K$ **params:** $(3*3*256)*256 = 589.824$

POOL2: [28x28x256] **memória:** $28*28*256=200K$ **params:** 0

CONV3-512: [28x28x512] **memória:** $28*28*512=400K$ **params:** $(3*3*256)*512 = 1.179.648$

CONV3-512: [28x28x512] **memória:** $28*28*512=400K$ **params:** $(3*3*512)*512 = 2.359.296$

CONV3-512: [28x28x512] **memória:** $28*28*512=400K$ **params:** $(3*3*512)*512 = 2.359.296$

POOL2: [14x14x512] **memória:** $14*14*512=100K$ **params:** 0

CONV3-512: [14x14x512] **memória:** $14*14*512=100K$ **params:** $(3*3*512)*512 = 2.359.296$

CONV3-512: [14x14x512] **memória:** $14*14*512=100K$ **params:** $(3*3*512)*512 = 2.359.296$

CONV3-512: [14x14x512] **memória:** $14*14*512=100K$ **params:** $(3*3*512)*512 = 2.359.296$

POOL2: [7x7x512] **memória:** $7*7*512=25K$ **params:** 0

FC: [1x1x4096] **memória:** 4096 **params:** $7*7*512*4096 = 102.760.448$

FC: [1x1x4096] **memória:** 4096 **params:** $4096*4096 = 16.777.216$

FC: [1x1x1000] **memória:** 1000 **params:** $4096*1000 = 4.096.000$

TOTAL memória: 24M * 4 bytes $\sim$ 93MB / imagem (apenas *forward* ! $\sim$ x2 para *backward*)

TOTAL params: 138M parâmetros

ConvNet Configuration			
B	C	D	E
13 weight layers	16 weight layers	16 weight layers	19 weight layers
Input (224 × 224 RGB image)			
conv3-64	conv3-64	conv3-64	conv3-64
conv3-64	conv3-64	conv3-64	conv3-64
maxpool			
conv3-128	conv3-128	conv3-128	conv3-128
conv3-128	conv3-128	conv3-128	conv3-128
maxpool			
conv3-256	conv3-256	conv3-256	conv3-256
conv3-256	conv3-256	conv3-256	conv3-256
	conv1-256	conv3-256	conv3-256
maxpool			
conv3-512	conv3-512	conv3-512	conv3-512
conv3-512	conv3-512	conv3-512	conv3-512
	conv1-512	conv3-512	conv3-512
maxpool			
conv3-512	conv3-512	conv3-512	conv3-512
conv3-512	conv3-512	conv3-512	conv3-512
	conv1-512	conv3-512	conv3-512
maxpool			
FC-4096			
FC-4096			
FC-1000			
soft-max			

ConvNets – VGG

[Simonyan and Zisserman, 2014]

ENTRADA: [224x224x3] memória: $224*224*3=150K$ params: 0 (sem contar vieses)
CONV3-64: [224x224x64] memória: $224*224*64=3.2M$ params: $(3*3*3)*64 = 1.728$
CONV3-64: [224x224x64] memória: $224*224*64=3.2M$ params: $(3*3*64)*64 = 36.864$
POOL2: [112x112x64] memória: $112*112*64=800K$ params: 0
CONV3-128: [112x112x128] memória: $112*112*128=1.6M$ params: $(3*3*64)*128 = 73.728$
CONV3-128: [112x112x128] memória: $112*112*128=1.6M$ params: $(3*3*128)*128 = 147.456$
POOL2: [56x56x128] memória: $56*56*128=400K$ params: 0
CONV3-256: [56x56x256] memória: $56*56*256=800K$ params: $(3*3*128)*256 = 294.912$
CONV3-256: [56x56x256] memória: $56*56*256=800K$ params: $(3*3*256)*256 = 589.824$
CONV3-256: [56x56x256] memória: $56*56*256=800K$ params: $(3*3*256)*256 = 589.824$
POOL2: [28x28x256] memória: $28*28*256=200K$ params: 0
CONV3-512: [28x28x512] memória: $28*28*512=400K$ params: $(3*3*256)*512 = 1.179.648$
CONV3-512: [28x28x512] memória: $28*28*512=400K$ params: $(3*3*512)*512 = 2.359.296$
CONV3-512: [28x28x512] memória: $28*28*512=400K$ params: $(3*3*512)*512 = 2.359.296$
POOL2: [14x14x512] memória: $14*14*512=100K$ params: 0
CONV3-512: [14x14x512] memória: $14*14*512=100K$ params: $(3*3*512)*512 = 2.359.296$
CONV3-512: [14x14x512] memória: $14*14*512=100K$ params: $(3*3*512)*512 = 2.359.296$
CONV3-512: [14x14x512] memória: $14*14*512=100K$ params: $(3*3*512)*512 = 2.359.296$
POOL2: [7x7x512] memória: $7*7*512=25K$ params: 0
FC: [1x1x4096] memória: 4096 params: $7*7*512*4096 = 102.760.448$
FC: [1x1x4096] memória: 4096 params: $4096*4096 = 16.777.216$
FC: [1x1x1000] memória: 1000 params: $4096*1000 = 4.096.000$

OBS:

Maior parte da memória está nas camadas CONV iniciais

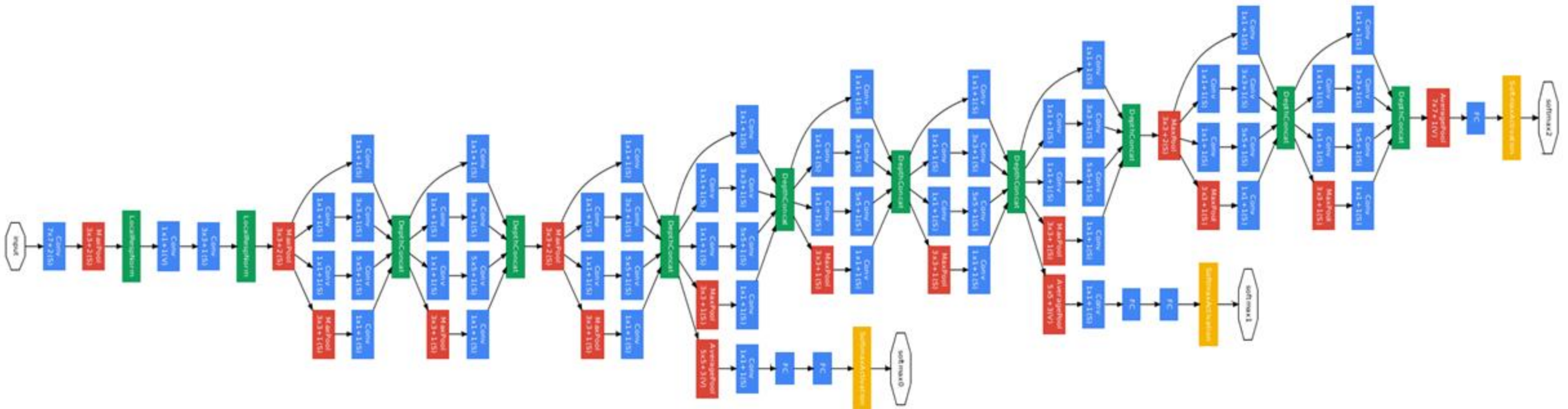
Maioria dos params estão nas camadas FC finais

TOTAL memória: $24M * 4 \text{ bytes} \approx 93MB$ / imagem (apenas *forward*! ~ $\times 2$ para *backward*)
TOTAL params: 138M parâmetros

ConvNets – GoogLeNet

[Szegedy et al., 2014]

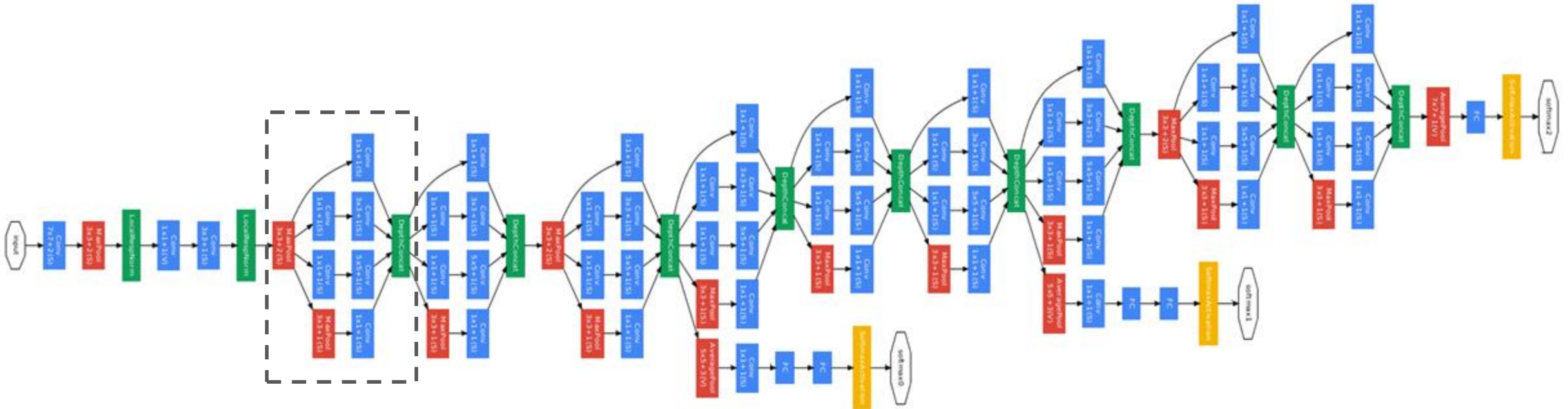
Uso de vários filtros em paralelo



ConvNets – GoogLeNet

[Szegedy et al., 2014]

Uso de vários filtros em paralelo

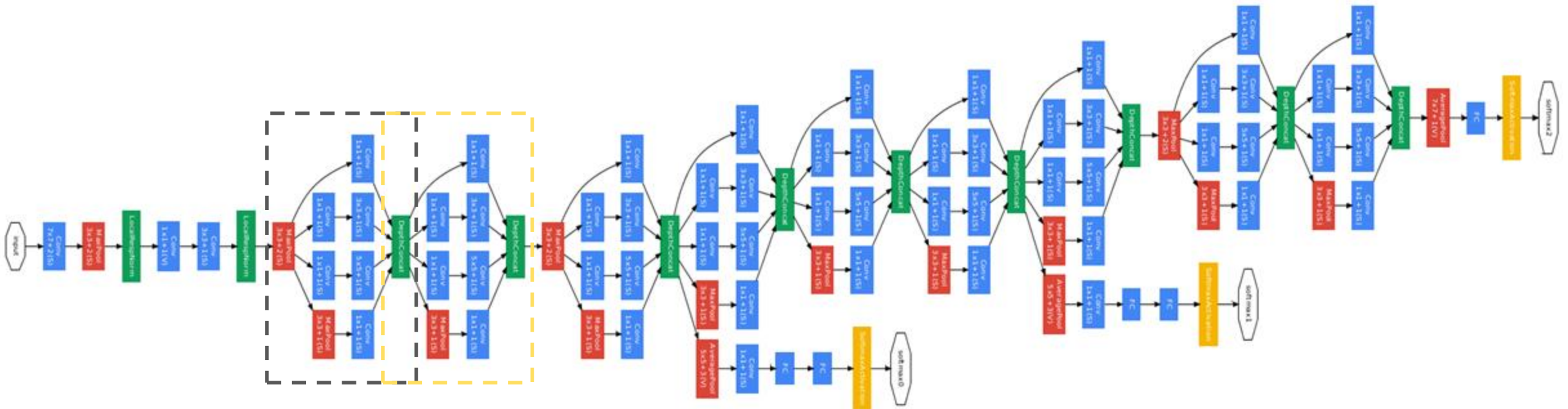


Módulo *Inception*

ConvNets – GoogLeNet

[Szegedy et al., 2014]

Uso de vários filtros em paralelo

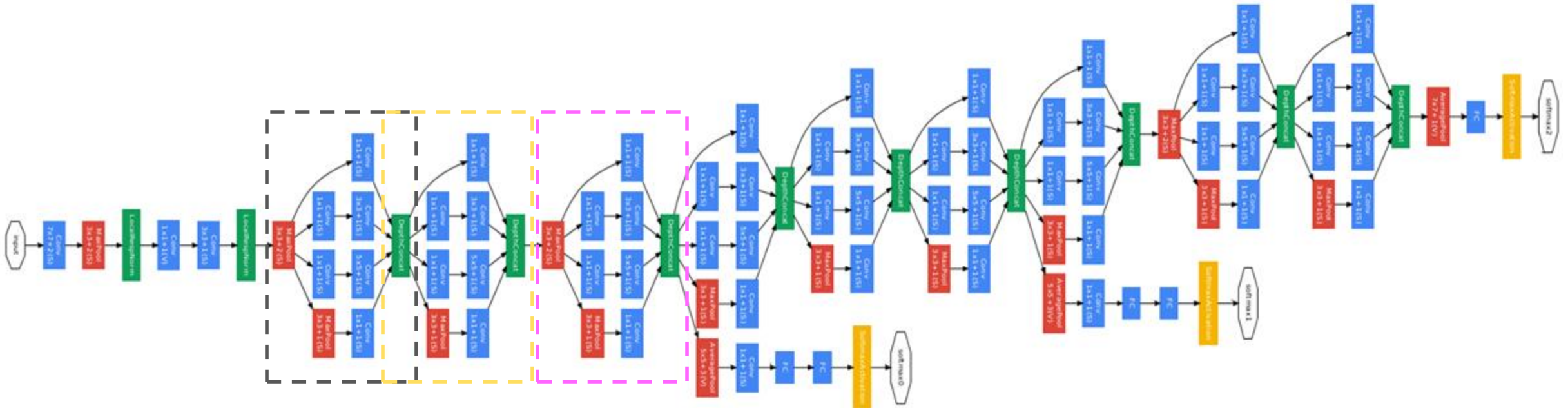


Módulo *Inception*

ConvNets – GoogLeNet

[Szegedy et al., 2014]

Uso de vários filtros em paralelo

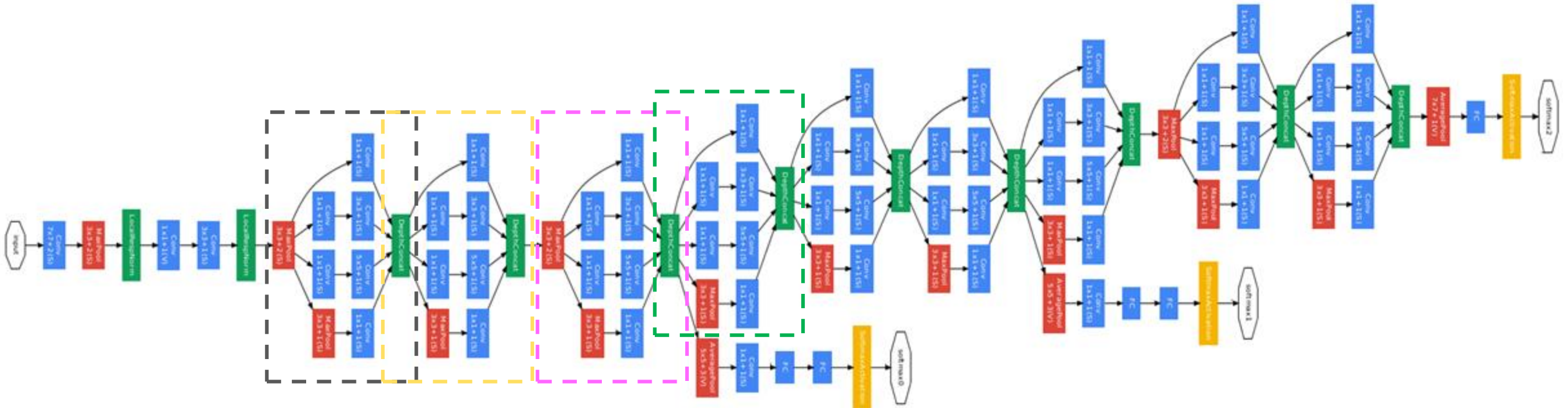


Módulo *Inception*

ConvNets – GoogLeNet

[Szegedy et al., 2014]

Uso de vários filtros em paralelo

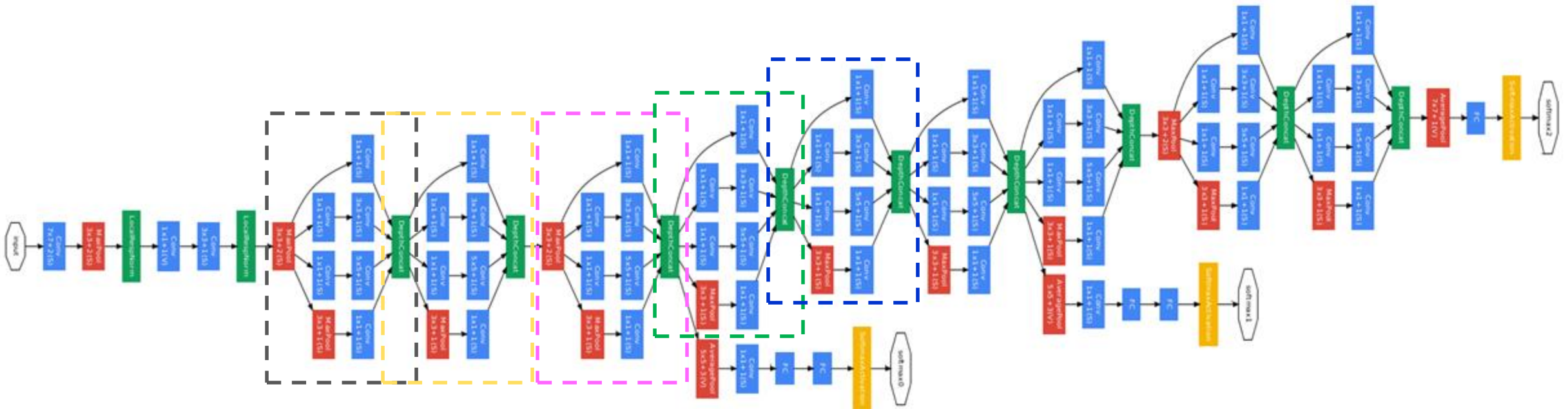


Módulo *Inception*

ConvNets – GoogLeNet

[Szegedy et al., 2014]

Uso de vários filtros em paralelo

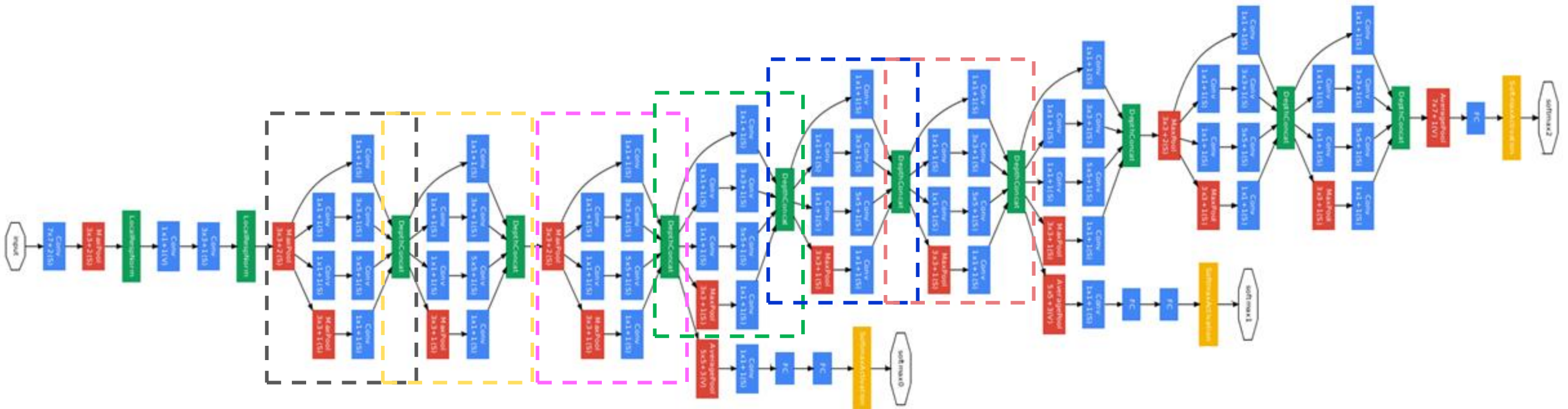


Módulo *Inception*

ConvNets – GoogLeNet

[Szegedy et al., 2014]

Uso de vários filtros em paralelo

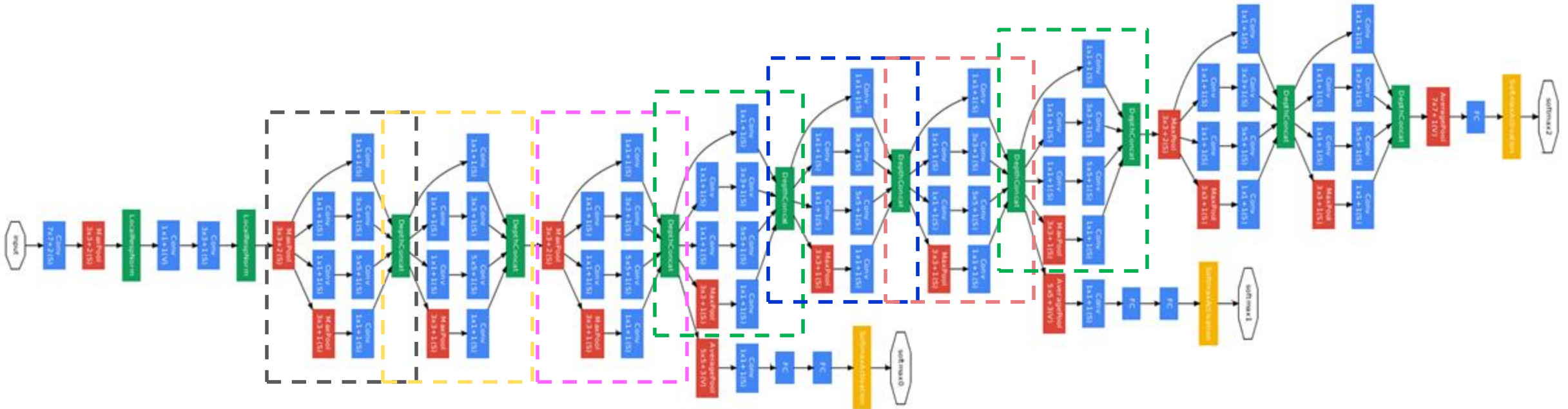


Módulo *Inception*

ConvNets – GoogLeNet

[Szegedy et al., 2014]

Uso de vários filtros em paralelo

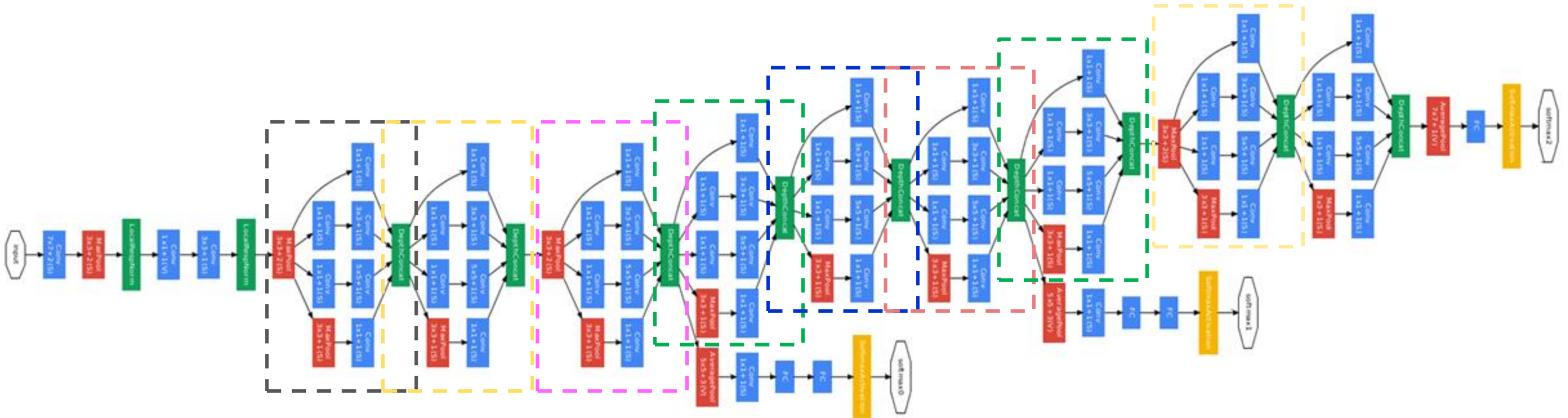


Módulo *Inception*

ConvNets – GoogLeNet

[Szegedy et al., 2014]

Uso de vários filtros em paralelo

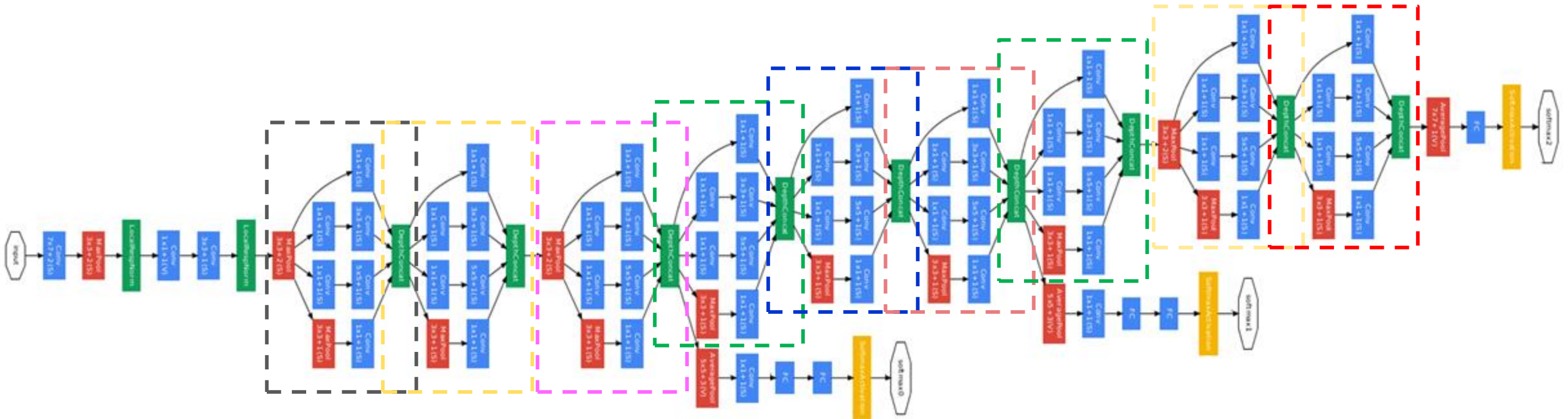


Módulo *Inception*

ConvNets – GoogLeNet

[Szegedy et al., 2014]

Uso de vários filtros em paralelo

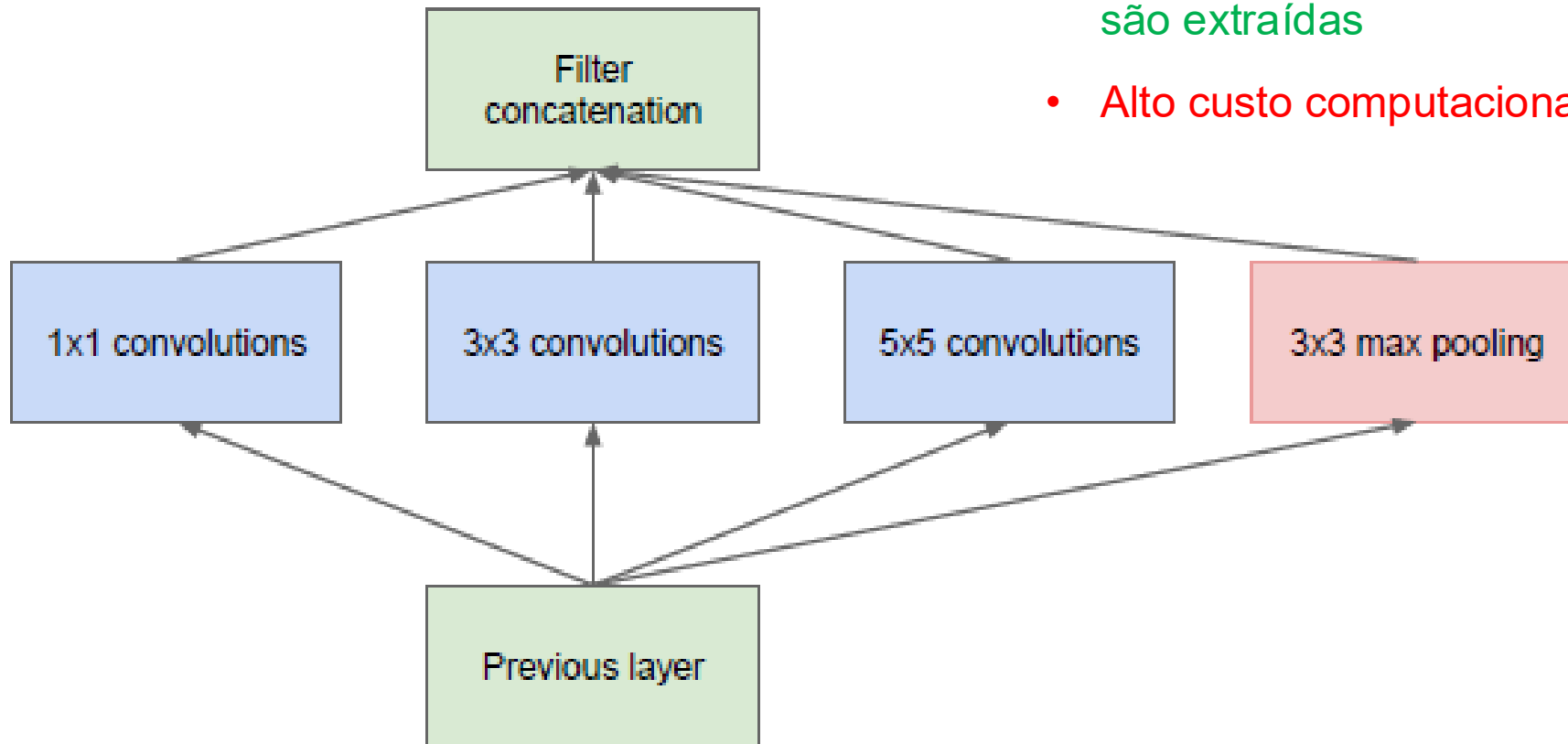


Módulo *Inception*

ConvNets – GoogLeNet

[Szegedy et al., 2014]

Módulo *Inception* – Versão básica

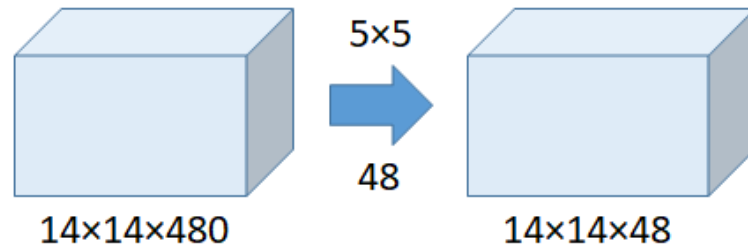


- Diferentes tipos de características são extraídas
- Alto custo computacional

ConvNets – GoogLeNet

[Szegedy et al., 2014]

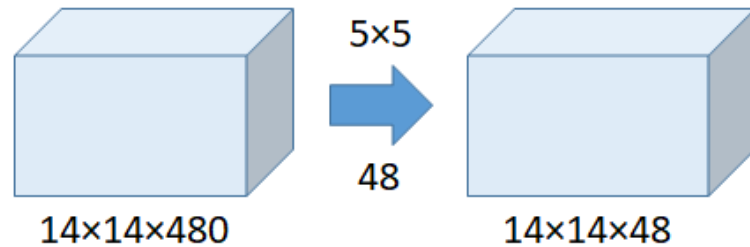
Módulo *Inception* – Versão básica



ConvNets – GoogLeNet

[Szegedy et al., 2014]

Módulo *Inception* – Versão básica

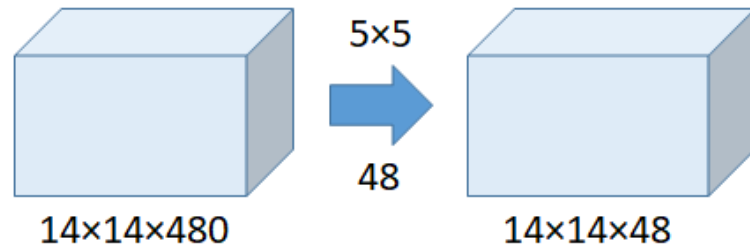


$$\begin{aligned}\text{Número de operações} &= (14 \times 14 \times 48) \times (5 \times 5 \times 480) = \\ &= 112,9\text{M}\end{aligned}$$

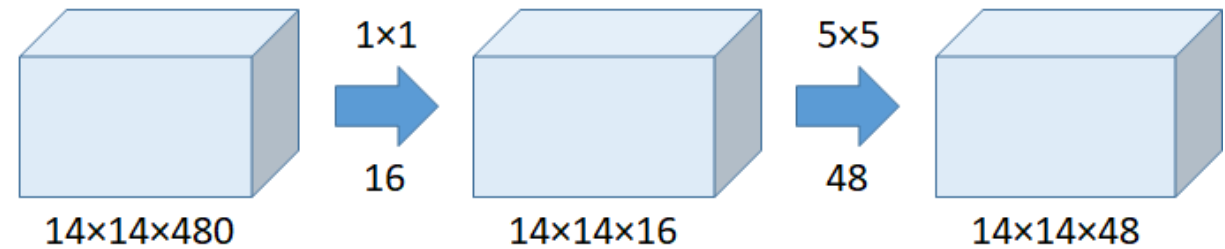
ConvNets – GoogLeNet

[Szegedy et al., 2014]

Módulo *Inception* – Versão básica



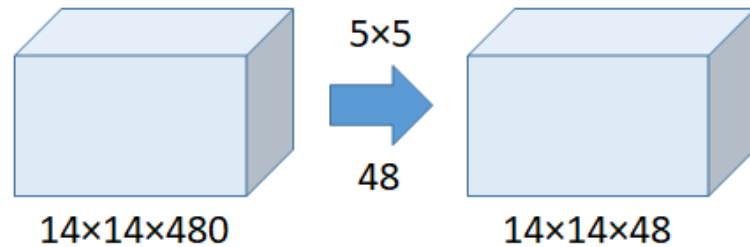
Número de operações = $(14 \times 14 \times 48) \times (5 \times 5 \times 480) =$
 $= 112,9\text{M}$



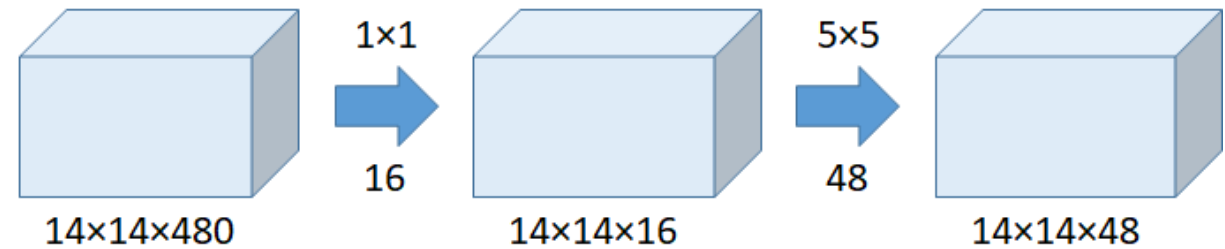
ConvNets – GoogLeNet

[Szegedy et al., 2014]

Módulo *Inception* – Versão básica



Número de operações = $(14 \times 14 \times 48) \times (5 \times 5 \times 480) =$
 $= 112,9\text{M}$

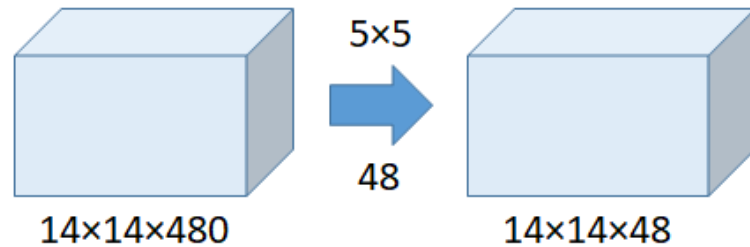


Núm. operações para $1 \times 1 = (14 \times 14 \times 16) \times (1 \times 1 \times 480) = 1,5\text{M}$

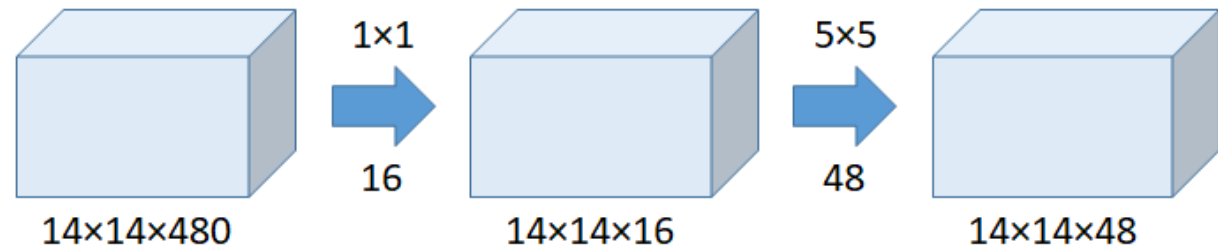
ConvNets – GoogLeNet

[Szegedy et al., 2014]

Módulo *Inception* – Versão básica



Número de operações = $(14 \times 14 \times 48) \times (5 \times 5 \times 480) =$
 $= 112,9\text{M}$



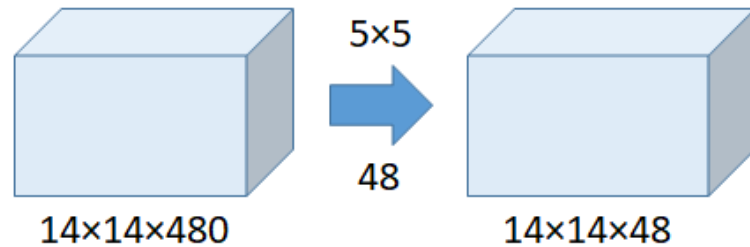
Núm. operações para $1 \times 1 = (14 \times 14 \times 16) \times (1 \times 1 \times 480) = 1,5\text{M}$

Núm. operações para $5 \times 5 = (14 \times 14 \times 48) \times (5 \times 5 \times 16) = 3,8\text{M}$

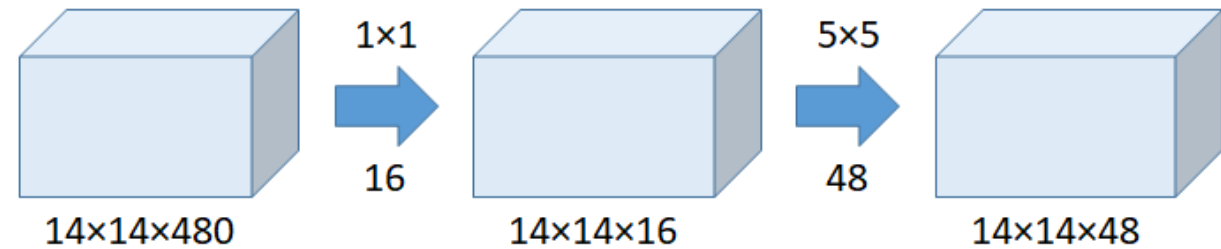
ConvNets – GoogLeNet

[Szegedy et al., 2014]

Módulo *Inception* – Versão básica



Número de operações = $(14 \times 14 \times 48) \times (5 \times 5 \times 480) =$
= **112,9M**



Núm. operações para 1×1 = $(14 \times 14 \times 16) \times (1 \times 1 \times 480) = 1,5\text{M}$

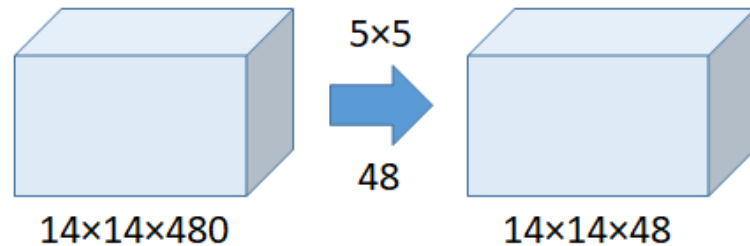
Núm. operações para 5×5 = $(14 \times 14 \times 48) \times (5 \times 5 \times 16) = 3,8\text{M}$

Núm. total de operações = $1,5\text{M} + 3,8\text{M} =$ **5,3M**

ConvNets – GoogLeNet

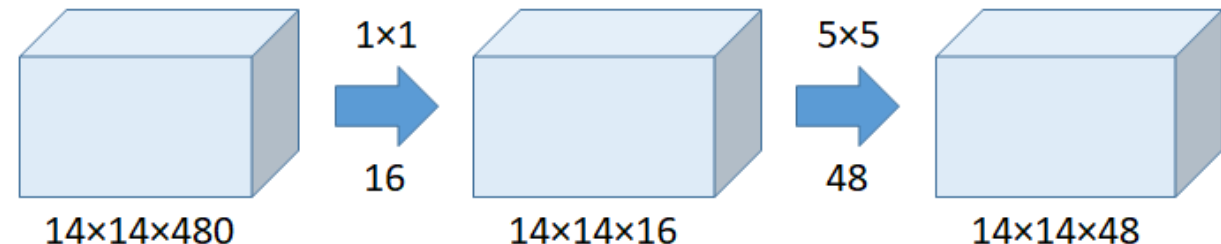
[Szegedy et al., 2014]

Módulo *Inception* – Versão básica



- CONV 1x1 redução de dimensionalidade
- Redução do custo computacional
- Aumento do número de parâmetros

Número de operações = $(14 \times 14 \times 48) \times (5 \times 5 \times 480) =$
 $= \mathbf{112,9M}$



Núm. operações para $1 \times 1 = (14 \times 14 \times 16) \times (1 \times 1 \times 480) = 1,5M$

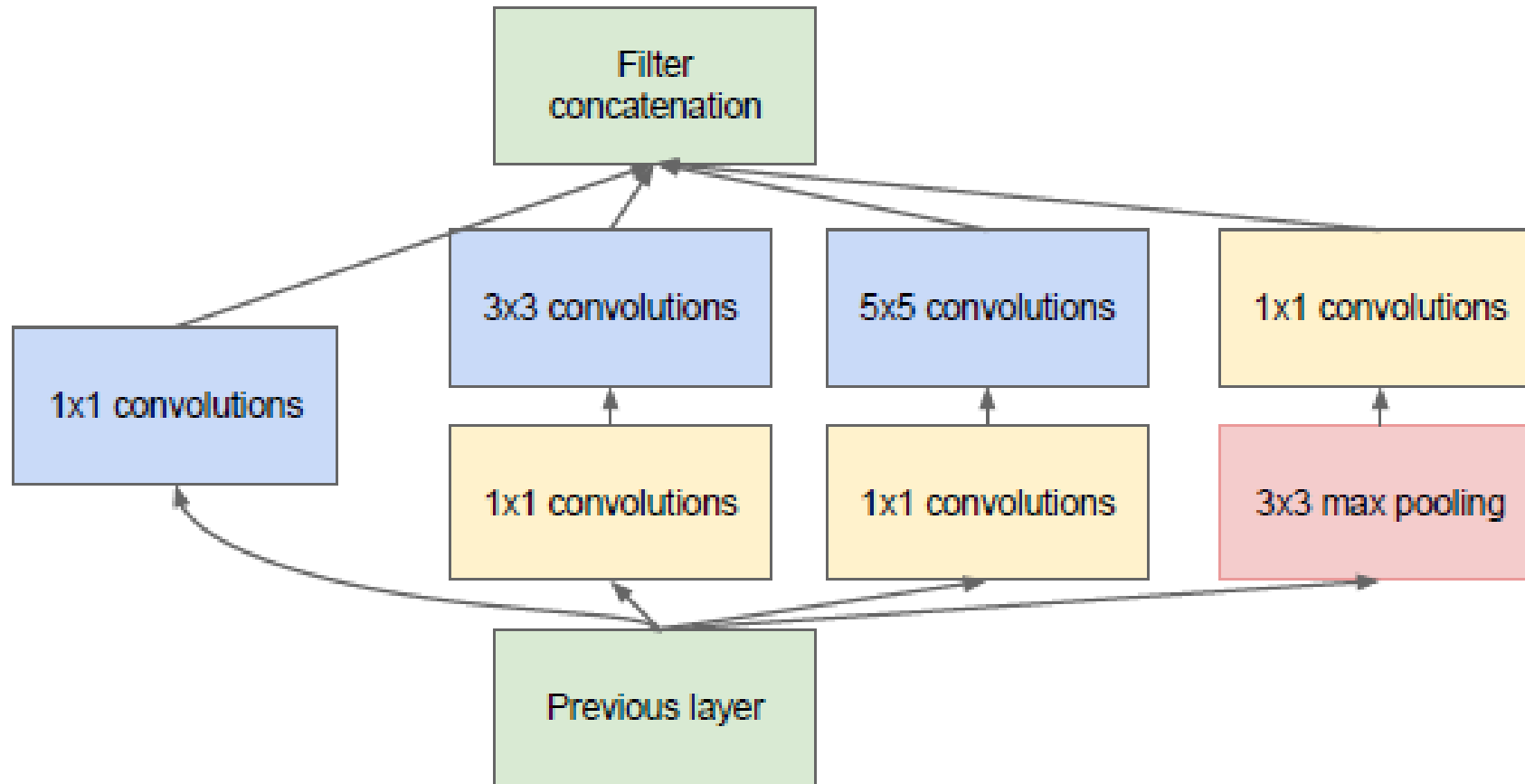
Núm. operações para $5 \times 5 = (14 \times 14 \times 48) \times (5 \times 5 \times 16) = 3,8M$

Núm. total de operações = $1,5M + 3,8M = \mathbf{5,3M}$

ConvNets – GoogLeNet

[Szegedy et al., 2014]

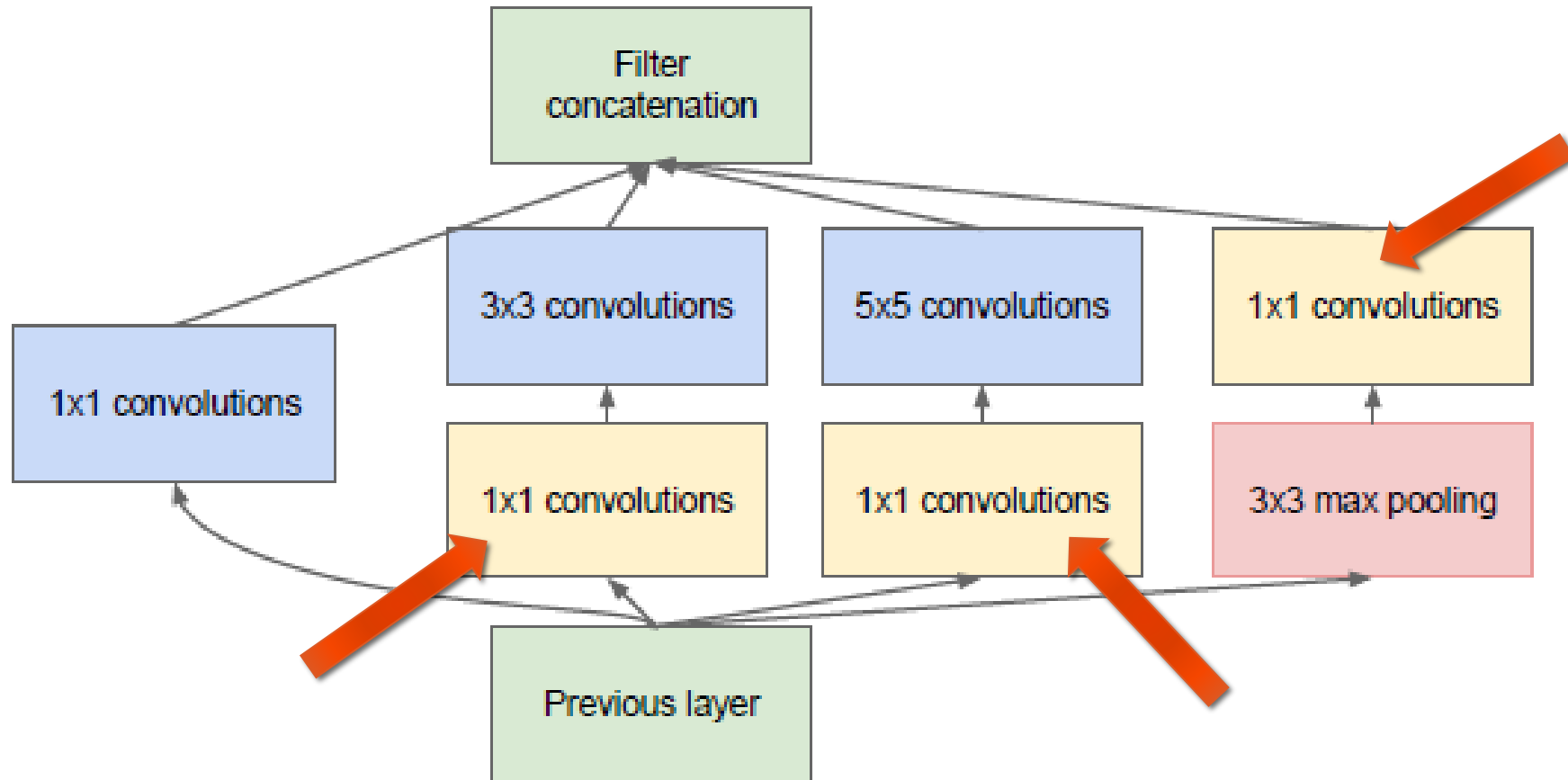
Módulo *Inception* – Versão completa



ConvNets – GoogLeNet

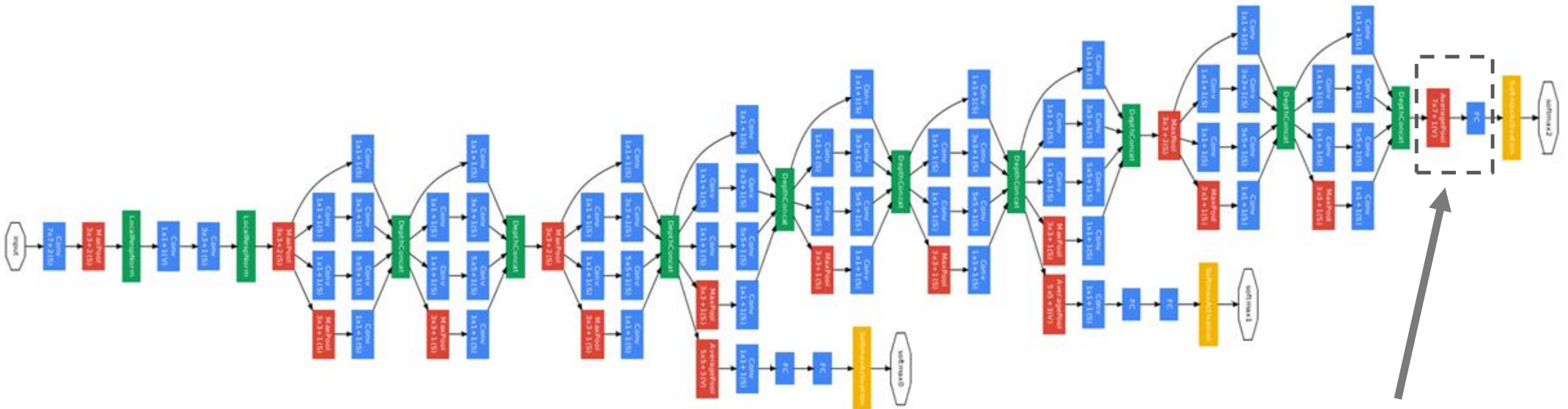
[Szegedy et al., 2014]

Módulo *Inception* – Versão completa



ConvNets – GoogLeNet

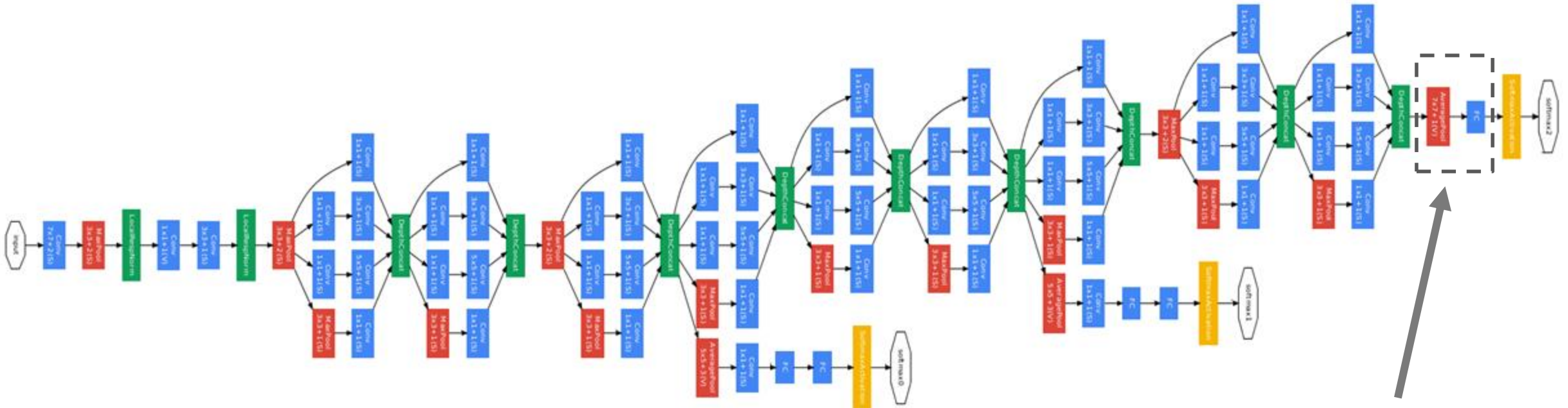
[Szegedy et al., 2014]



Agrupamento
pela Média
(average pooling)

ConvNets – GoogLeNet

[Szegedy et al., 2014]

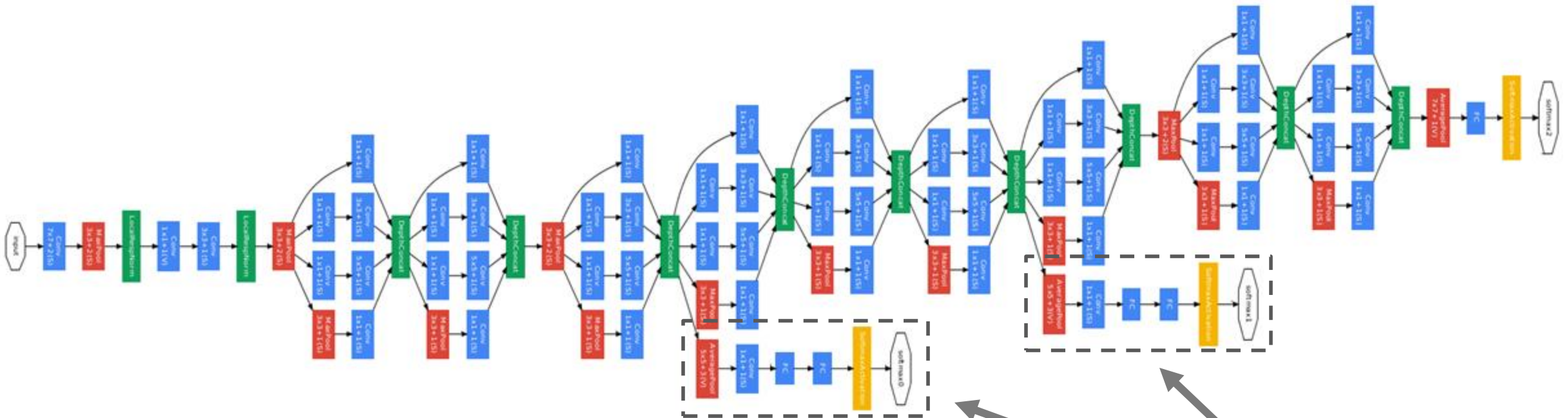


Agrupamento
pela Média
(*average pooling*)

Redução params

ConvNets – GoogLeNet

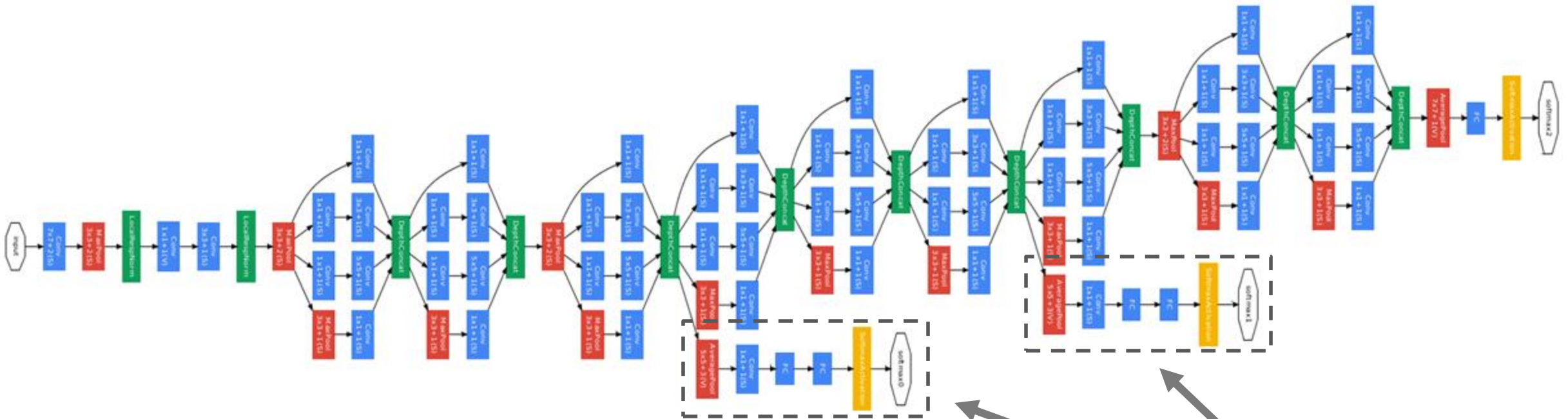
[Szegedy et al., 2014]



Classificadores
auxiliares

ConvNets – GoogLeNet

[Szegedy et al., 2014]



Classificadores
auxiliares

Facilita treinamento

ConvNets – GoogLeNet

[Szegedy et al., 2014]

type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
convolution	7×7/2	112×112×64	1							2.7K	34M
max pool	3×3/2	56×56×64	0								
convolution	3×3/1	56×56×192	2		64	192				112K	360M
max pool	3×3/2	28×28×192	0								
inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480	0								
inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832	0								
inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024	0								
dropout (40%)		1×1×1024	0								
linear		1×1×1000	1							1000K	1M
softmax		1×1×1000	0								

Vencedora ILSVRC 2014 – 6,7% de erro (top 5)

ConvNets – GoogLeNet

[Szegedy et al., 2014]

type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
convolution	7×7/2	112×112×64	1							2.7K	34M
max pool	3×3/2	56×56×64	0								
convolution	3×3/1	56×56×192	2		64	192				112K	360M
max pool	3×3/2	28×28×192	0								
inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480	0								
inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832	0								
inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024	0								
dropout (40%)		1×1×1024	0								
linear		1×1×1000	1							1000K	1M
softmax		1×1×1000	0								

Detalhes:

- Apenas $\approx 5\text{M}$ params!
(Remoção completa das camadas FC)

Vencedora ILSVRC 2014 – 6,7% de erro (top 5)

ConvNets – GoogLeNet

[Szegedy et al., 2014]

type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
convolution	7×7/2	112×112×64	1							2.7K	34M
max pool	3×3/2	56×56×64	0								
convolution	3×3/1	56×56×192	2		64	192				112K	360M
max pool	3×3/2	28×28×192	0								
inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480	0								
inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832	0								
inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024	0								
dropout (40%)		1×1×1024	0								
linear		1×1×1000	1							1000K	1M
softmax		1×1×1000	0								

Detalhes:

- Apenas $\approx 5\text{M}$ params!
(Remoção completa das camadas FC)

Comparada a AlexNet:

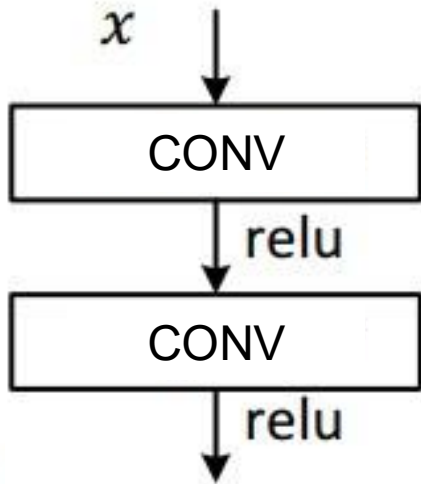
- 12× menos params
- 6,67% de erro (vs. 16,4%)

Vencedora ILSVRC 2014 – 6,7% de erro (top 5)

ConvNets – ResNet

[He et al., 2015]

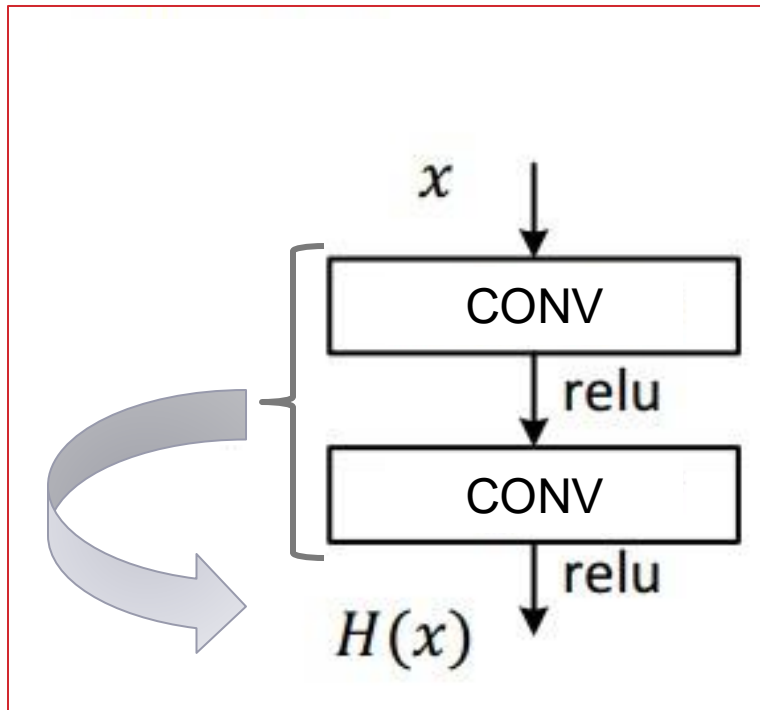
Rede Tradicional



ConvNets – ResNet

[He et al., 2015]

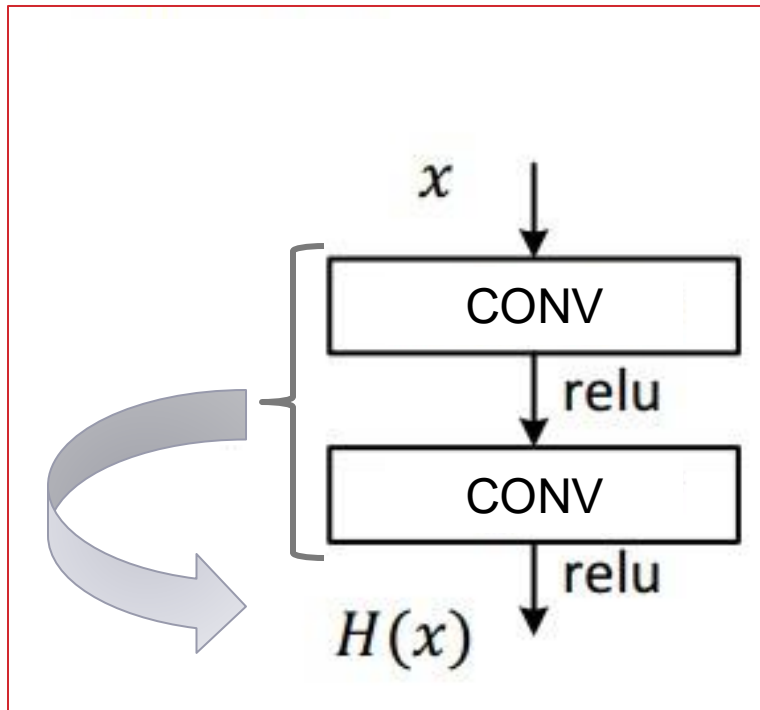
Rede Tradicional



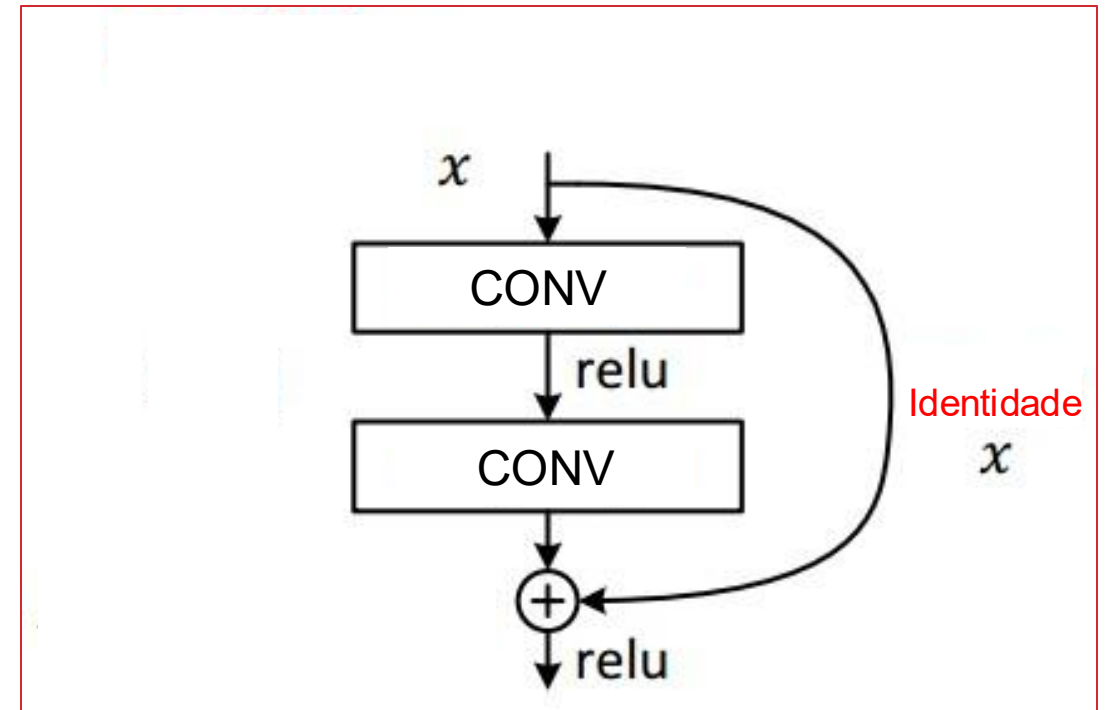
ConvNets – ResNet

[He et al., 2015]

Rede Tradicional



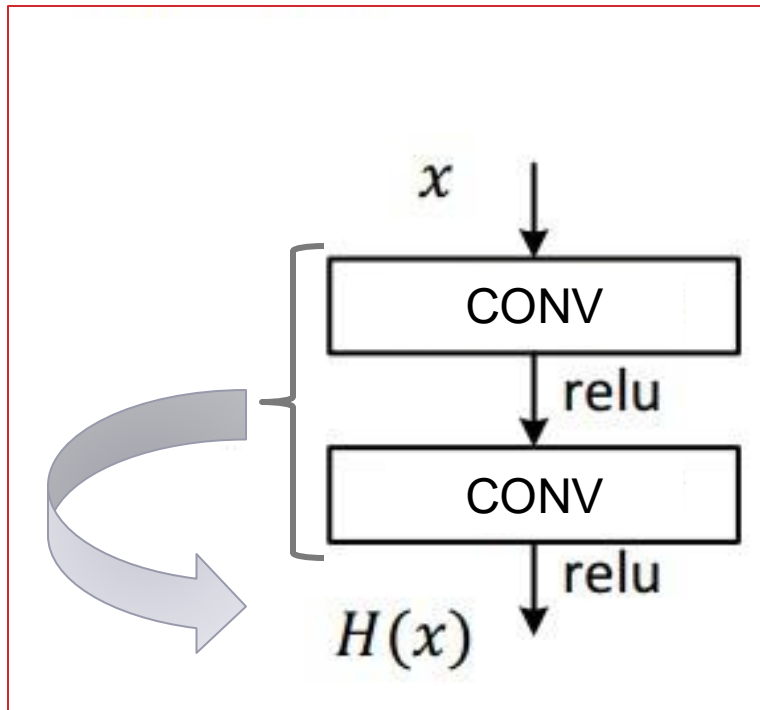
Rede Residual



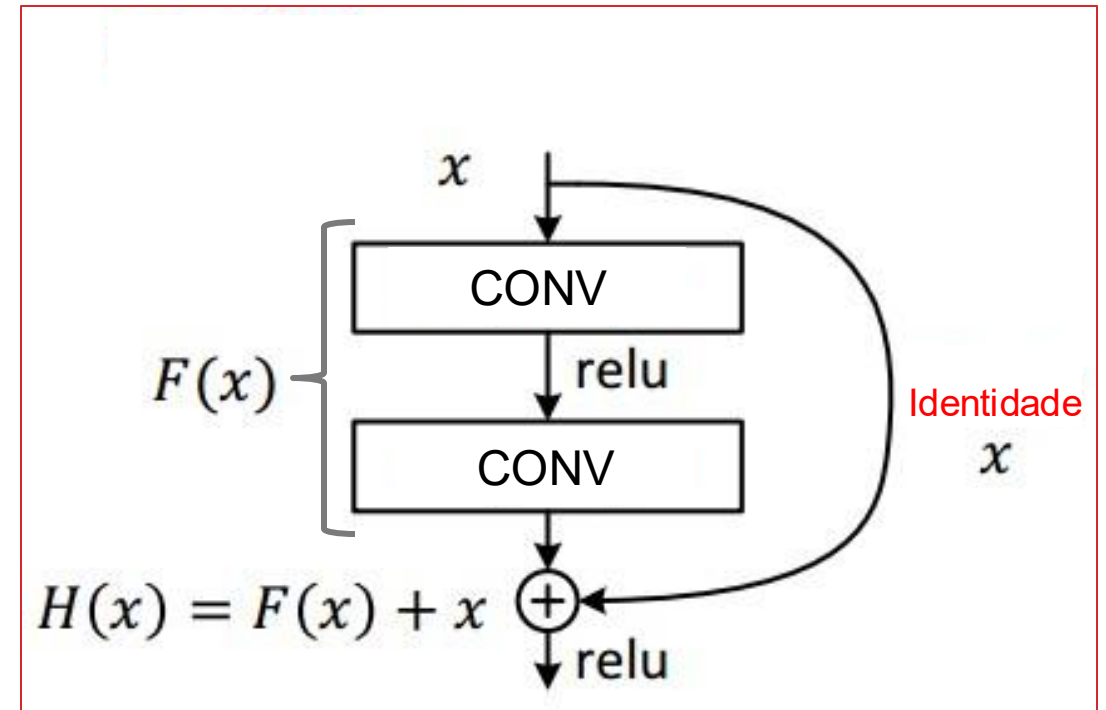
ConvNets – ResNet

[He et al., 2015]

Rede Tradicional



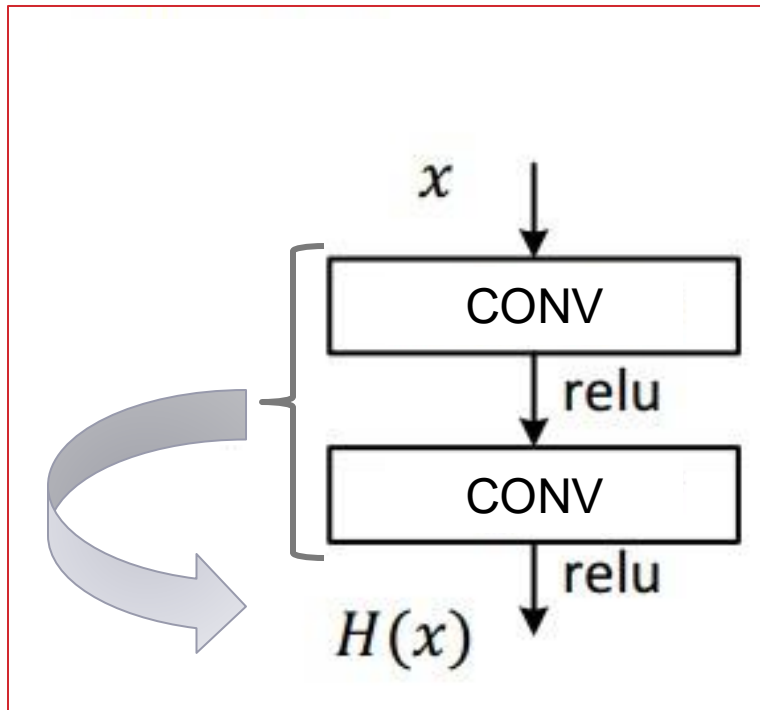
Rede Residual



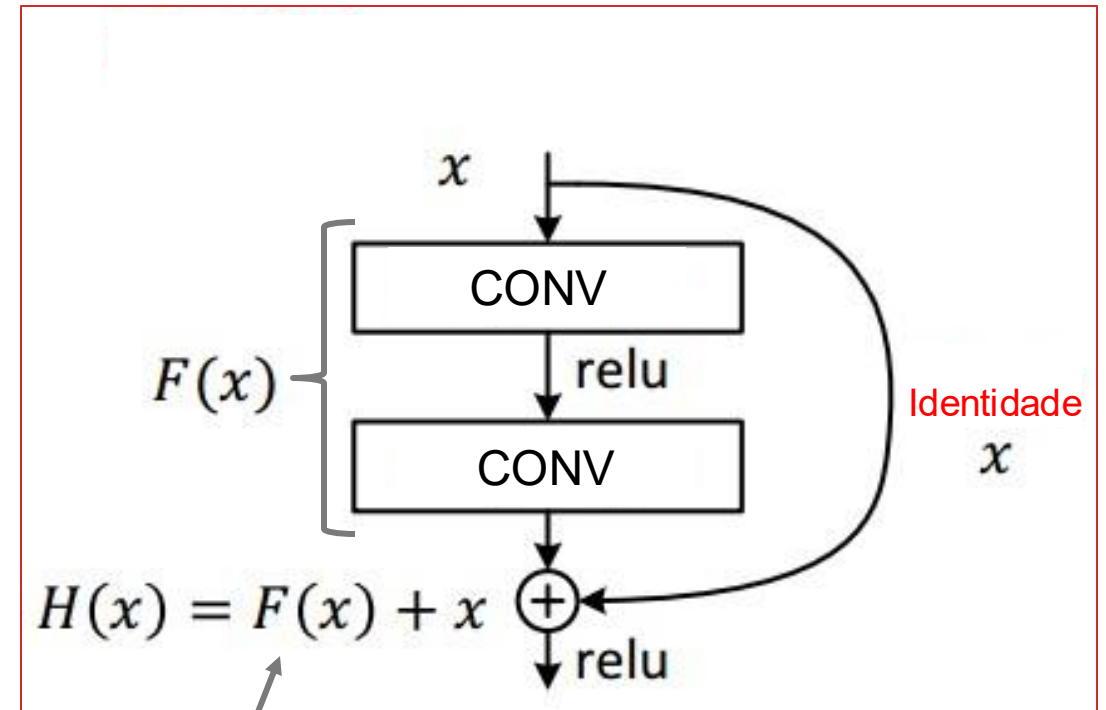
ConvNets – ResNet

[He et al., 2015]

Rede Tradicional



Rede Residual



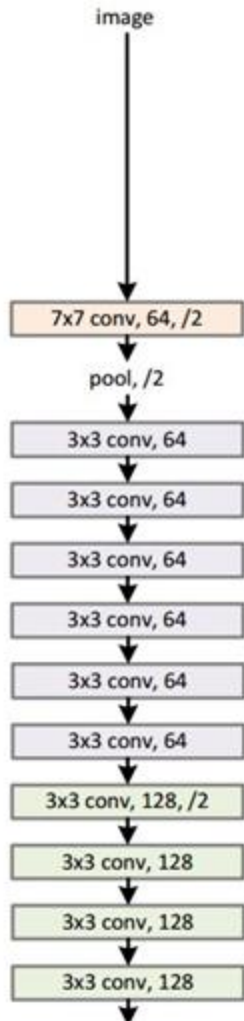
Resíduo

$$F(x) = H(x) - x$$

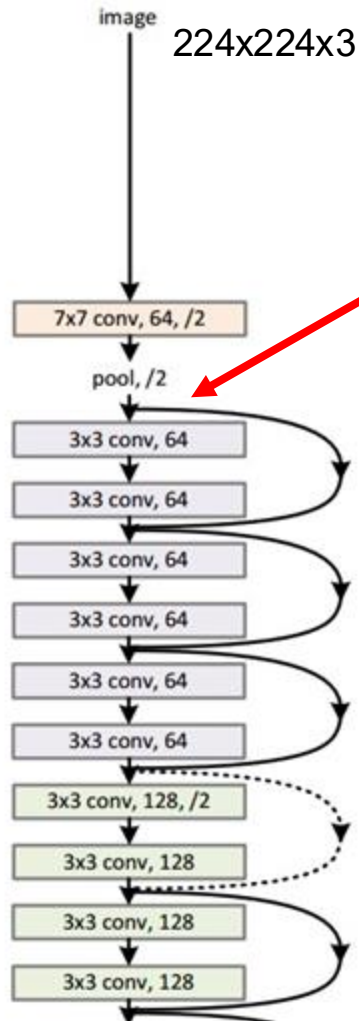
ConvNets – ResNet

[He et al., 2015]

34-layer plain



34-layer residual

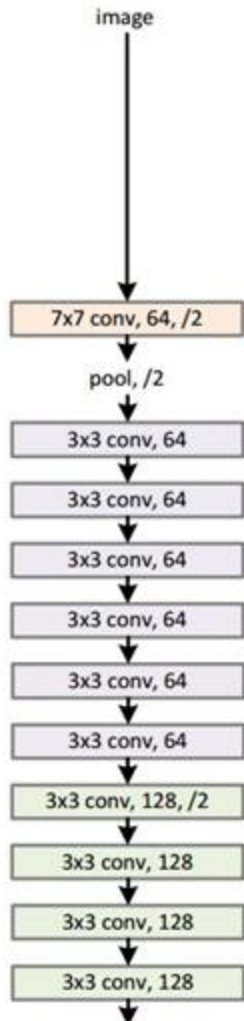


Dimensão espacial
apenas 56×56!

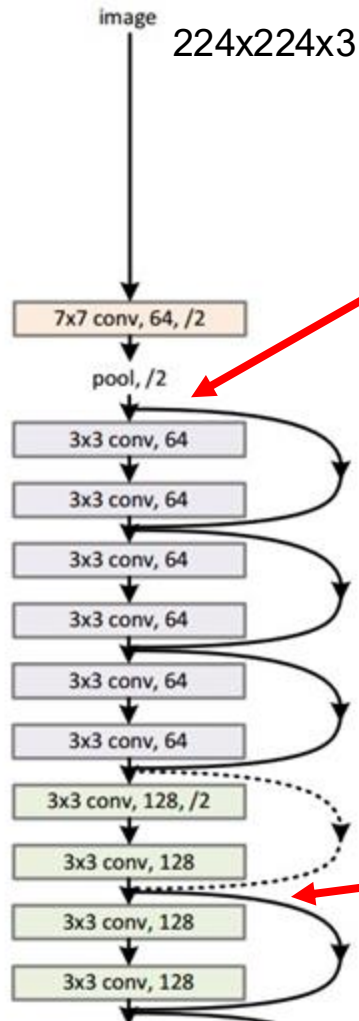
ConvNets – ResNet

[He et al., 2015]

34-layer plain



34-layer residual



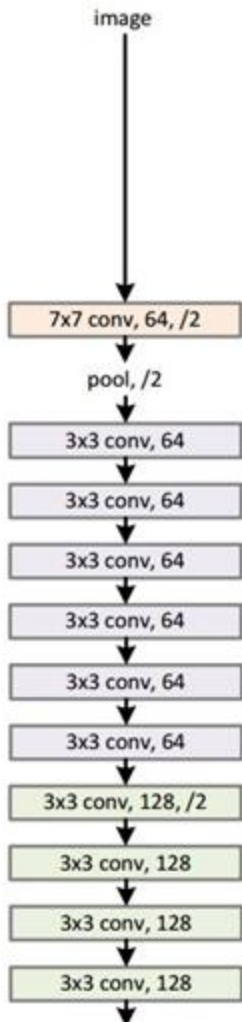
Dimensão espacial
apenas 56×56!

Dimensão espacial
apenas 28×28!

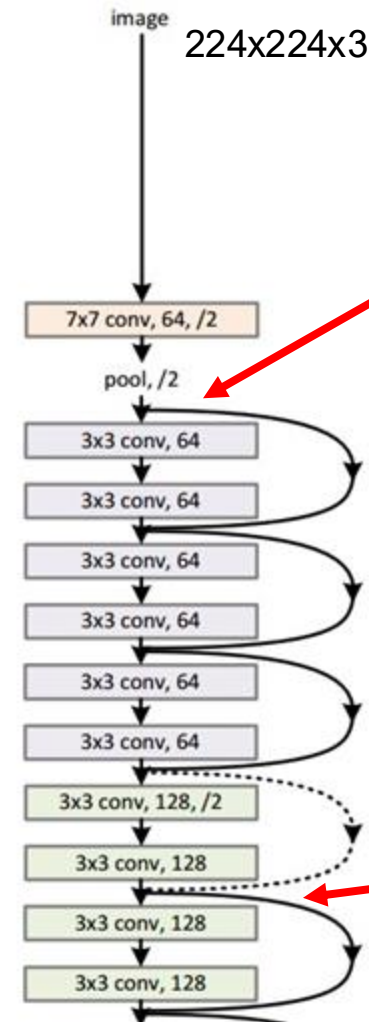
ConvNets – ResNet

[He et al., 2015]

34-layer plain



34-layer residual



Dimensão espacial apenas 56×56!

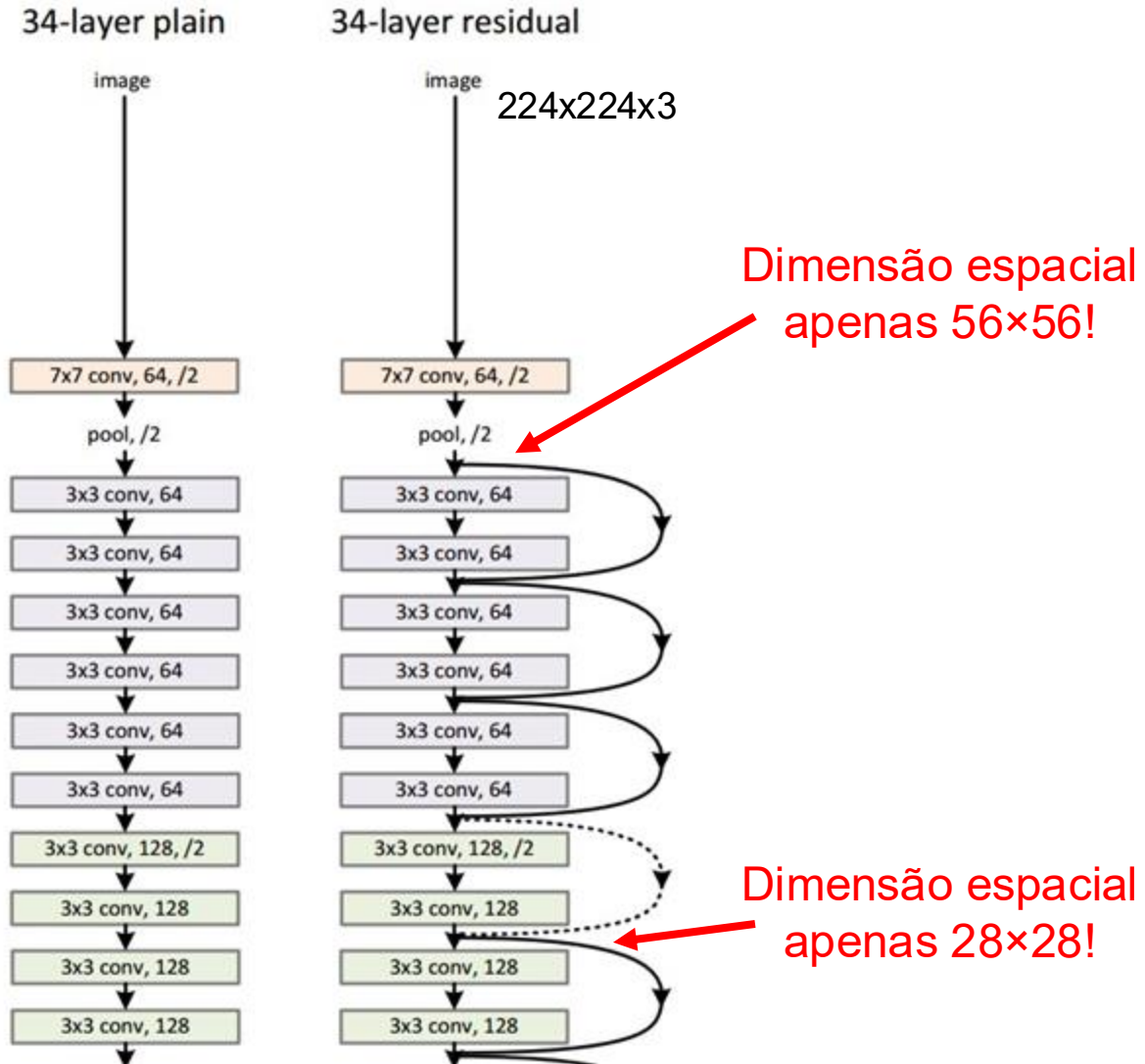
Dimensão espacial apenas 28×28!

Detalhes:

- Normalização em lote após cada CONV

ConvNets – ResNet

[He et al., 2015]

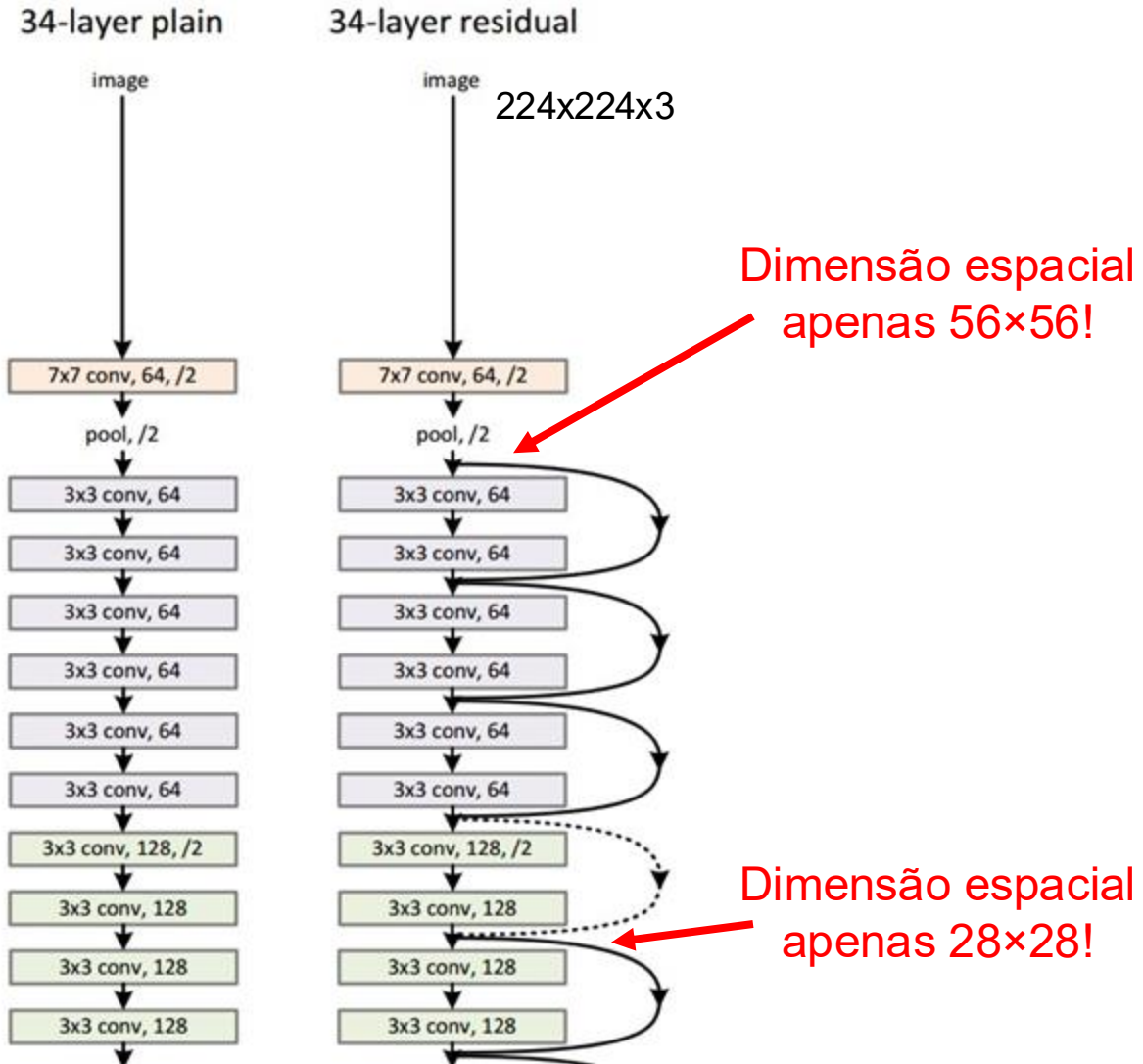


Detalhes:

- Normalização em lote após cada CONV
- Inicialização modificada (Xavier + /2)

ConvNets – ResNet

[He et al., 2015]

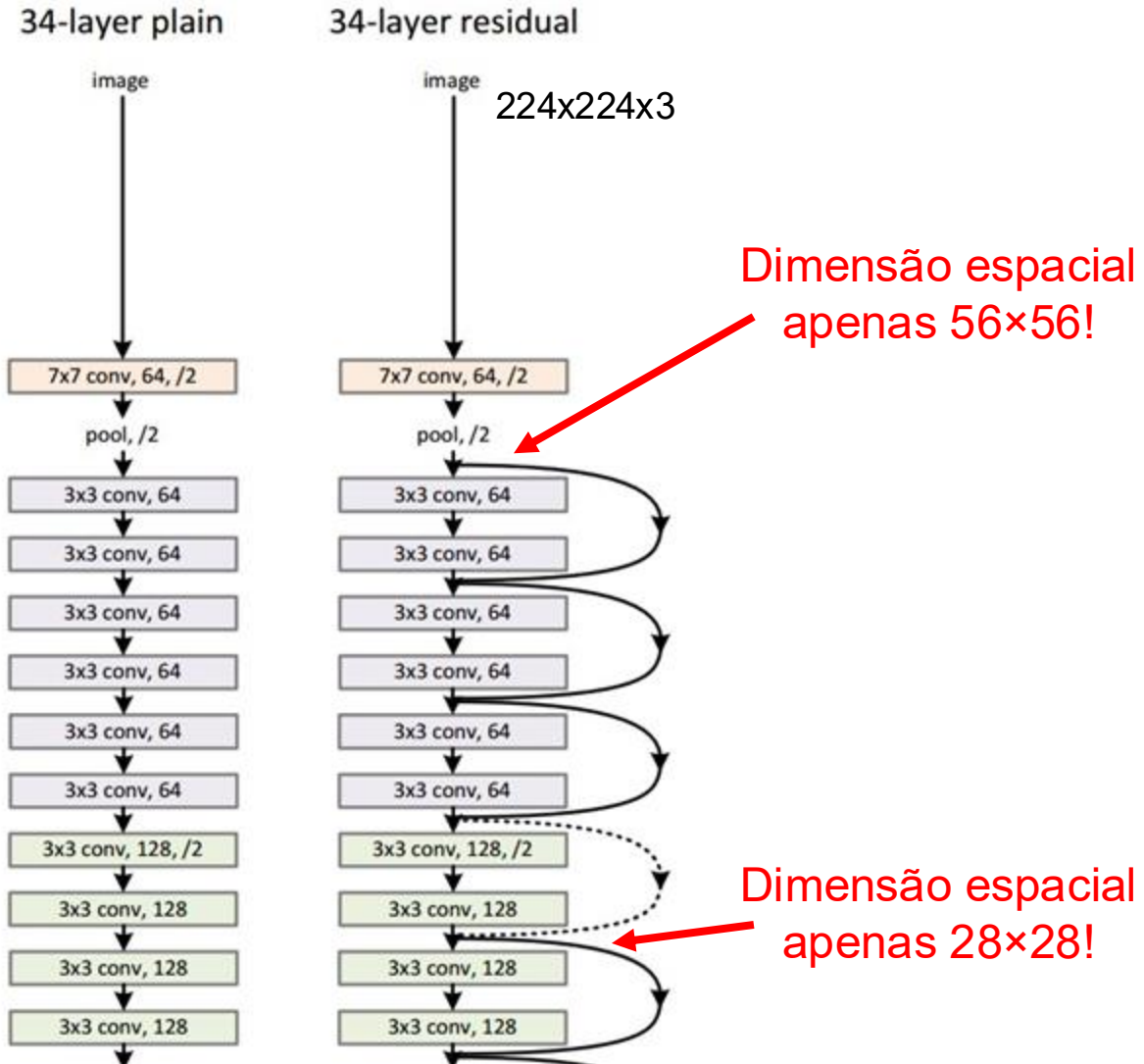


Detalhes:

- Normalização em lote após cada CONV
- Inicialização modificada (Xavier + /2)
- SGD+*Momentum* (0,9)

ConvNets – ResNet

[He et al., 2015]

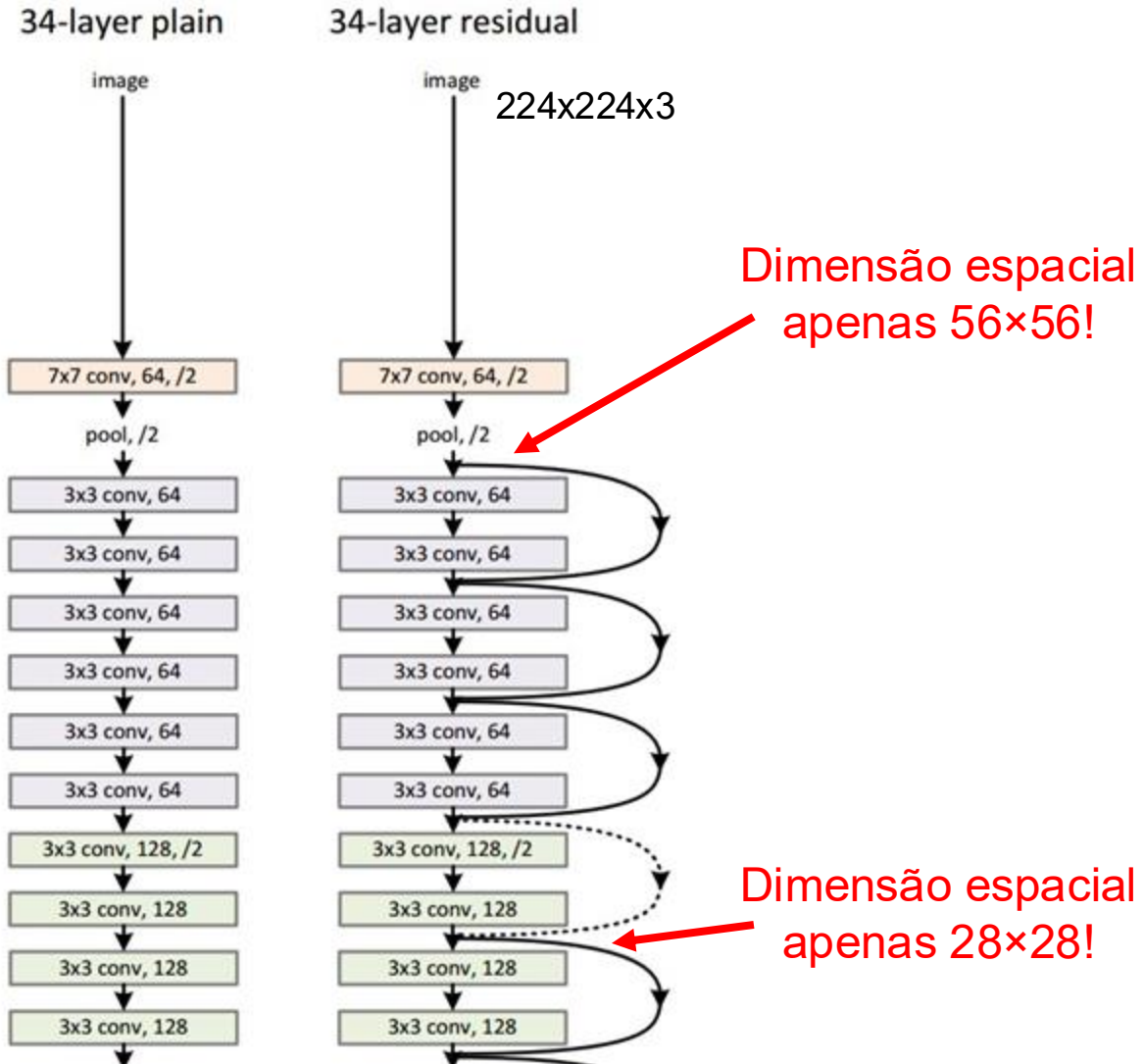


Detalhes:

- Normalização em lote após cada CONV
- Inicialização modificada (Xavier + /2)
- SGD+*Momentum* (0,9)
- Tx aprendizado = 10^{-1} , dividida por 10 qdo erro de validação para de reduzir

ConvNets – ResNet

[He et al., 2015]

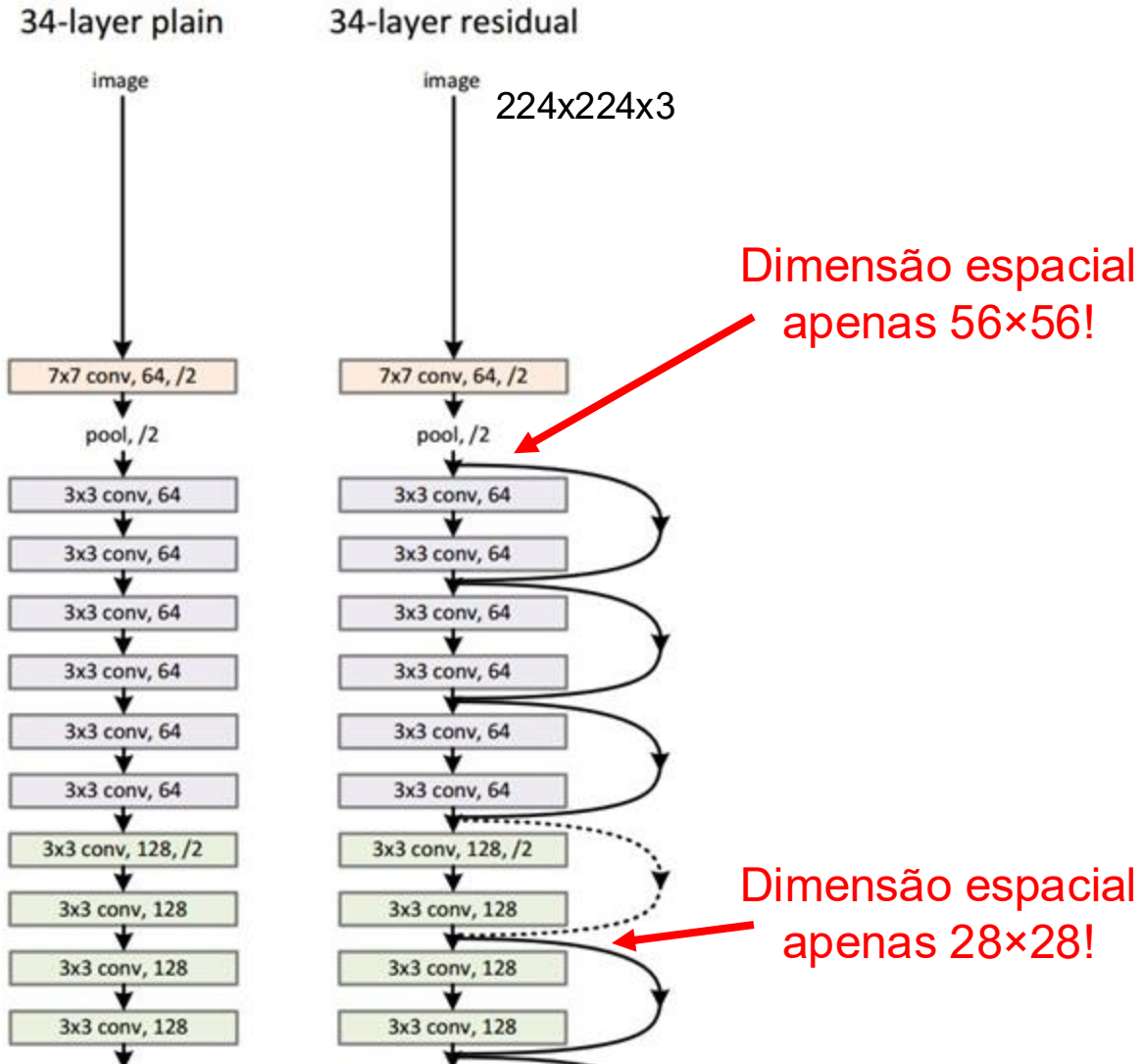


Detalhes:

- Normalização em lote após cada CONV
- Inicialização modificada (Xavier + /2)
- SGD+*Momentum* (0,9)
- Tx aprendizado = 10^{-1} , dividida por 10 qdo erro de validação para de reduzir
- Tamanho do *minibatch* = 256

ConvNets – ResNet

[He et al., 2015]

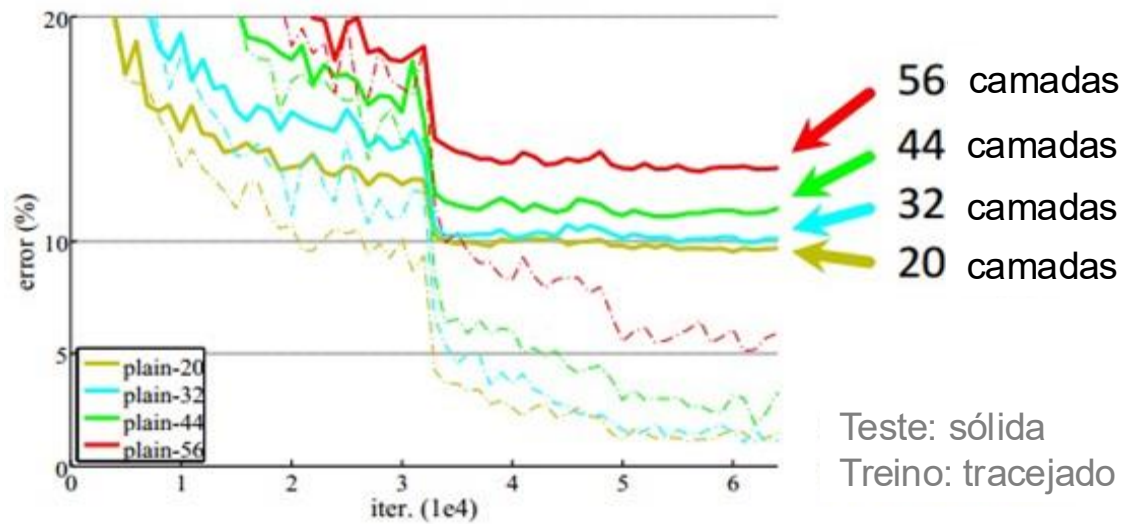


Detalhes:

- Normalização em lote após cada CONV
- Inicialização modificada (Xavier + /2)
- SGD+*Momentum* (0,9)
- Tx aprendizado = 10^{-1} , dividida por 10 qdo erro de validação para de reduzir
- Tamanho do *minibatch* = 256
- *Dropout* não é usado

ConvNets – ResNet

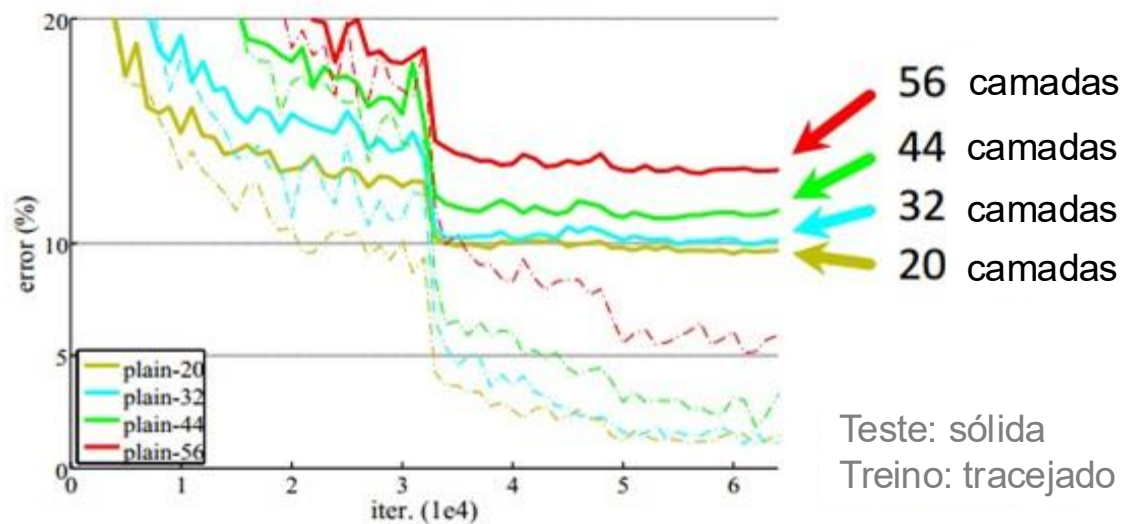
Experimentos com o conjunto de dados CIFAR-10



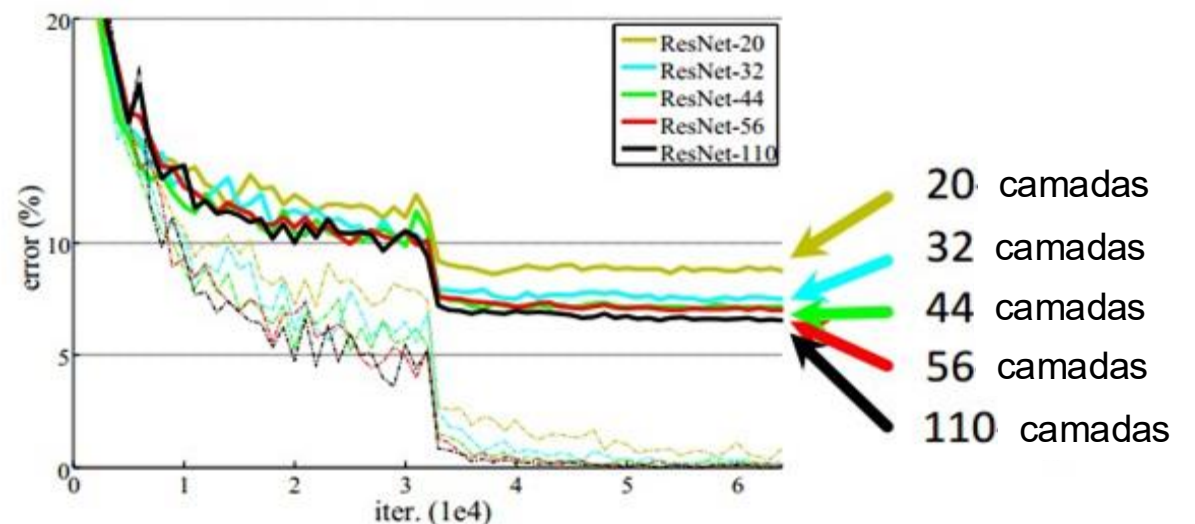
Rede Tradicional

ConvNets – ResNet

Experimentos com o conjunto de dados CIFAR-10



Rede Tradicional

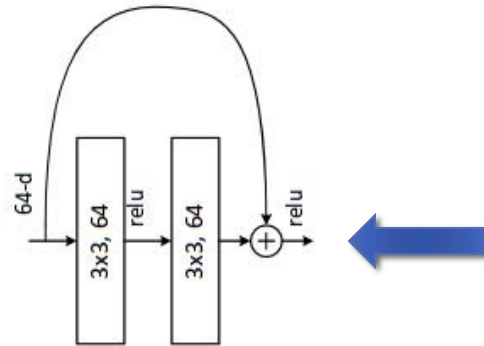


Rede Residual

ConvNets – ResNet

[He et al., 2015]

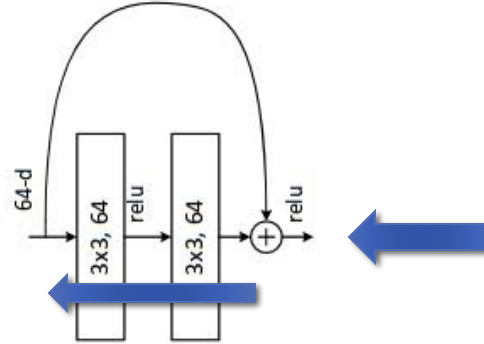
Fluxo de Gradientes



ConvNets – ResNet

[He et al., 2015]

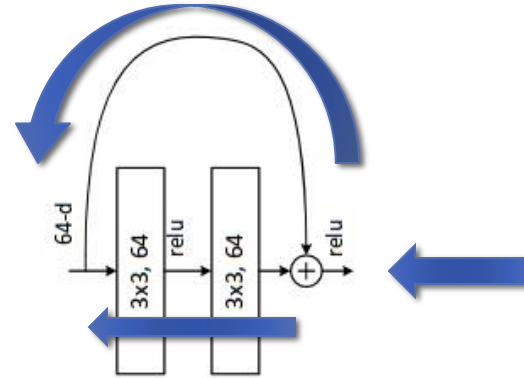
Fluxo de Gradientes



ConvNets – ResNet

[He et al., 2015]

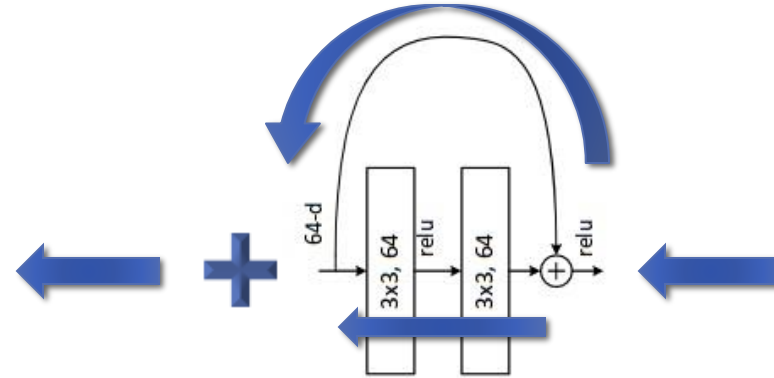
Fluxo de Gradientes



ConvNets – ResNet

[He et al., 2015]

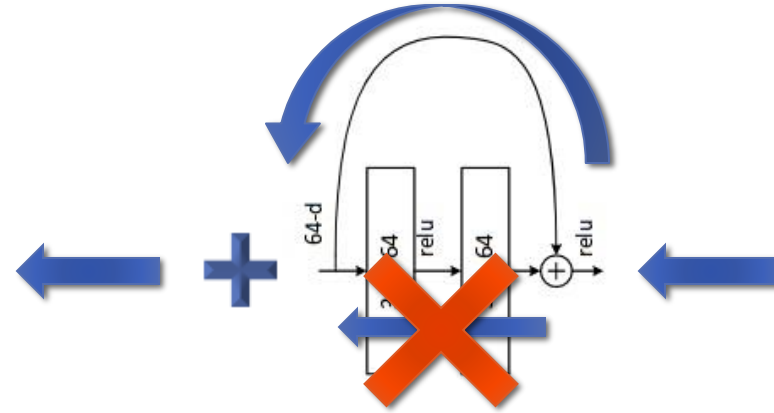
Fluxo de Gradientes



ConvNets – ResNet

[He et al., 2015]

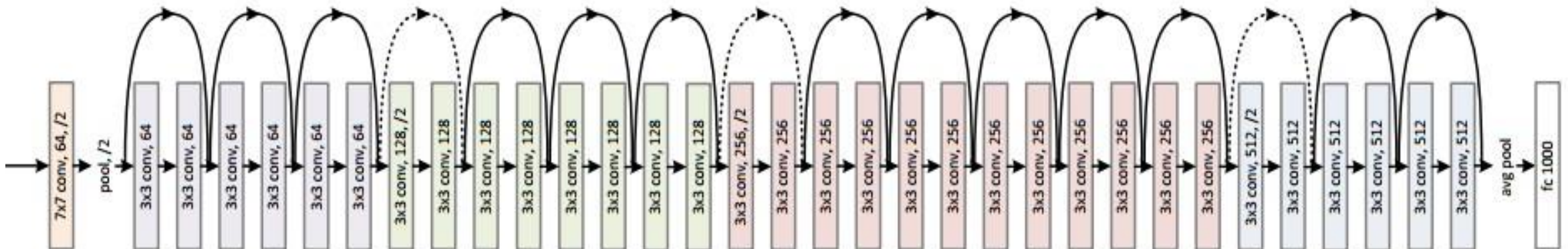
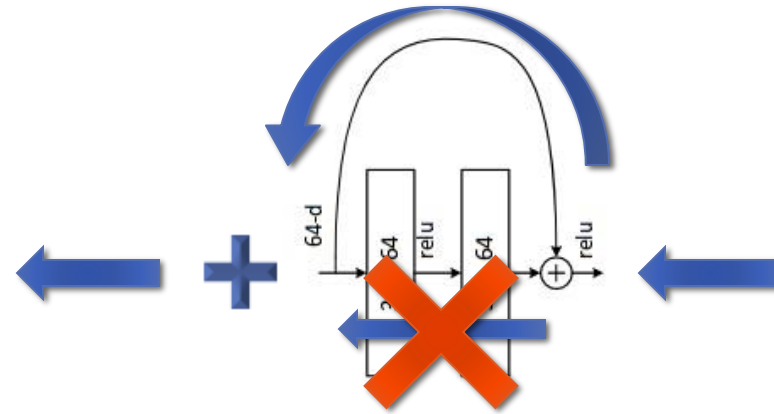
Fluxo de Gradientes



ConvNets – ResNet

[He et al., 2015]

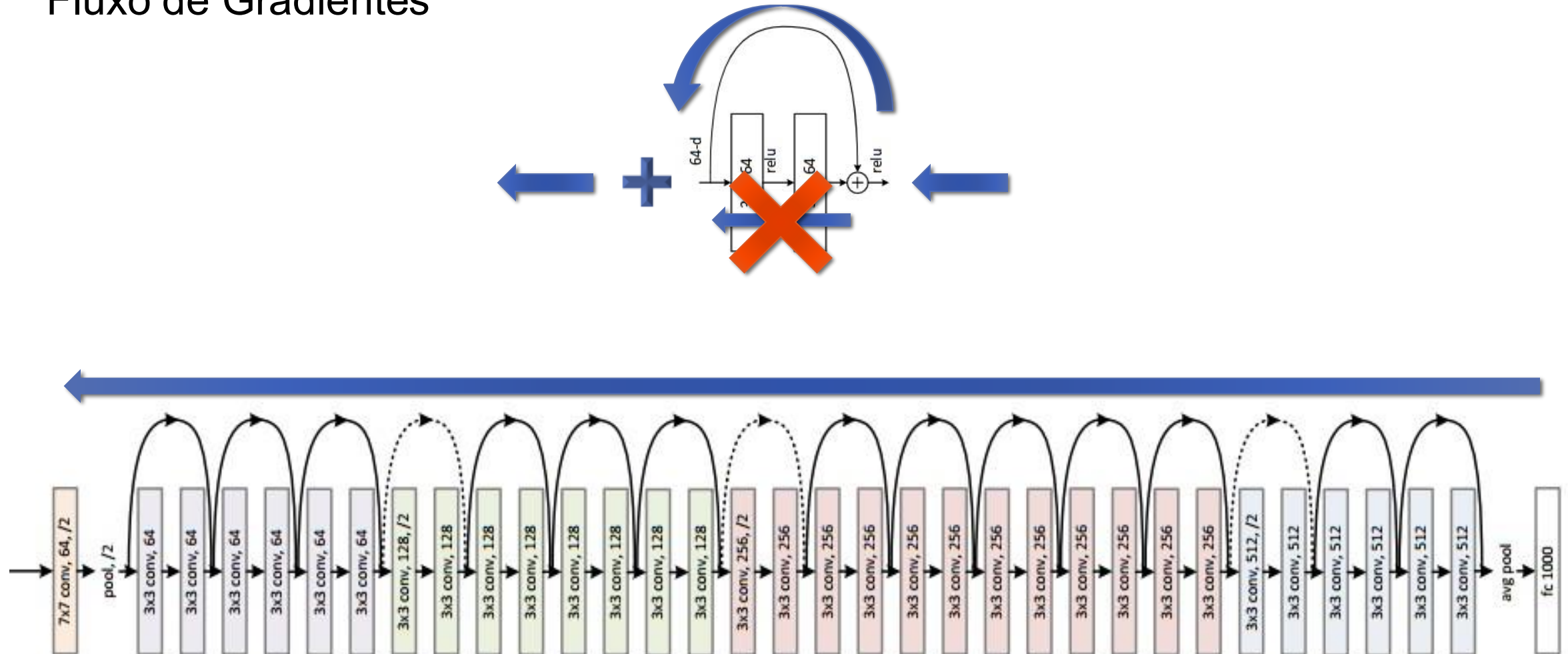
Fluxo de Gradientes



ConvNets – ResNet

[He et al., 2015]

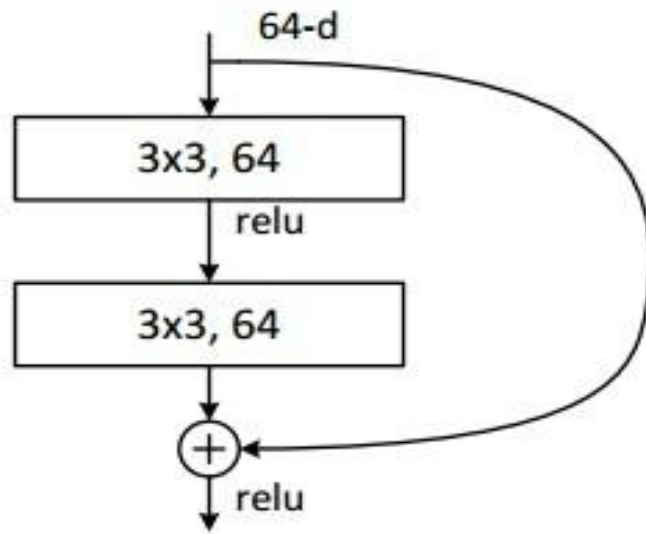
Fluxo de Gradientes



ConvNets – ResNet

[He et al., 2015]

Lidando com redes de 50+ camadas

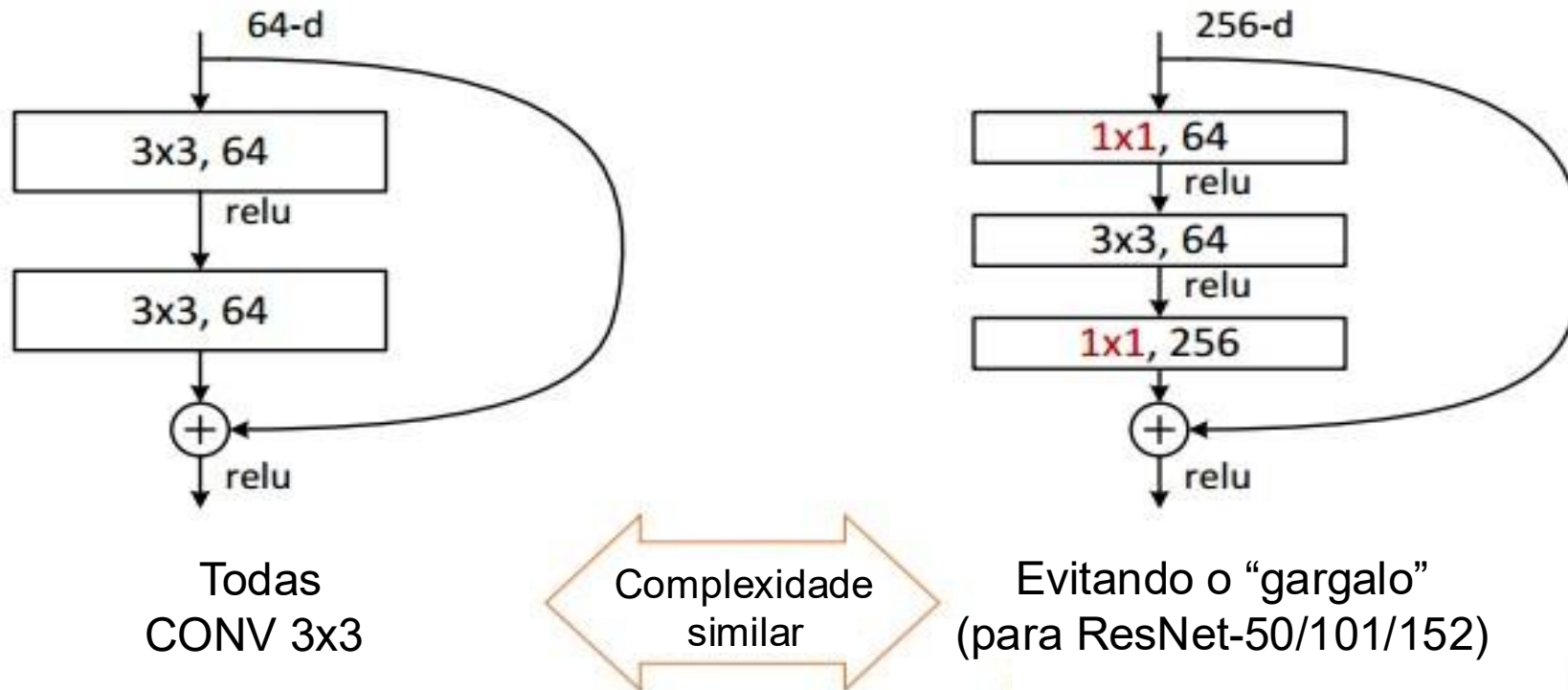


Todas
CONV 3x3

ConvNets – ResNet

[He et al., 2015]

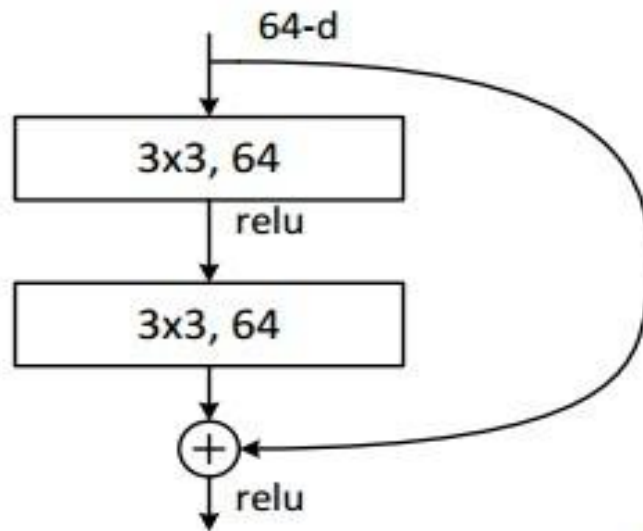
Lidando com redes de 50+ camadas



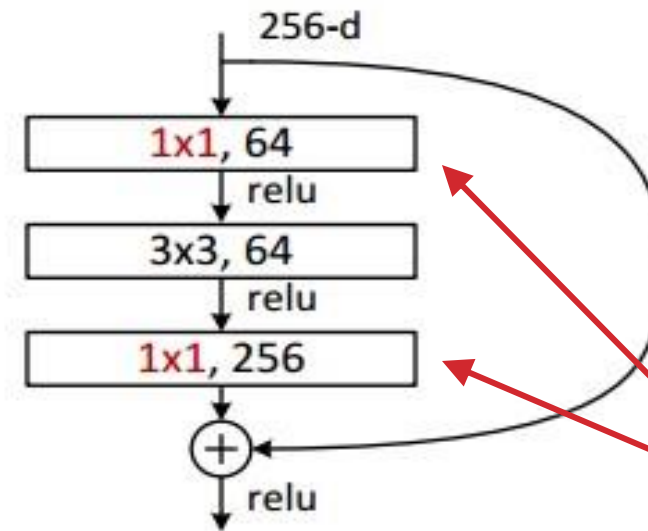
ConvNets – ResNet

[He et al., 2015]

Lidando com redes de 50+ camadas



Todas
CONV 3x3

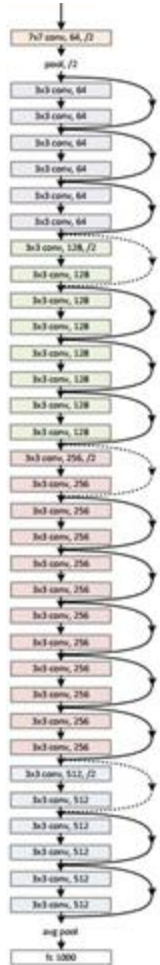


Truque similar
a GoogLeNet

Evitando o “gargalo”
(para ResNet-50/101/152)

ConvNets – ResNet

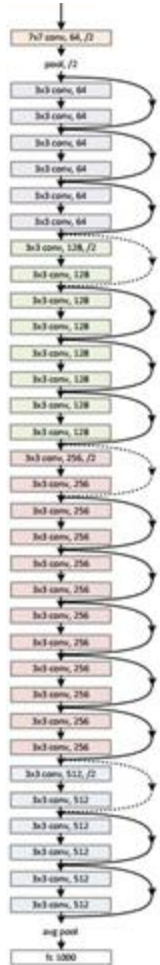
[He et al., 2015]



layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

ConvNets – ResNet

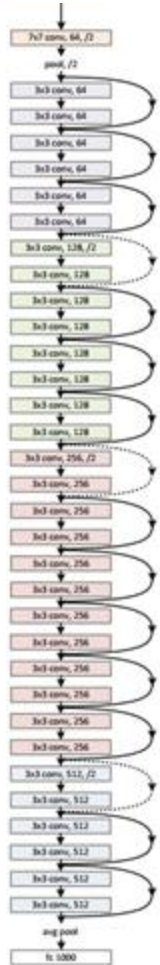
[He et al., 2015]



layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

ConvNets – ResNet

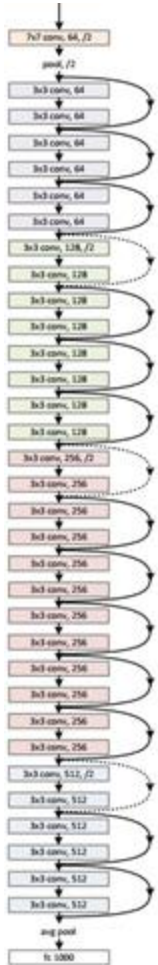
[He et al., 2015]



layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

ConvNets – ResNet

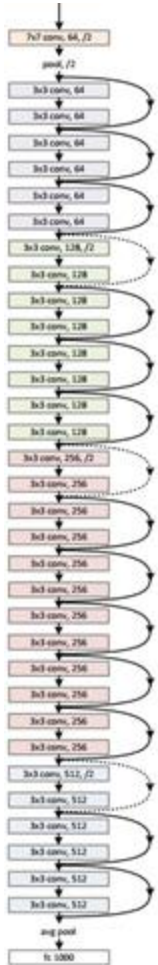
[He et al., 2015]



layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 23$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

ConvNets – ResNet

[He et al., 2015]

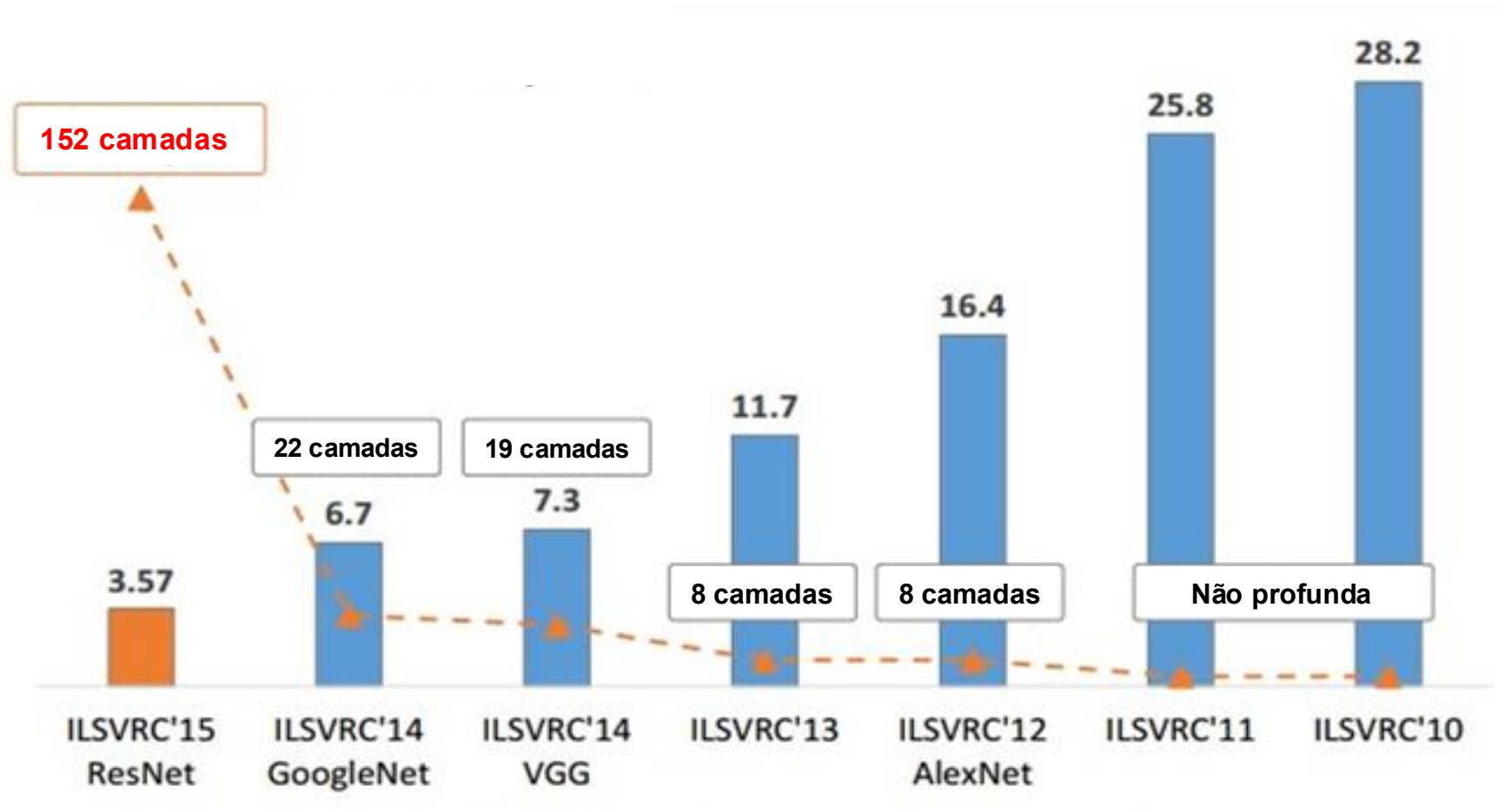


layer name	output size	18-layer	34-layer	50-layer	101-layer	152-layer
conv1	112×112	7×7, 64, stride 2				
conv2_x	56×56	3×3 max pool, stride 2				
		$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 64 \\ 3 \times 3, 64 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 64 \\ 3 \times 3, 64 \\ 1 \times 1, 256 \end{bmatrix} \times 3$
conv3_x	28×28	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 128 \\ 3 \times 3, 128 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 4$	$\begin{bmatrix} 1 \times 1, 128 \\ 3 \times 3, 128 \\ 1 \times 1, 512 \end{bmatrix} \times 8$
conv4_x	14×14	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 256 \\ 3 \times 3, 256 \end{bmatrix} \times 6$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 256 \\ 3 \times 3, 256 \\ 1 \times 1, 1024 \end{bmatrix} \times 36$
conv5_x	7×7	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 2$	$\begin{bmatrix} 3 \times 3, 512 \\ 3 \times 3, 512 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$	$\begin{bmatrix} 1 \times 1, 512 \\ 3 \times 3, 512 \\ 1 \times 1, 2048 \end{bmatrix} \times 3$
	1×1	average pool, 1000-d fc, softmax				
FLOPs		1.8×10^9	3.6×10^9	3.8×10^9	7.6×10^9	11.3×10^9

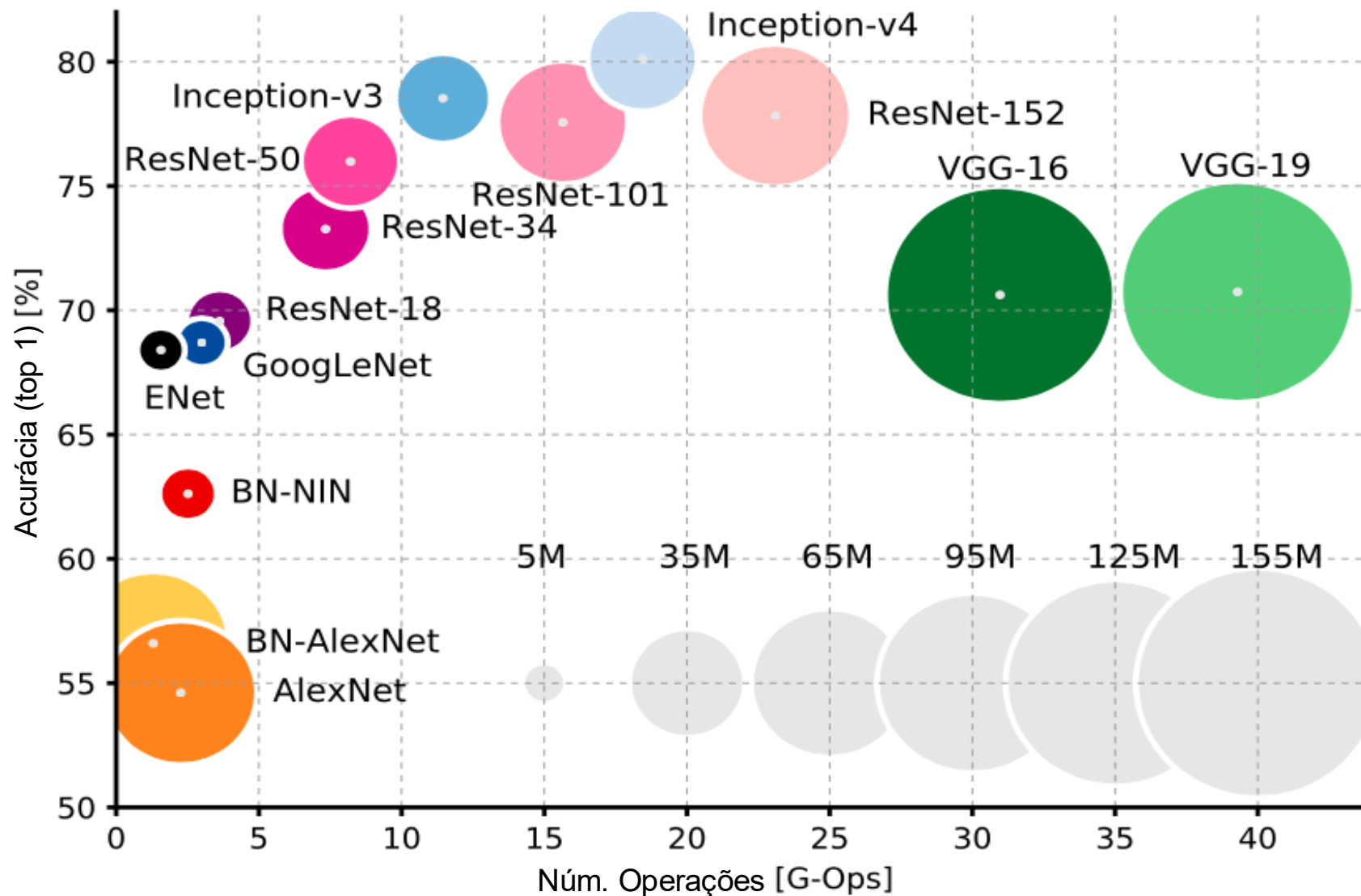
VGG16 – $15,3 \times 10^9$
VGG19 – $19,6 \times 10^9$

Vencedora ILSVRC 2015 – 3,6% de erro (top 5)

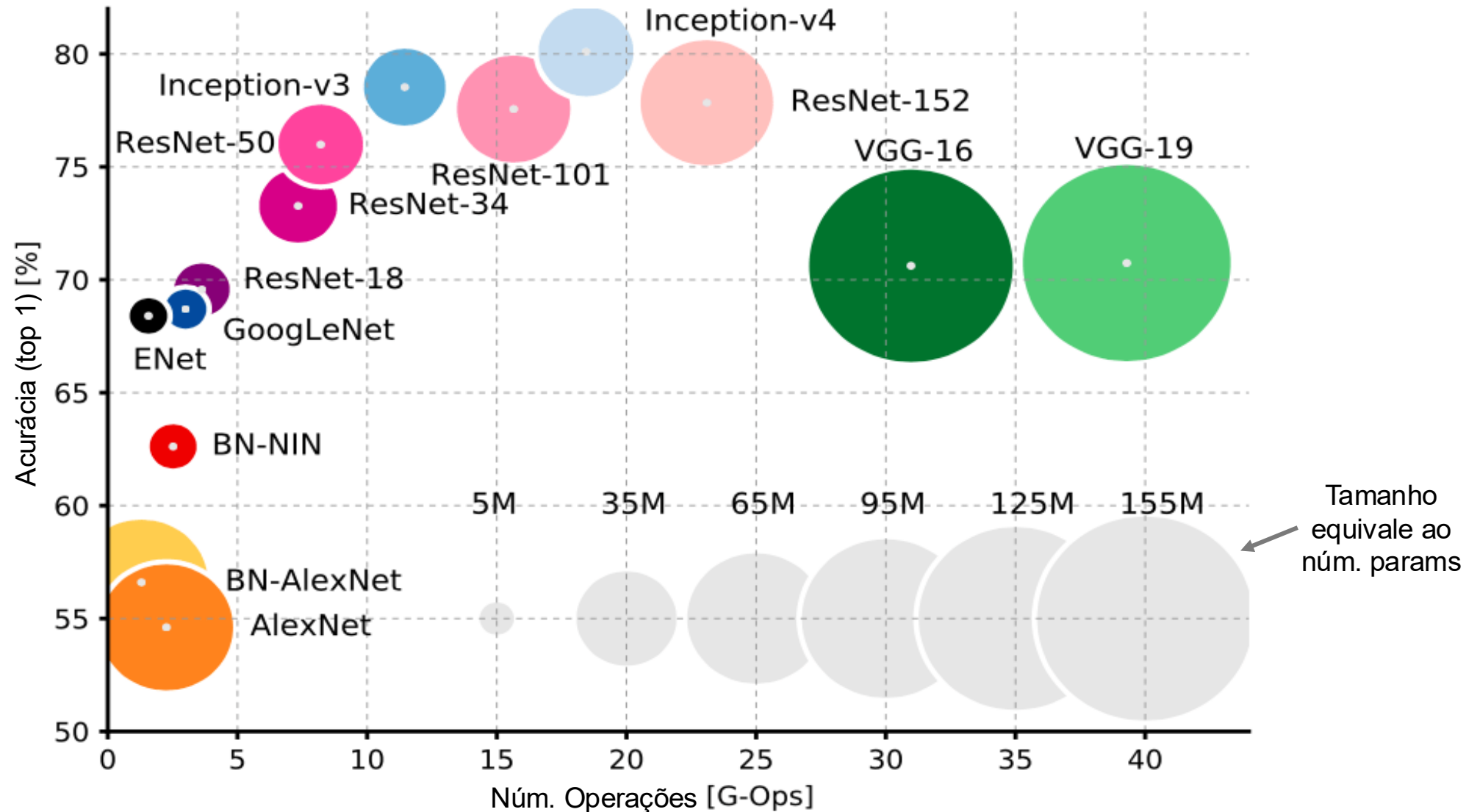
ConvNets – Análise Comparativa



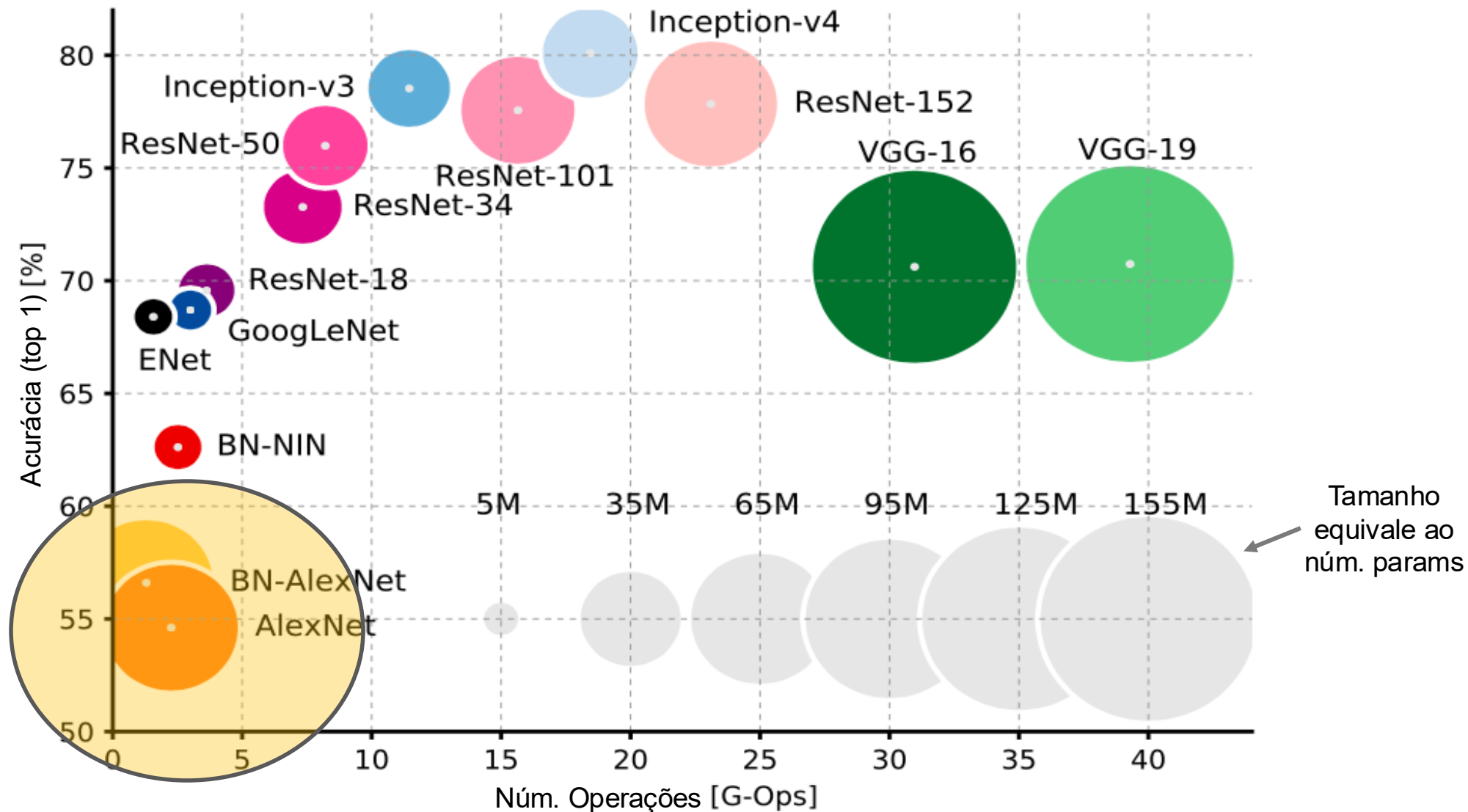
ConvNets – Análise Comparativa



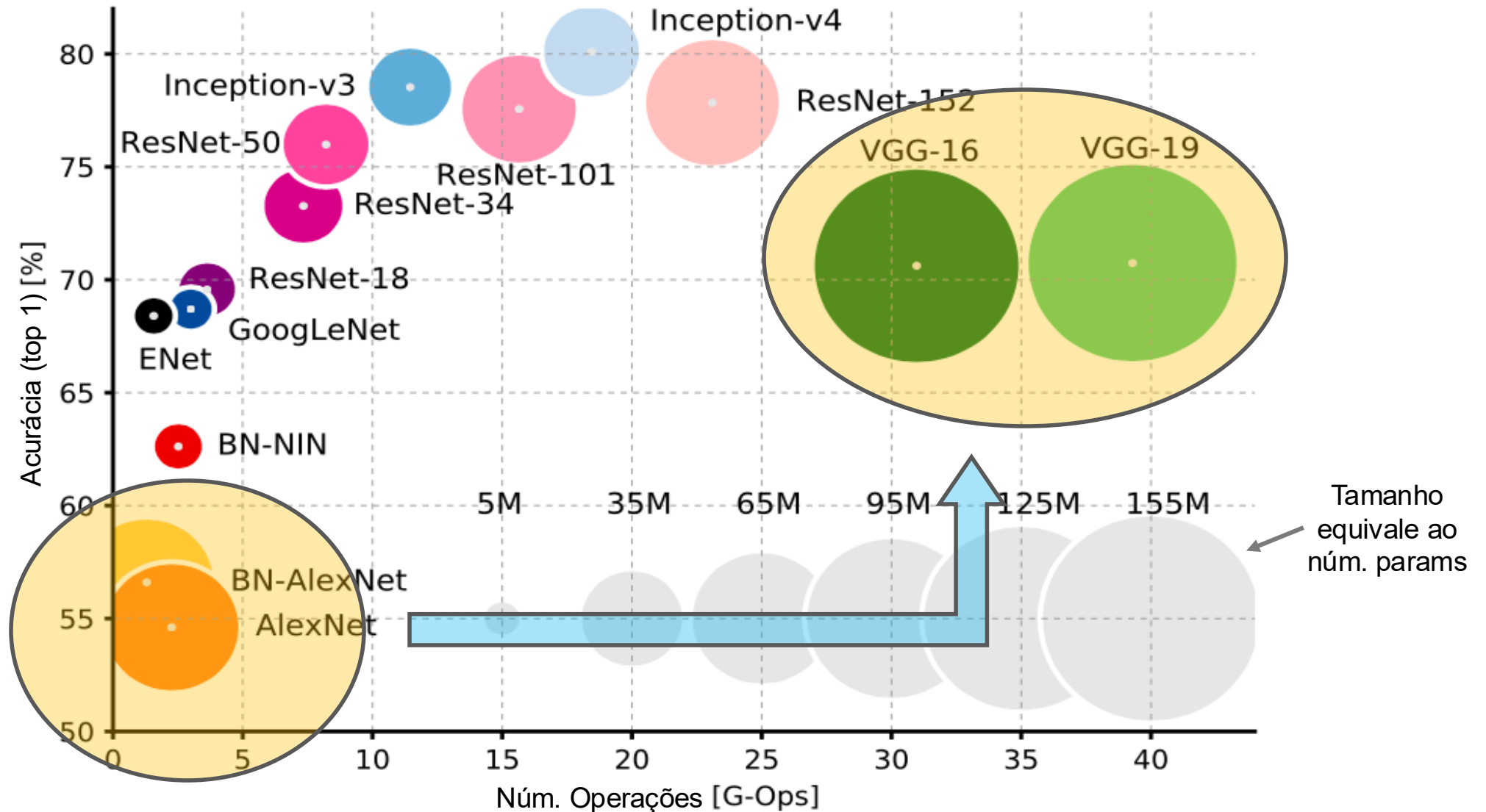
ConvNets – Análise Comparativa



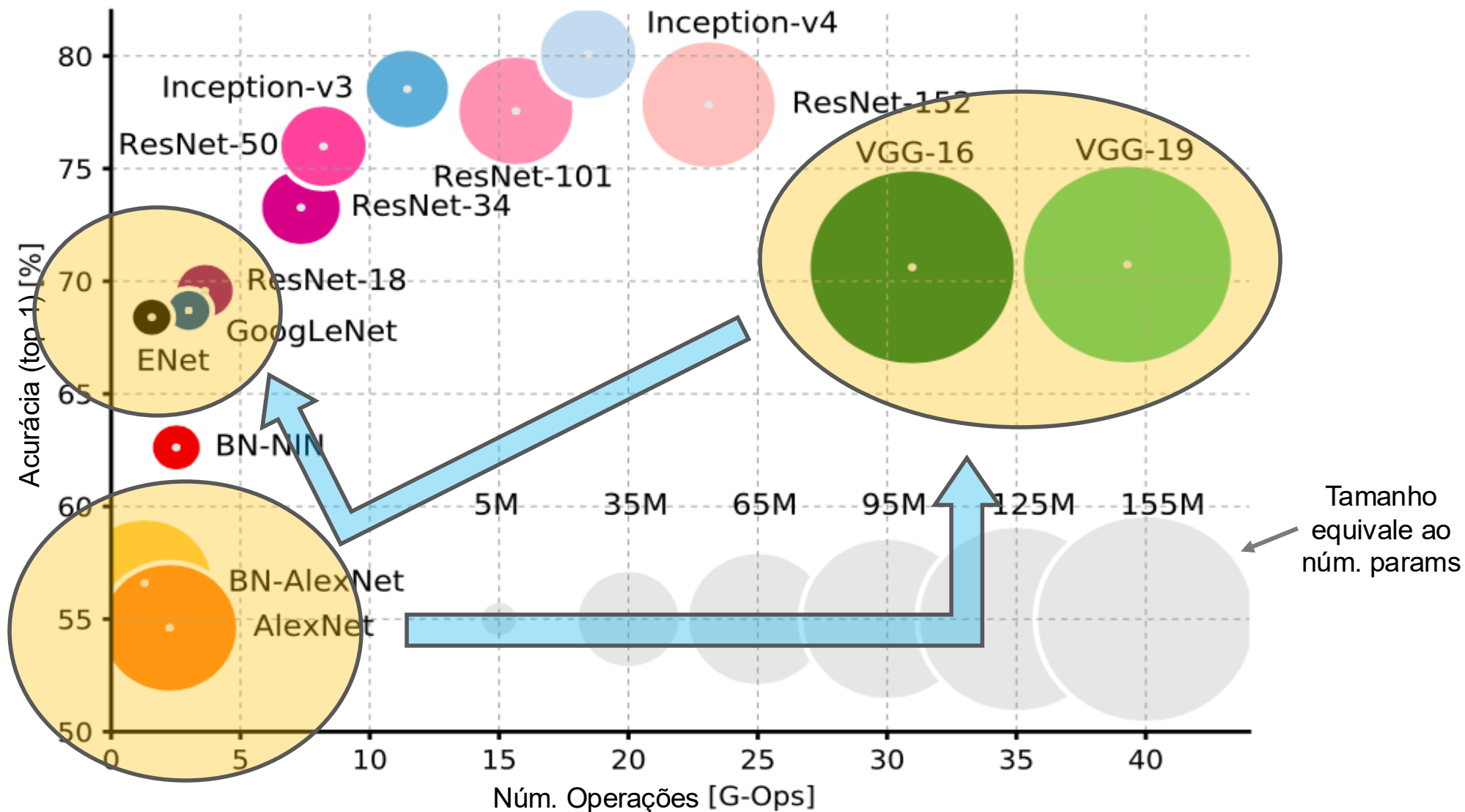
ConvNets – Análise Comparativa



ConvNets – Análise Comparativa



ConvNets – Análise Comparativa



ConvNets – Análise Comparativa

